Algoritma nedir? Sözde kod, Akış diyagramı

Mod alma algoritması gördük.

Asal sayı bulma algoritması gördük

Selection sort → karmaşıklığı $\frac{N^2}{2}$ ← karşılaştırma sayısı

N ← yer değiştirme

27.02.2023

Akış diyagramı - Pseudo Code

Pseudo code → Değişken tanımı yapmaya gerek yok, düzgün isimlendirme

Dizilim - Seçim - Döngü
↳ sıralı yazım ↳ for - while
↳ if

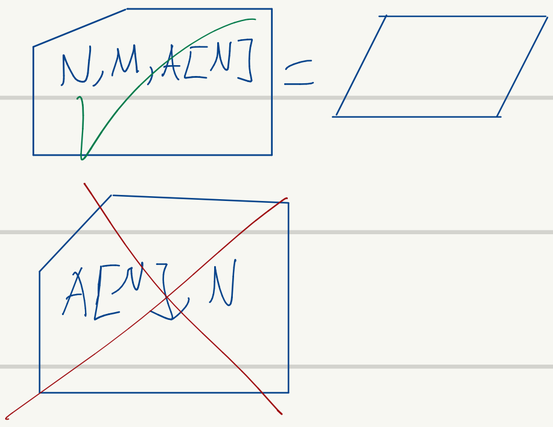
if ... end if

loop ... end loop

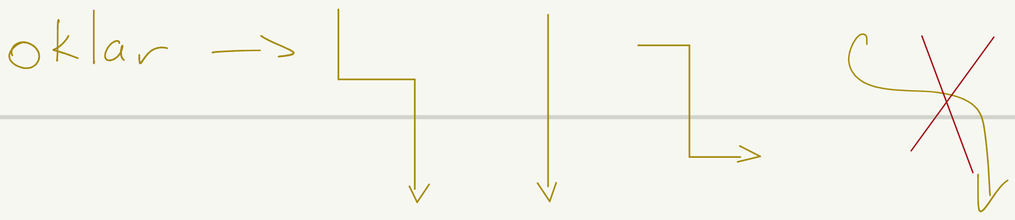
a=b → kontrol
a:=b or a←b atama
a[i] → indis

Akış diyagramı → START Stop 1 start 1 stop olmalı

1 yada 1'den fazla değer okunmalı



oklar →

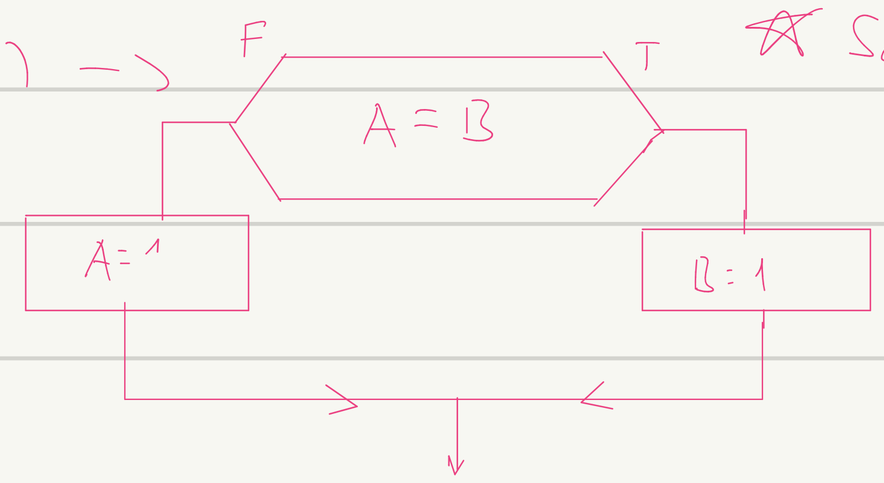


cemberler → 1 2

ifadeler → A=A+1

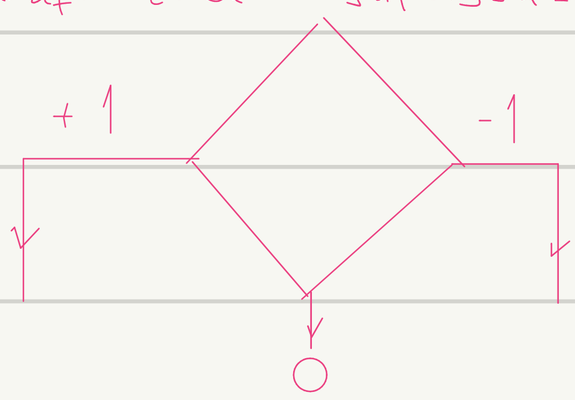
A++ → Bağımsız değil

karar (if) →



☆ Sağ tarafı true yaptıysak true hep sağda olsun

=

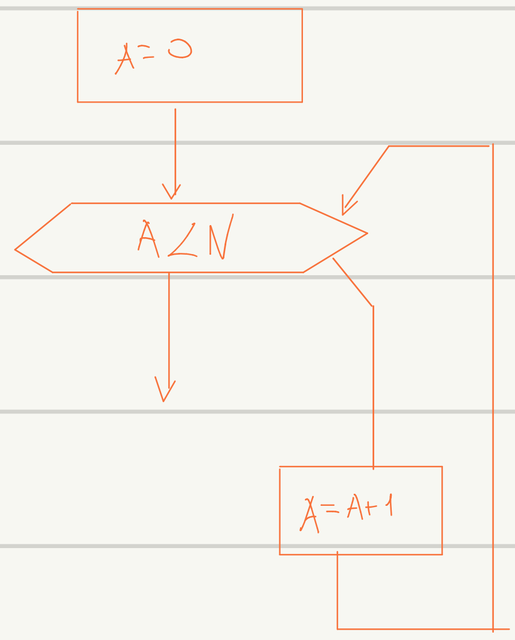


☆ True kolu boş olamaz

for döngüsü

opsiyonel

while döngüsü



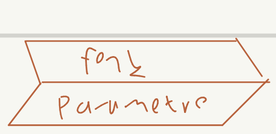
Bağlantı problemi →

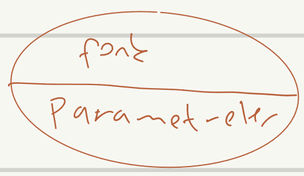
0 1 2 3 4 5 6 7 8 9

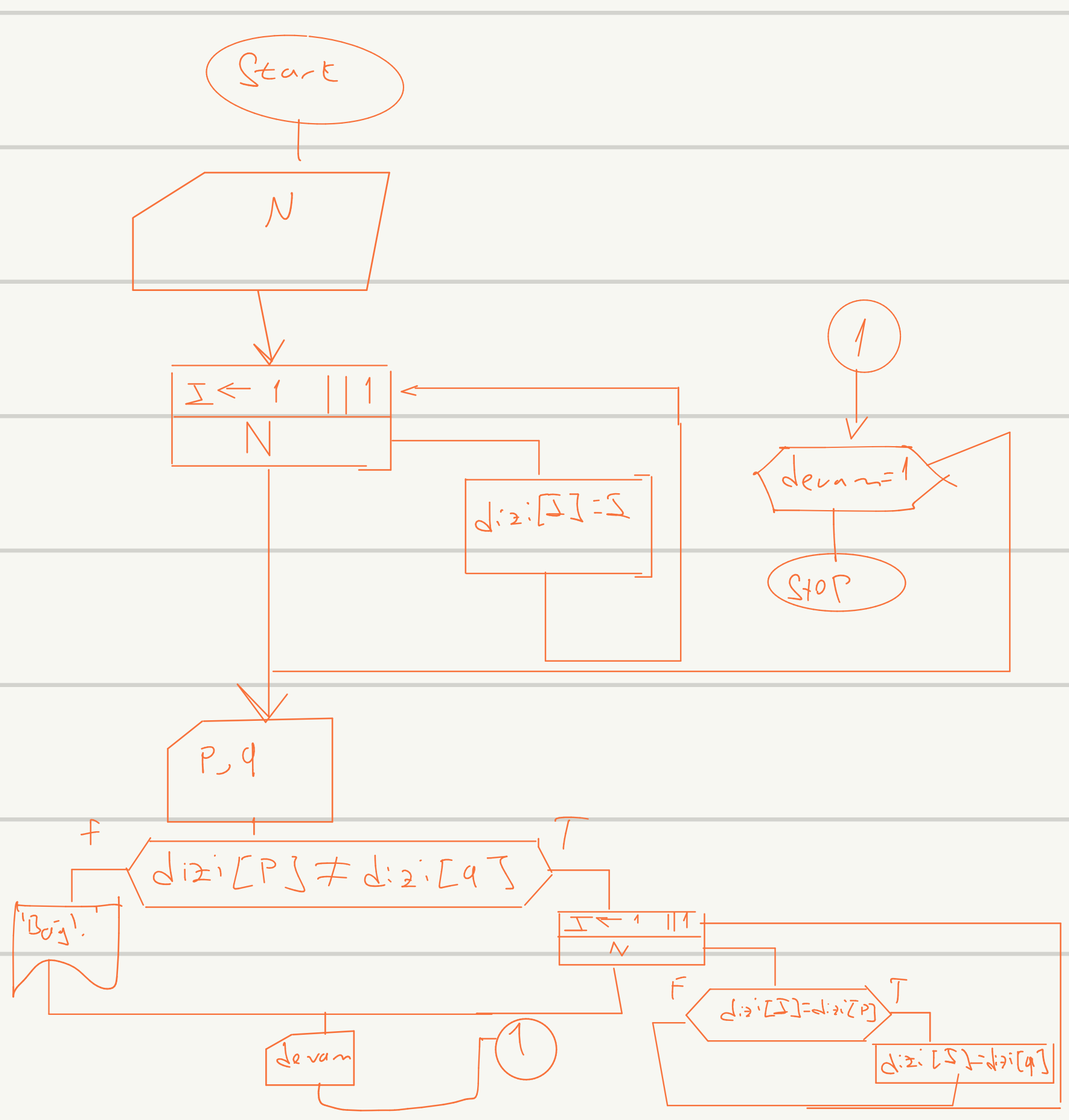
0 1 1 8 8 0 0 1 8 8 ← dizi

```
if ( dizi[1] != dizi[6] ) {
        count = dizi[6]
    for (i=0 ; i<N ; i++) {
      if( dizi[i] = count)
          dizi[i] = dizi[1] }
}
```

Fonksiyon →







09.03.2022
2.

Algoritma tasarım teknikleri

Kaba kuvvet (Brute-force) *selection sort

Açgözlü (Greedy) * Prim MST, Dijkstra en kısa yol

Böl ve yönet (Divide and conquer) * Merge sort, Azalt ve yönet, Değiştir ve yönet
* Insertion sort * Heap sort

Dinamik programlama *Lineer karmaşıklıkla fibonacci hesabı

Obeb (GCD)

gcd(m,n) = gcd(n, m mod n)

N'e kadar olan asal sayıları bul

forln dizi'yi gez eğer 0 değilse tüm katlarını 0'la

Performans karşılaştırma

Programı yaz ve çalıştır süresi ölç

- Teorik
- Empirik → Tüm algoritmaları kıyasla

→ Kodlama diline ve cihaza bağımlı. Sadece fikir verebilir. Kesin değildir.

→ Matematiksel model; çalışma ortamından veri tipinden, cihazdan bağımsız.

"İşlem sayısını belirler ve büyüme fonksiyonları kullanırız.

Algoritma işletim sistemi

```
count = count + 1      -> c1
sum = sum + count      -> c2

i = 1; sum = 0;        -> c1, -> c2
while (i <= n) {       -> c3 -> n+1
    i = i + 1;         -> c4 -> n
    sum = sum + i;     -> c5 -> n
}
```

```
if (n < 0)             -> c1
    absV = -n;         -> c2
else
    absV = n;          -> c3
```

c1 + max(c2, c3)

-> c1 + c2 + (n+1)*c3 + n*c4 + n*c5

Bu algoritmanın maliyeti n ile doğru orantılıdır.

```
i = 1; sum = 0;        -> c1, -> c2
while (i <= n) {       -> c3
    j = 1;             -> c4
    while (j <= n) {   -> c5
        sum = sum + i; -> c6
        j = j + 1;     -> c7
    }
    i = i + 1;         -> c8
}
```

C_1	1
C_2	1
C_3	n+1
C_4	n
C_5	n*(n+1)
C_6	n*n
C_7	n*n
C_8	n

-> n^2 ile doğru orantılıdır.

A algoritması $3n^2$ B algoritması $2n+5$

↓ Karmaşıklığı n^2 → Büyüme oranı ← ↓ Karmaşıklığı n

Döngülerin Büyüme Oranları

for(i=0; i<n; i++) → f(n)=n doğrusal

for(i=0; i<n; i=i+2) → f(n)=n/2 ''

for (i=1; i<=100; i*=2) → fn=$log_2 n$ logaritmik

for(i=0; i<n; i++)
for(j=0; j<n; j++) → $f(n)=n^2$ → Quadratik

for(i=0; i<n; i++) → $f(n)=\frac{n(n+1)}{2}$
for(j=0; j<i; j++) ↔ bağıml. Quadratik

for(i=0; i<n; i++)
for(j=0; j<n; j=j*2) → f(n)= n.log n → doğrusal logaritmik

Büyük O

f(n)'in büyüme oranı g(n) ile gösterilirse O(g(n))

∴ $O(n^2) \approx$ Big-Oh

A= for(i=0; i<n; i++) {} → O(n)

A. Servisi ← Üst sınır

$k * n^2 > n^2 - 3*n + 10$ → eğer → bunu sağlayan n ve k değerleri bulursam algoritma servisi n^2 olur.

$O(n^2)$ → $3n^2 > n^2 - 3n + 10$
$n \geq 2$ için

$f(n) > k.g(n)$

$\Omega(g(n))$ ← Alt sınır
(big omega)

$\Theta(g(n))$ ← Orta sınır

} ikisi birden: doğ.

$k_1.g(n) \geq f(n) \geq k_2 g(n)$

Binary search $\lfloor \log_2 n \rfloor - 1$

07.03.22
LAB 1

Özyinelemeli fonksiyonlar

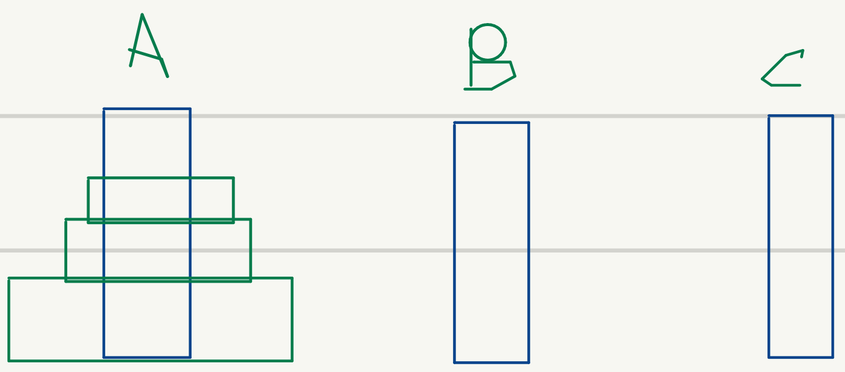
```
int main(){
  main();
  return 0;
}
```
-> Segmentation fault verir ve özyinelemeli değildir

Üs alma

```
power(x,y){
 if(y>0)
   return x*power(x,y-1);
 else return 1;
}
```

Print In Binary

Tower of hanoi ⟶ Bunu çözerlik



Özyinelemeli algoritmalarda Big-O

T(n) yineleme denklemi:

```
void f(n) {
  if (n>0) {
    printf(n);
    f(n-1); } }
```
-> T(0) = a

T(n) = b + T(n-1) -> n.b + a

```
int g(n) {
  if (n=1)
    return 2;
  else
    return 3* g(n/2)+g(n/2)+5;
}
```
T(1) = c T(n) = b + 2T(n/2)

$T(n) = T(n-1) * n \rightarrow \Theta(n^2)$

$T(n) = T(n/2) + c \rightarrow \Theta(\log n)$

$T(n) = T(n/2) + n \rightarrow \Theta(n)$

$T(n) = 2T(n/2) + 1 \rightarrow \Theta(n)$

Iteration Method

$T(n) = c + T(n/2)$ $T(n/2) = c + c + T(n/4)$ $n = 2^k$ $(n=4)$

$T(n/4)$ = $T(1)$

$2 \cdot \log_2 n \cdot c + T(1) \rightarrow \Theta(\log n)$

Fibonacci sayıları

```
int fib(n) {
  if (n<=1)
    return n;
  return fib(n-1) + fib(n-2); }
```

$T(n) = c + T(n-1) + T(n-2)$ $T(n-1) \sim T(n-2) \rightarrow T(n) = c + 2 \cdot T(n-1)$ $T(n) = c + 2 \cdot (2T(n-2) + c)$

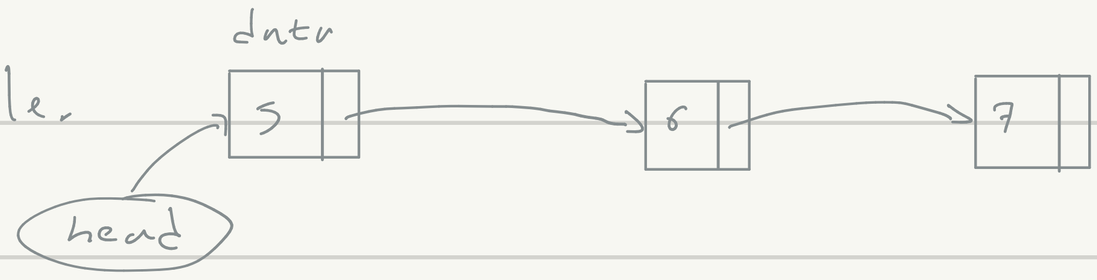
$n = k \rightarrow 2^k \cdot T(0) + (2^k - 1) c$ $T(n) \sim 2^n$ $\Theta(2^n)$

14.03.2022

3.

Diziler → Statik int a[10]; char a[]="naber"
                                  5+1 boyut
        → Dinamik int * a;

Linkl. Listele.

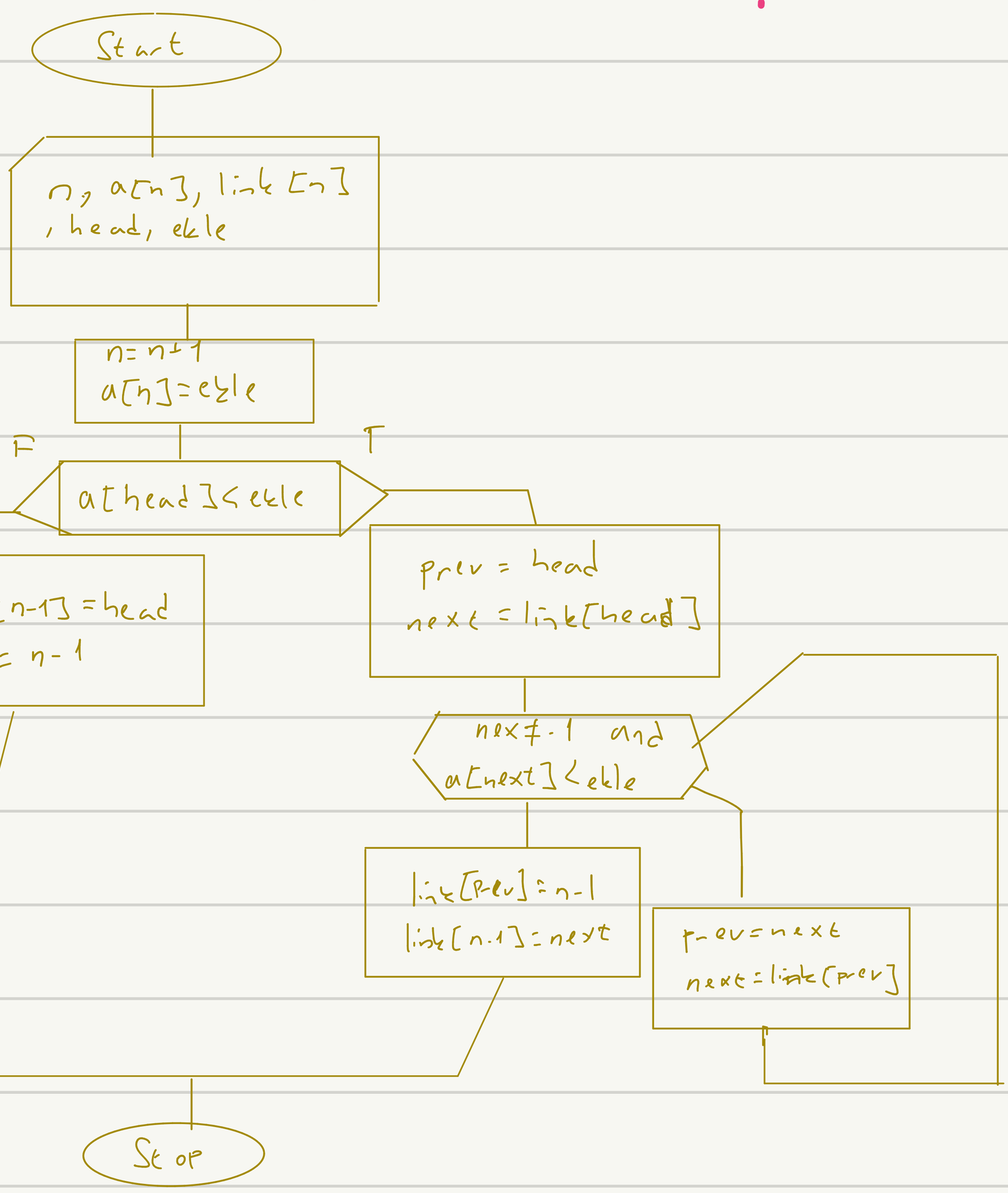


* Liste oluşturmak * Gezinti * Eleman ekleme-silme

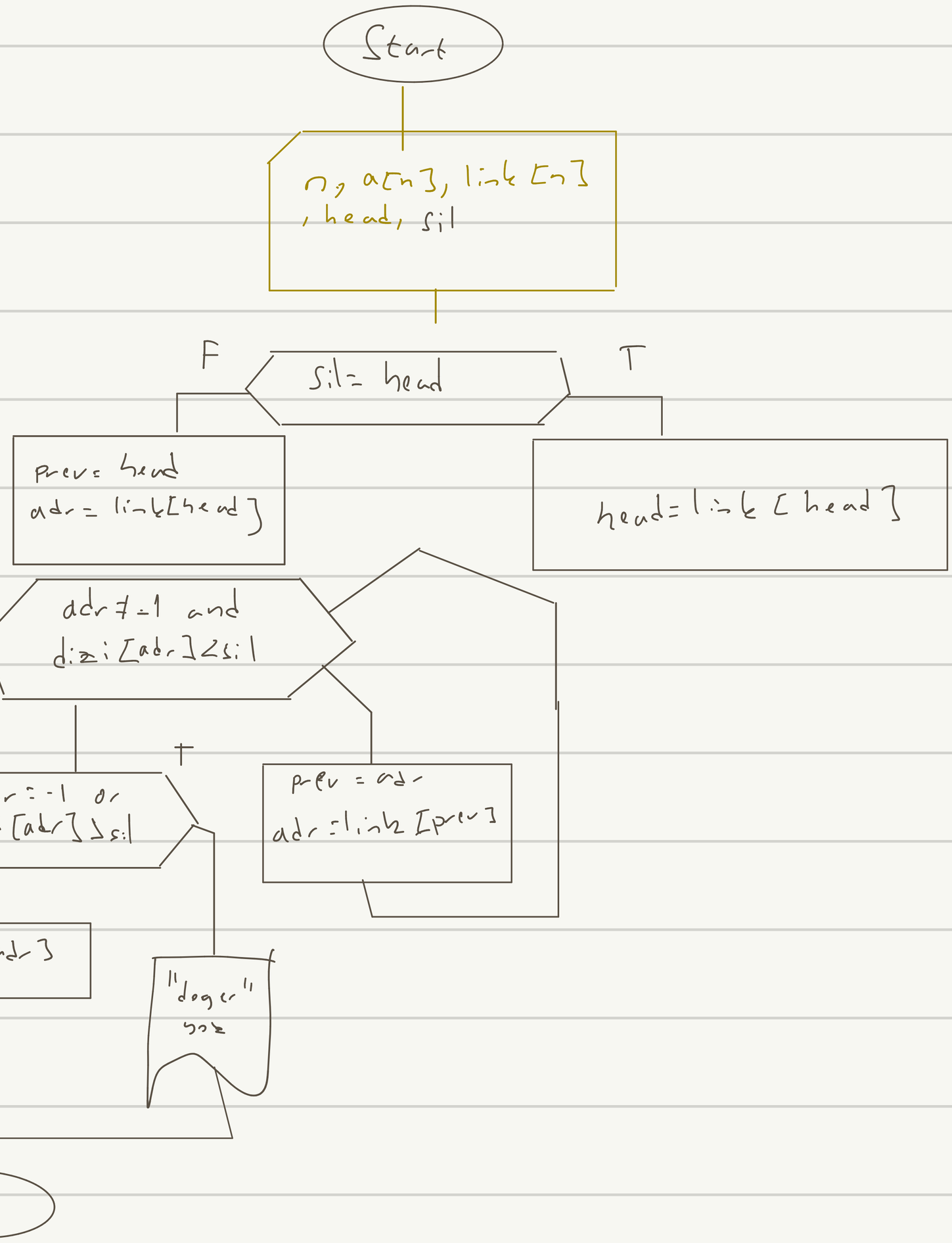
```
typedef struct link{
int a;
struct link * next;
}Link;
```

```
Link x=(node*)malloc(sizeof(node))
for(x=head;x!=null;x=x->next){
    Printf("x->a");}
```

Eleman ekle akış



Eleman silme akış



Double linked list

head, tail next, prev

* eleman ekle - sil * eleman ara * olustur

ekle

new -> next = curr -> next

curr -> next -> prev = new

curr -> next = new

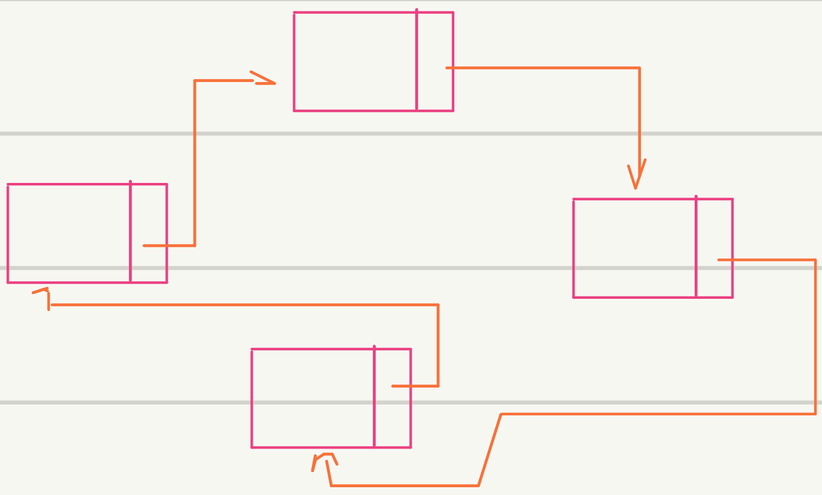
new -> prev = curr

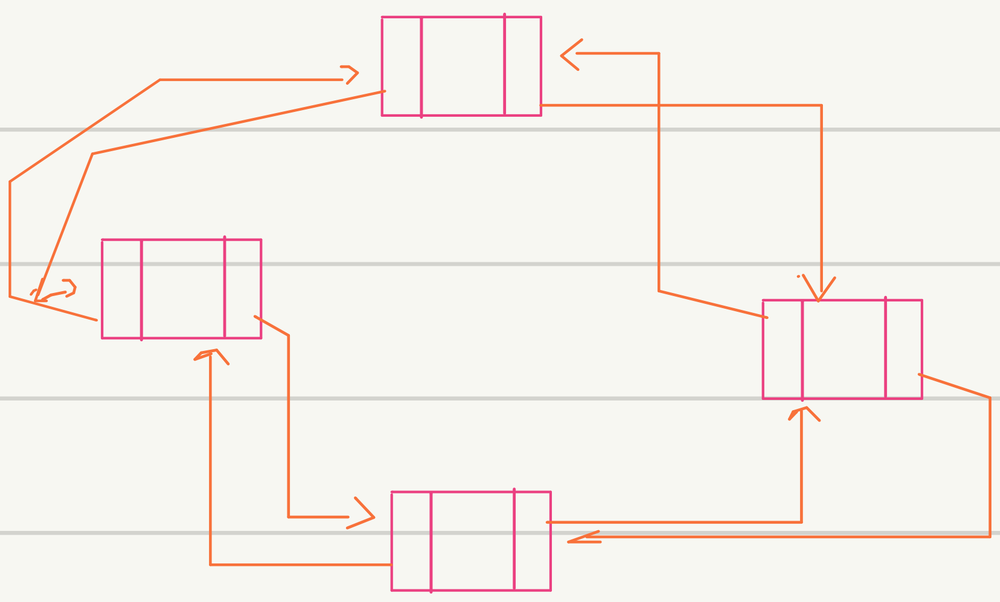
sil

del -> next -> prev = del -> prev

del -> prev -> next = del -> next

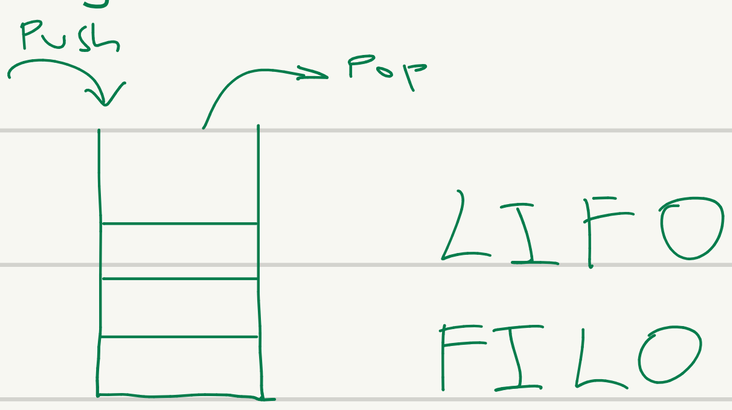
Circular linked list





21.03.22
4.

Y.giriş (stack)



LIFO
FILO

stack over flow

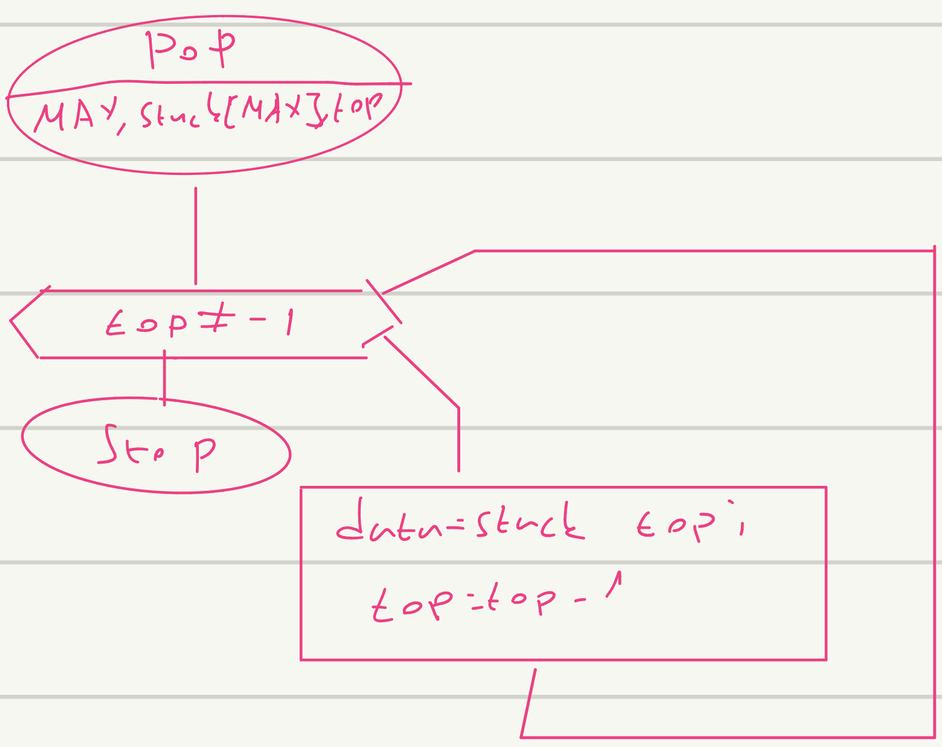
stack under flow

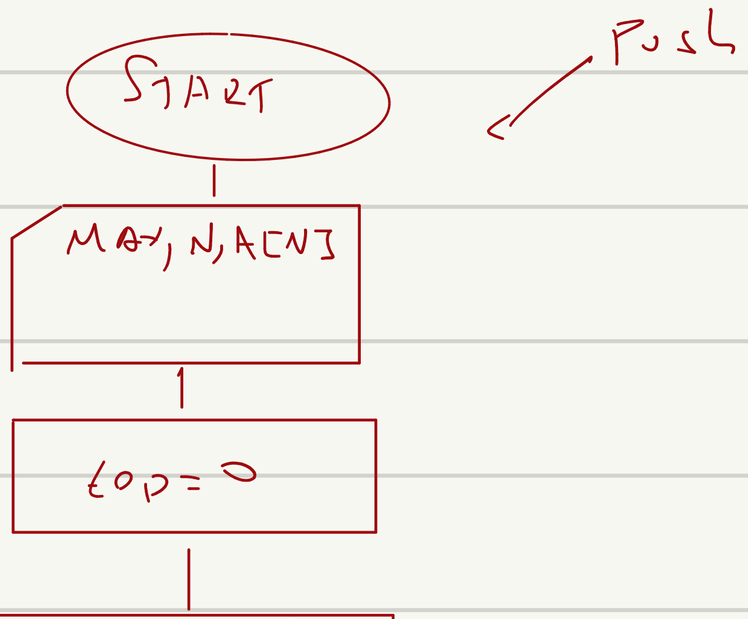
Diziyi stack olarak kullanabiliriz.

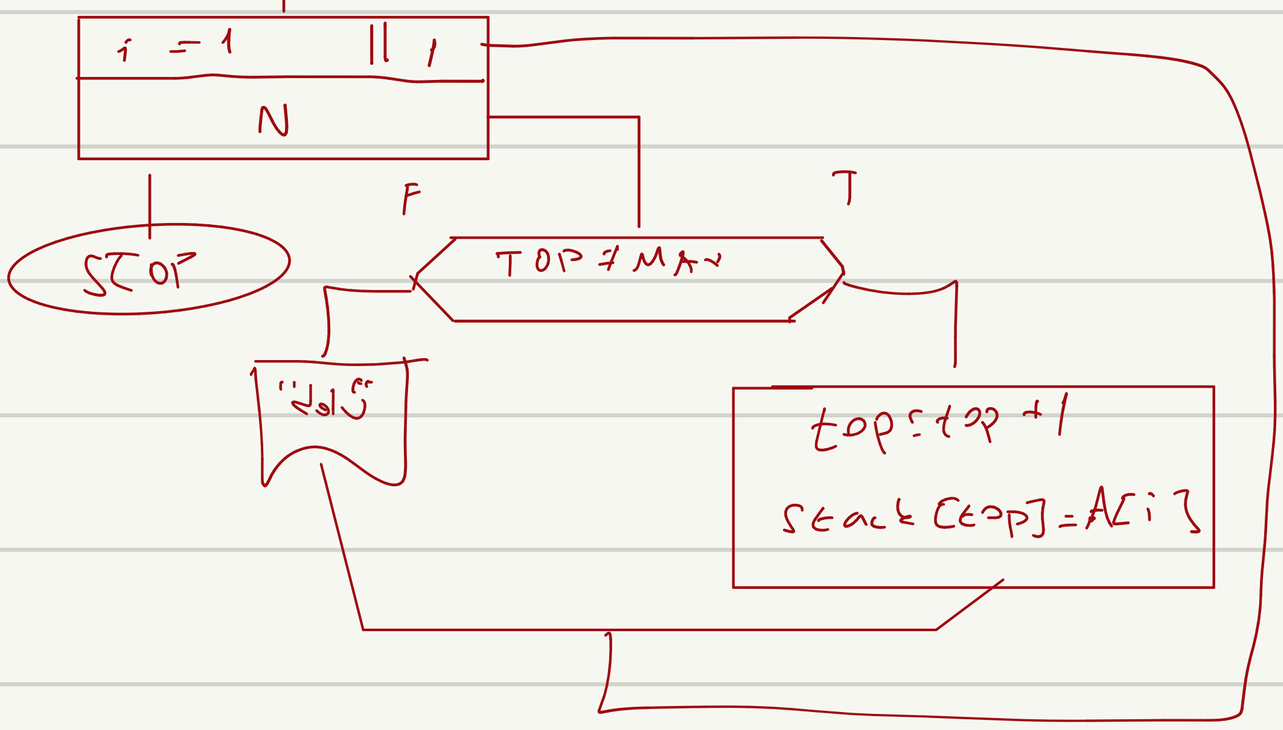
```
int stack[5];
int top=-1;

int pop(int * stack,int * top){
  if ((*top)==0){
     return 0;}
  else{
  (*top)--;
   return 1;}
   }
```

```
int push(int * stack, int * top, int max, int deger){
if (*top==max){
   printf("stack dolu");
   return 0;}
else{
  (*top)++;
   stack[top]=deger;
   return 1;
  }}
```

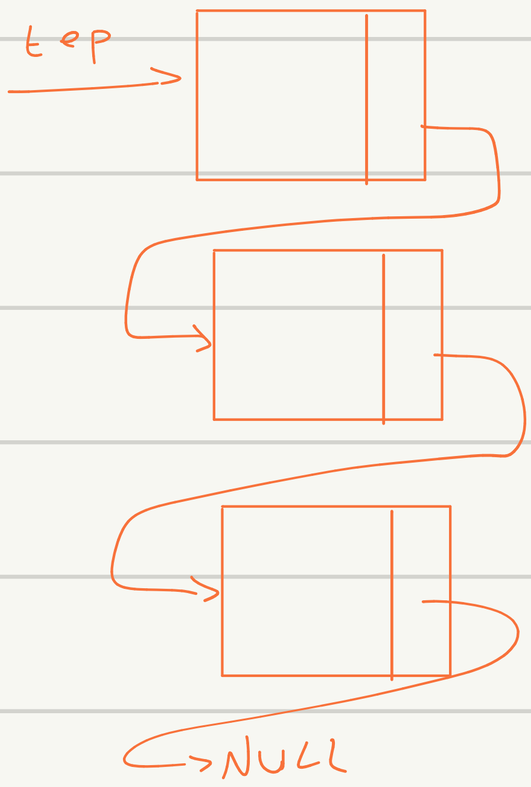






Palindrom kelime bulma akış.

2. yöntem linkli liste



Push

```
if(top == NULL){
    top=(node*) malloc (sizeof(node));
    top->next = NULL;
    top->data=data;
else {
    tmp=(node*) malloc (sizeof(node));
    tmp->next= top;
    tmp->data = data;
    top= tmp;}
```

Pop

```
if (top!=NULL){
    top1=top->next
    deger : top->deger
    free(top)
    top:=top1}
else{
    stack Los;}
```

Parantez kontrolü [, (, {

[()]{} push("["), push("("), if("(" == pop()), if("[" == pop()), push("{"), if("{" == pop())

if(isempty())

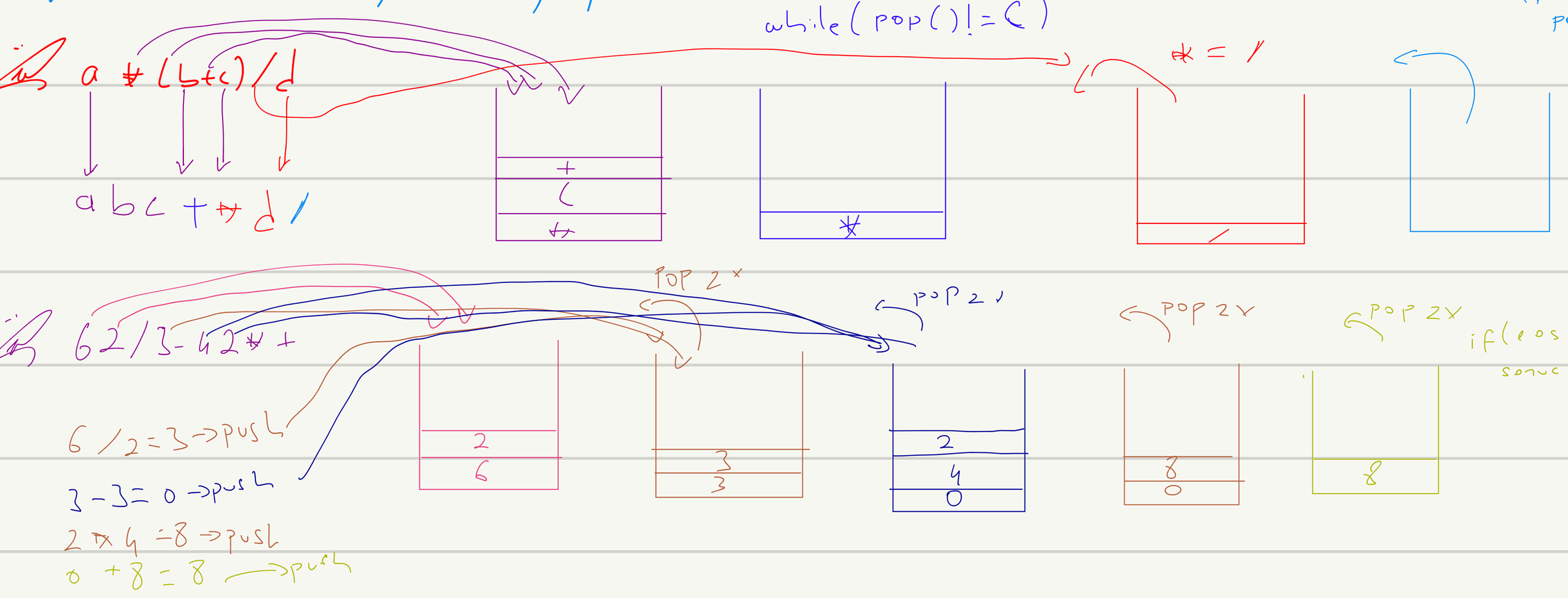
Aritmetik işlemler stack

3+2*5+1=14 → 325*+1+

a b/c - de*+ac*-

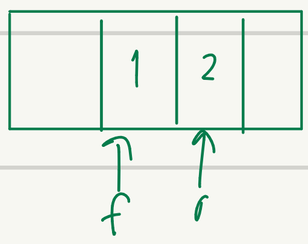
(a/b -c+d*e -a*c

operatörler → (), +, -, *, /, %



Push	Pop	Peek	isEmpty	isFull
O(1)	O(1)	O(1)	O(1)	O(1)

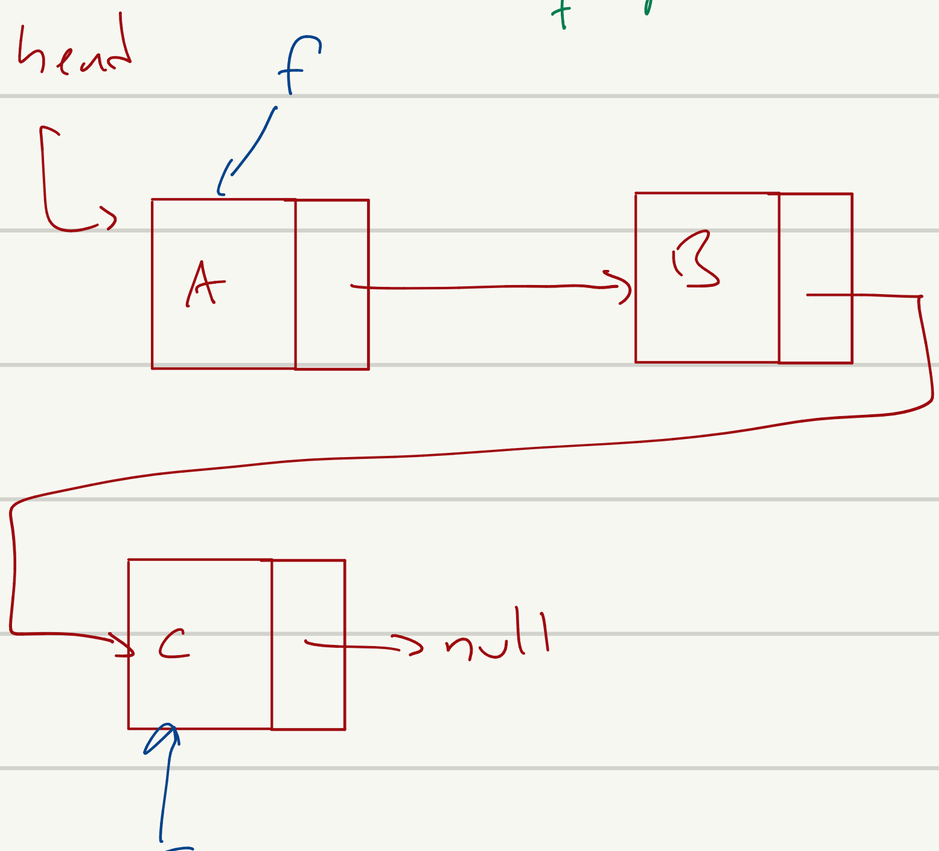
Kuyruk → FIFO

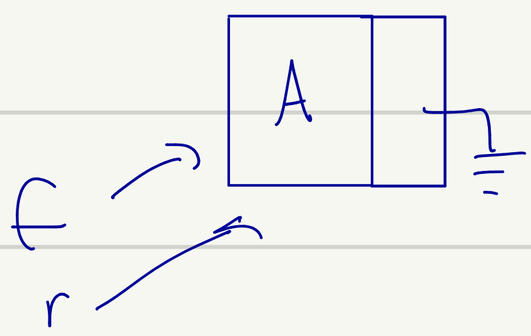


size
count

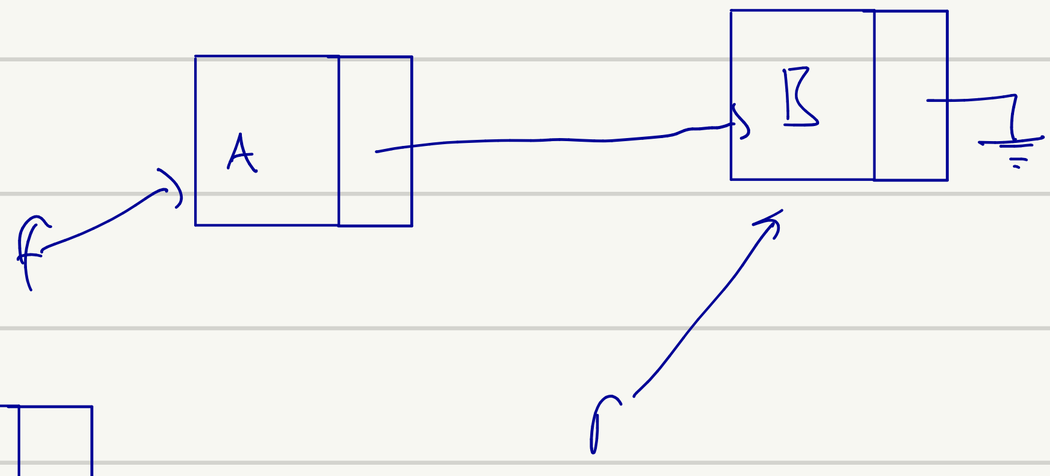
enqueue → push
dequeue → pop
peek → bakmak
isEmpty
isFull

```
#define max 6
int q[max];
int f=0)r=-1)count=0;
int isEmpty(){
return count;
}
int isFull(){
return count==max;}
int size(){
return count;}
void enqueue(int data){
if(!isFull()){
        if(r==max-1){
                r=-1;}
         q[++r]=data;
         cnt++;}}
int dequeue(){
int data=q[f++];
    if(f==max){
       f=0;}
    cnt--;
    return data;
}
```

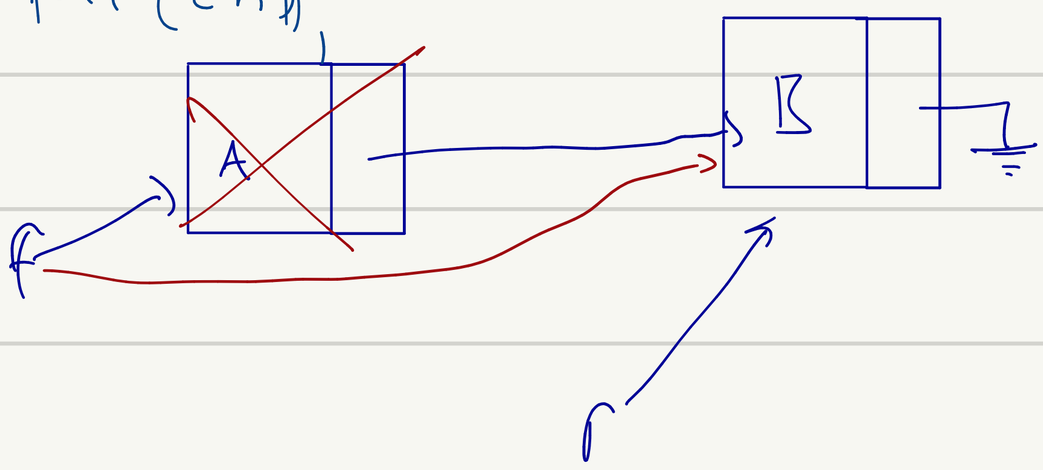




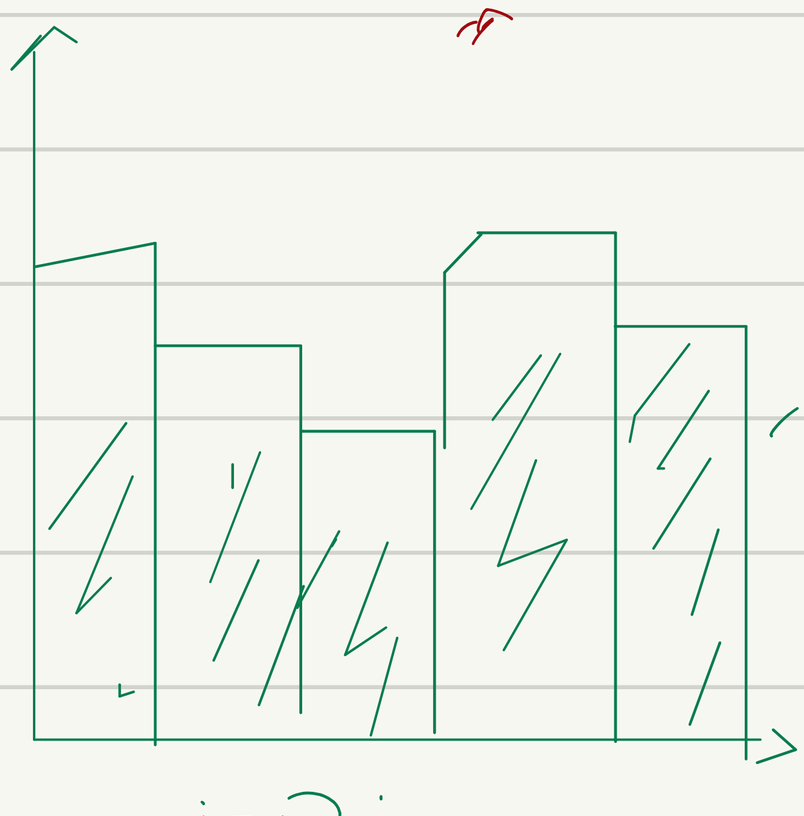

```
typedef struct queue{
        int count;
        node * f;
        node *r;} Queue;
```




```
tmp=f;
f=f->next;
free(tmp);
```



23.03.27
2. Lab



Alanı büyük dikdörtgeni bul

6	2	5	4	5	1	6

```
i=0;
while( büyüklük!=0) {
    if( isEmpty() or fseek(1)<=hist[i])
        push(i++);
    else {
    top=fseek1
```

Largest Rectangle Under Histogram Using Stack

```
#define MAX_STACK_SIZE 100
typedef struct Stack{
        int s[MAX_STACK_SIZE];
        int Top;
}stack;
int isFull(){
      if(stack.Top==MAX_STACK_SIZE)
      return 1;
      return 0;
}
int isEmpty(){
       if(stack.Top==-1)
         return 1;
        return 0;
}
void push( int data){
  if(!isFull())
    stack.s[
```

0 1 2 3 4

2 3 5 4 1

3

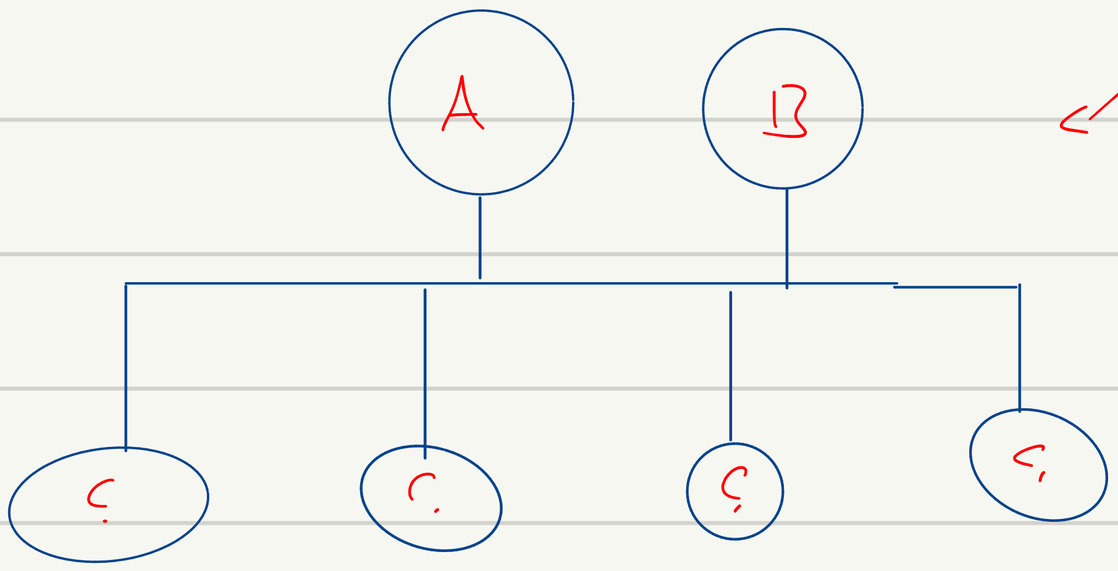
1

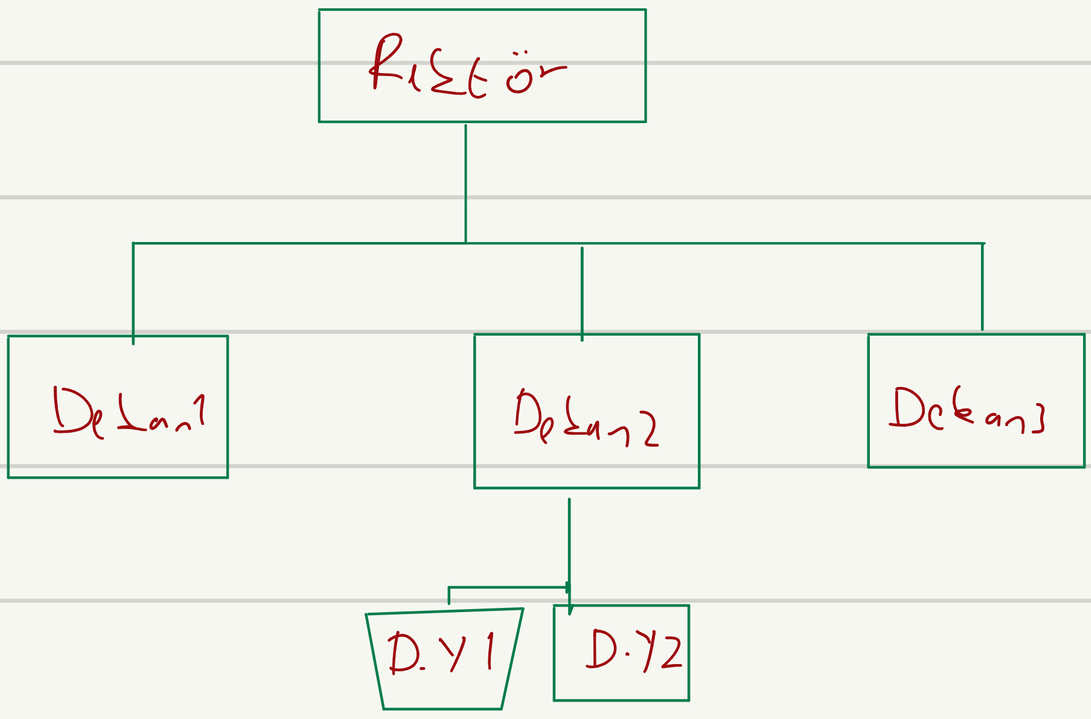
0

28.03.22
5.

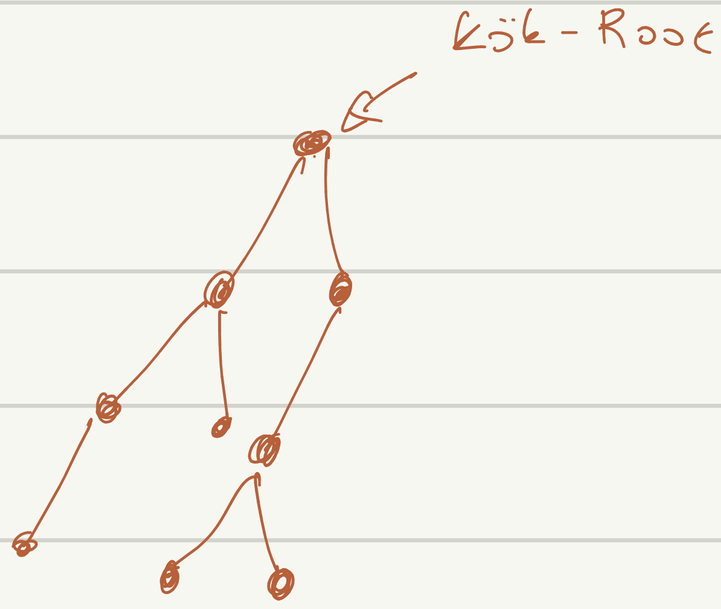
Ağaç

-Hiyerarşik - Birbirine bağlı düğümler - Kapalı çevrim yok - Sonlu düğüm

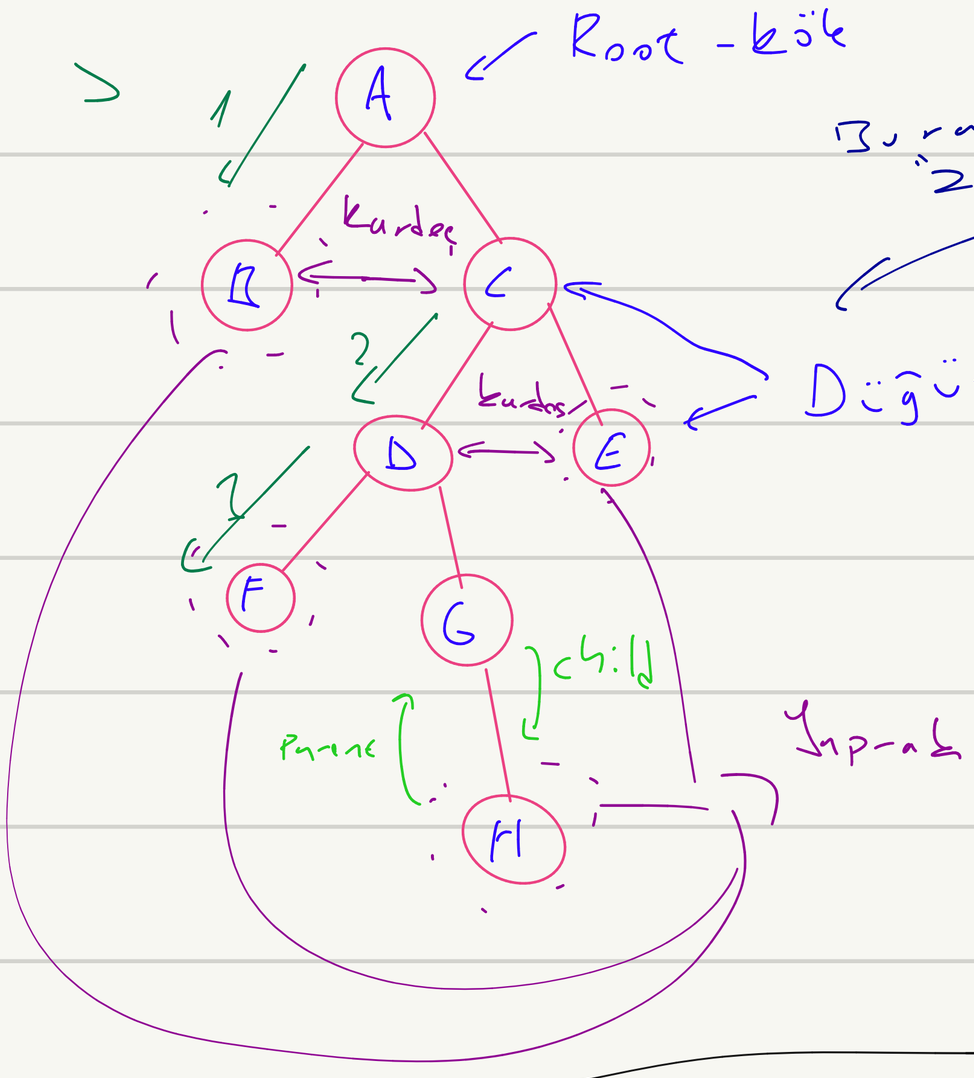




İkili ağaç



Üçlü ağaç



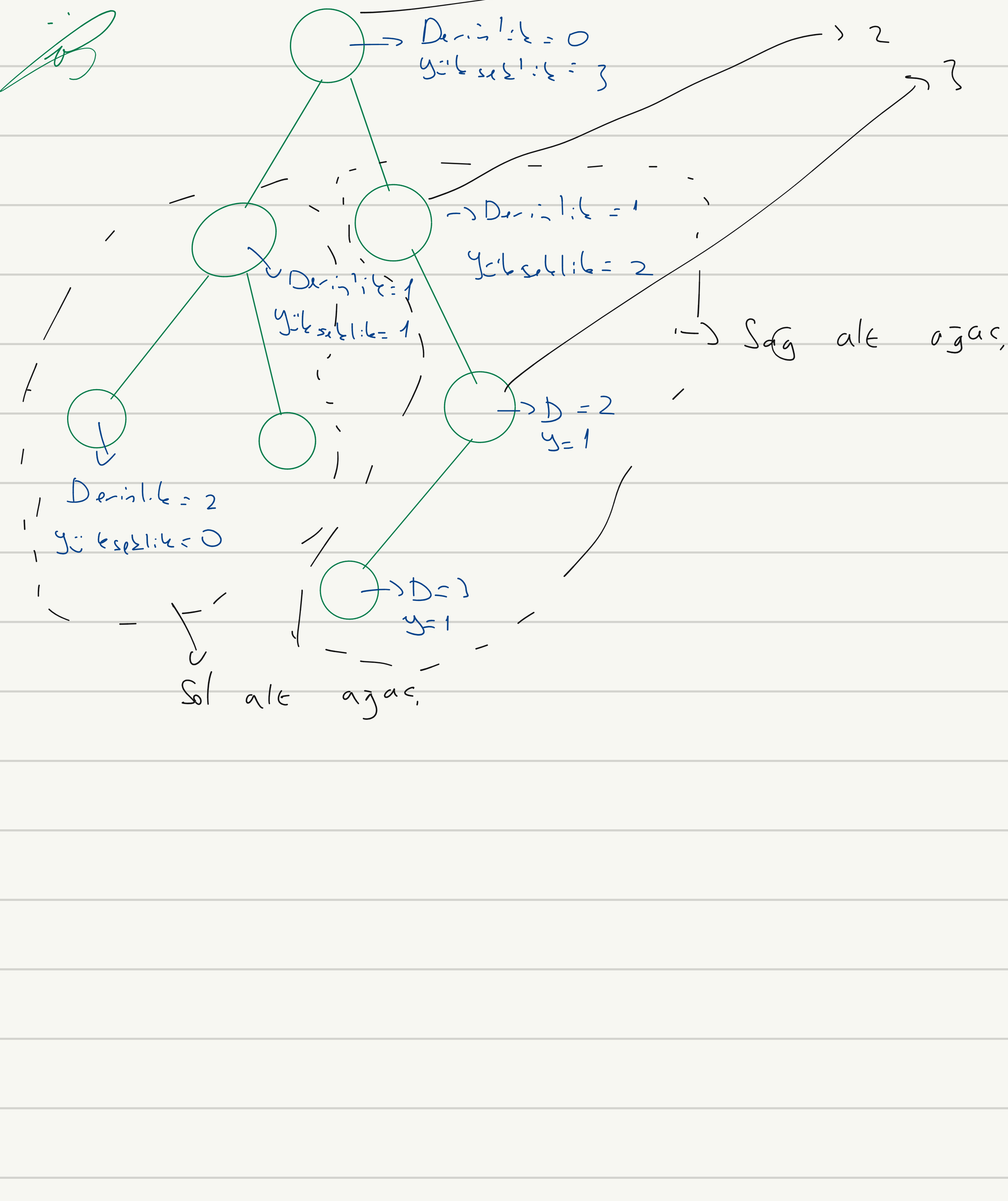
Burada Derece → En çok çocuğu olan düğümün çocuk sayısı (binary tree)

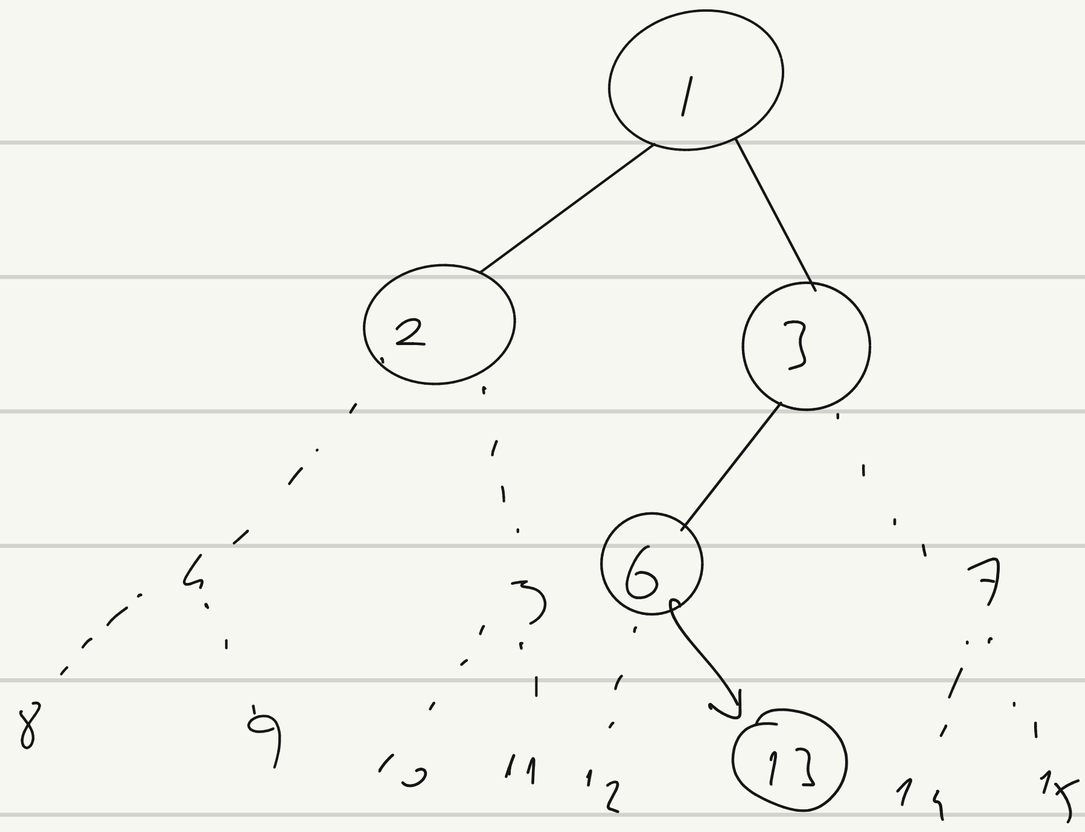
"2"

Yükseklik (Height) → Düğümden en dibe kadarki düğüm sayısı. → Burada 6

Derinlik: Düğümün köke olan uzaklığı

Seviye: 1'den başlayan



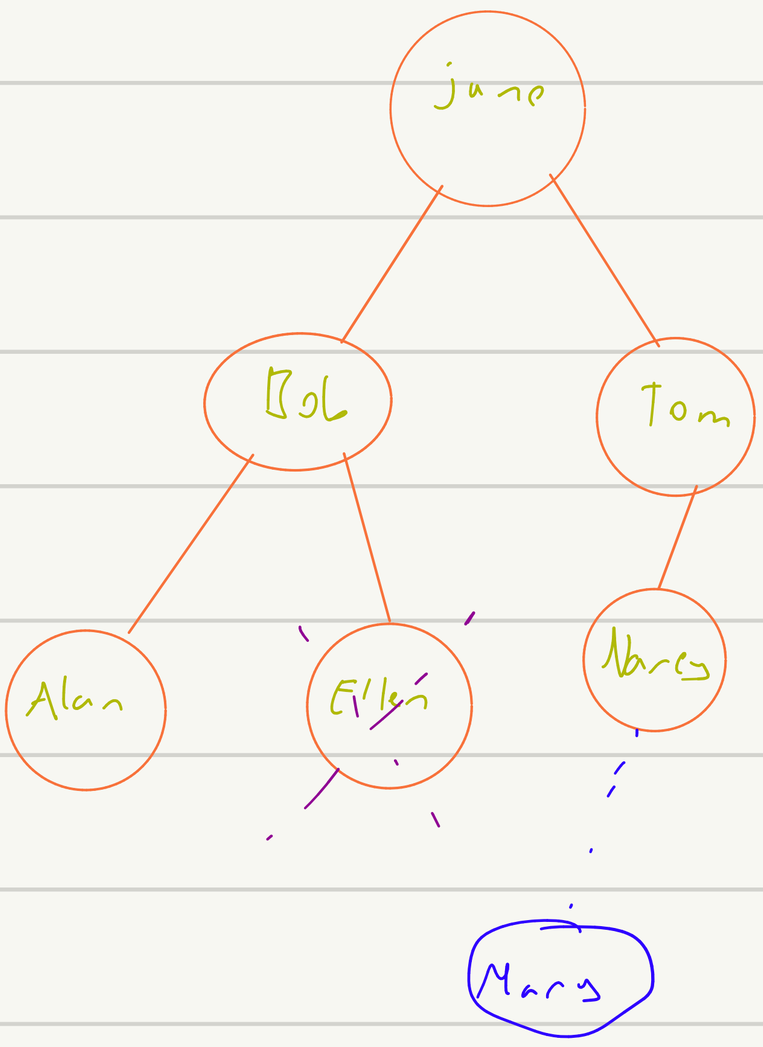


T =

1	2	3			6							13		

Sonraki elemanı 2k+1

Verimlilig düşük çok yer kaplıser



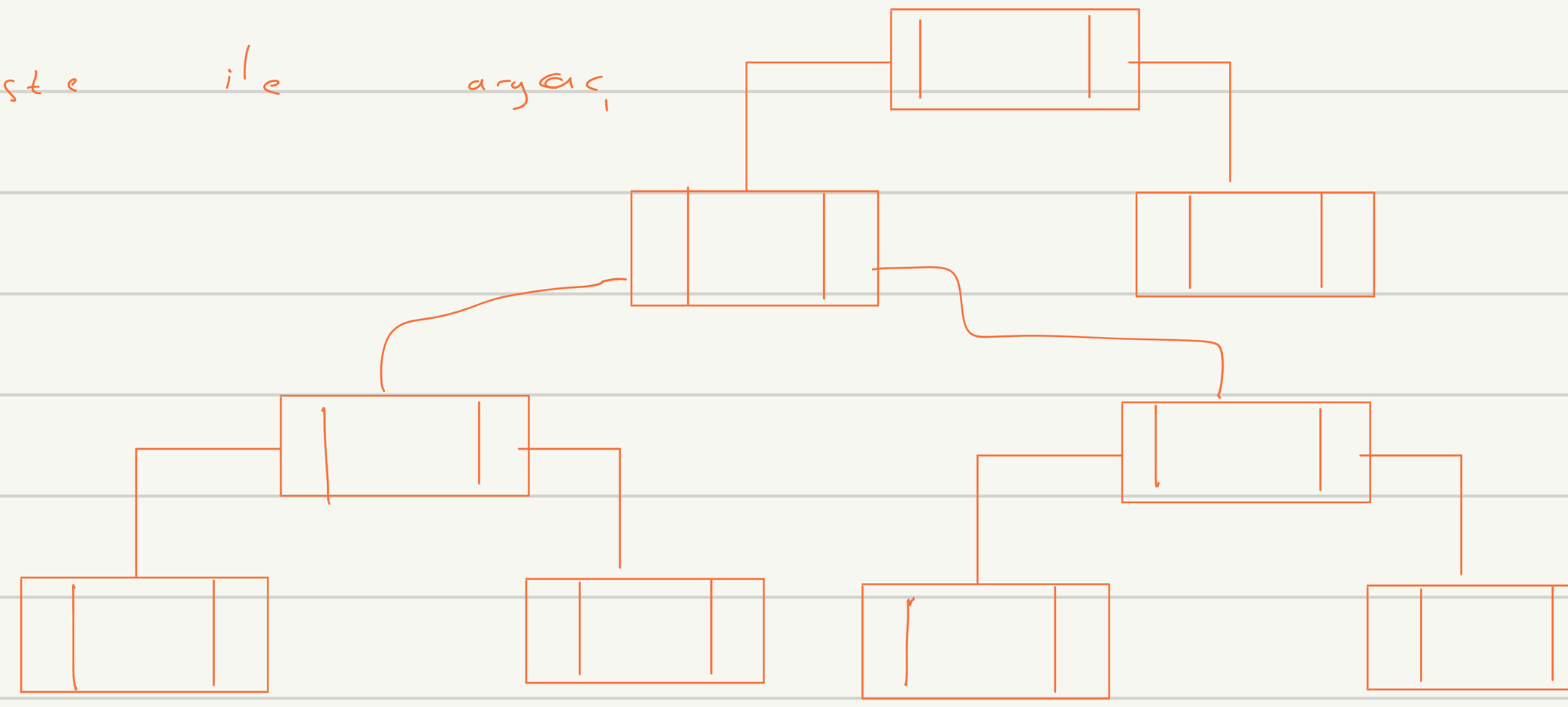
indis	veri	Sol indis	Sağ indis
0	Jane	1	2
1	Bob	3	~~4~~ -1
2	Tom	5	-1
3	Alan	-1	-1
4	~~Ellen~~	~~-1~~	~~-1~~
5	Nancy	~~-1~~ 6	-1
6	Mary	-1	-1
7			
8			
9			

kök = 0

Boş indis = ~~6~~ 7

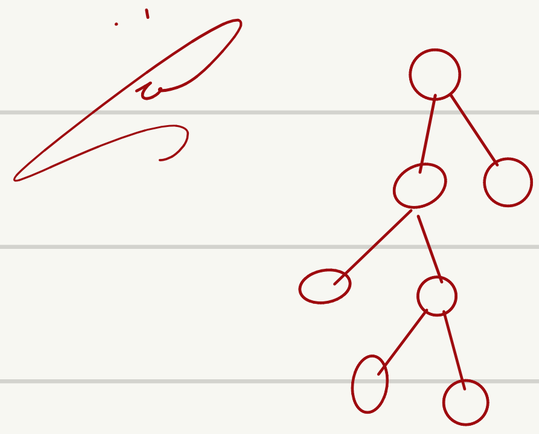
```
typedef struct node{
int data;
int left;
int right;} ;
node * tree;
```

Linkli liste ile ağaç

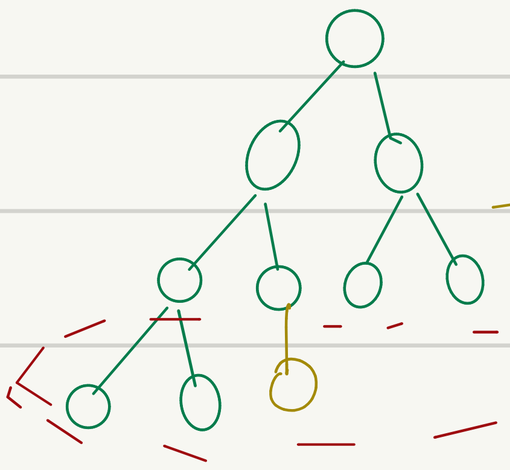


```
typedef struct tree{
int data;
 struct tree* left;
struct tree* right;} Tree;
```

Full binary tree → tek çocuğu olan düğüm yok.



Complete binary tree → son seviye hariç tüm seviyeler dolu olacak.

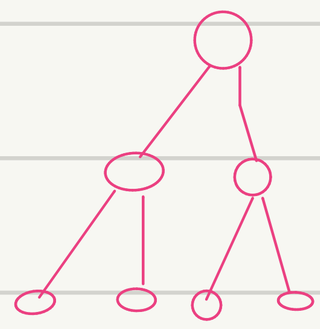


→ soldan sağa

→ complete ama full değil

÷> son seviye buası buası hariç hepsi dolu

Perfect binary tree → tüm düğümlerin 2 çocuğu var ve tüm yaprak lar aynı seviyede. Dengeli.



Balanced binary tree N düğüm için yükseklik<=log n ise balanced

Tree gezinimi önce derinlik

→ LVR

Leftten rightta

rightcum leftta

RVL

inorder

önce genişlik seviye

VLR

VRL

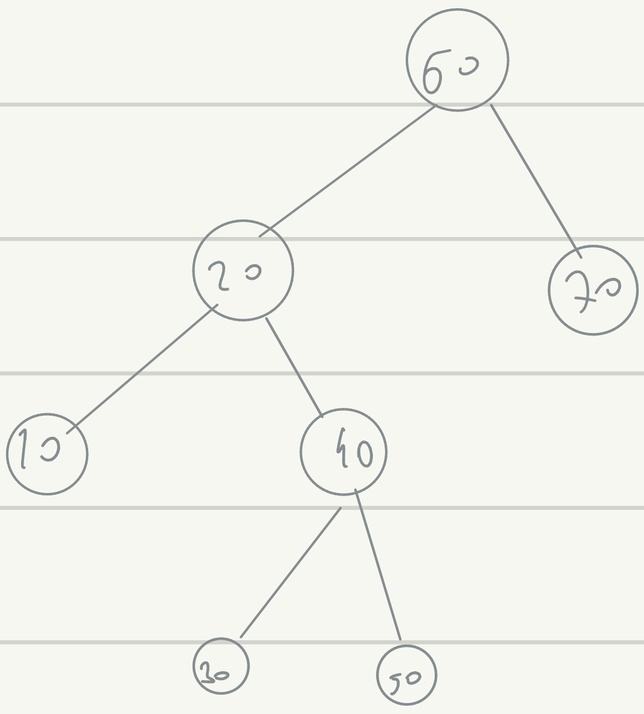
değeri önce

kullanışlı preorder

LRV

RLV

Postorder



VLR -> 60 - 20 - 10 - 40 - 30 - 50 - 70

VRL -> 60 - 70 - 20 - 40 - 50 - 30 - 10

```
void preTraverse(tree t){
if (t==NULL)
    return;
visit(t);
preTraverse(t->left);
preTraverse(t->right);
}
```

(VLR)

LVR -> 10 - 20 - 30 - 40 - 50 - 60 - 70

RVL -> 70 - 60 - 50 - 40 - 30 - 20 - 10

```
void preTraverse(tree t){
if (t==NULL)
    return;
preTraverse(t->left);
visit(t);
preTraverse(t->right);
}
```

(LVR)

LRV -> 10 - 30 - 50 - 40 - 20 - 70 - 60

RLV -> 70 - 50 - 30 - 40 - 10 - 20 - 60

```
void preTraverse(tree t){
if (t==NULL)
    return;
preTraverse(t->left);
preTraverse(t->right);
void(t->right) ;
}
```

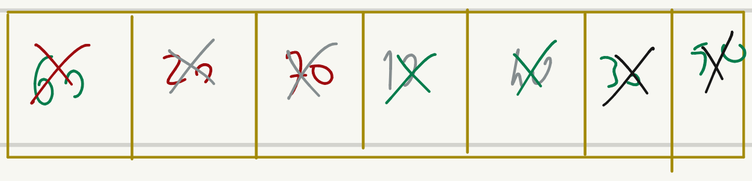
(LRV)

Gezinti işlemi stackle olabilir

```
curr=root;
while(current!=NULL){
push(current);
current=current->left;}
 while(current!=NULL && !isEmpty()){
x=pop();
print(x);
current=current->right;}
if(current==NULL && isEmpty());
stop.
```

Önce sonuç yazdırma -> bu şekilde yapılır.

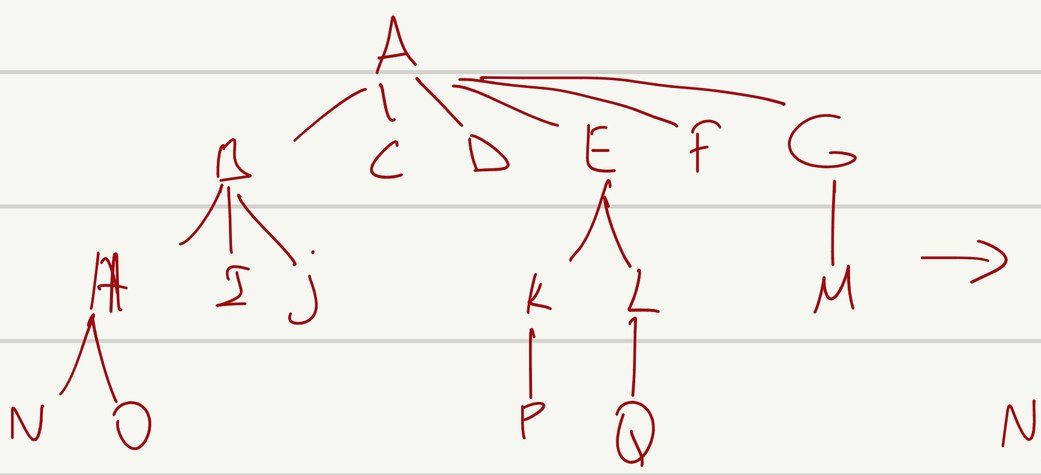
Kuyruk boşalana kadar

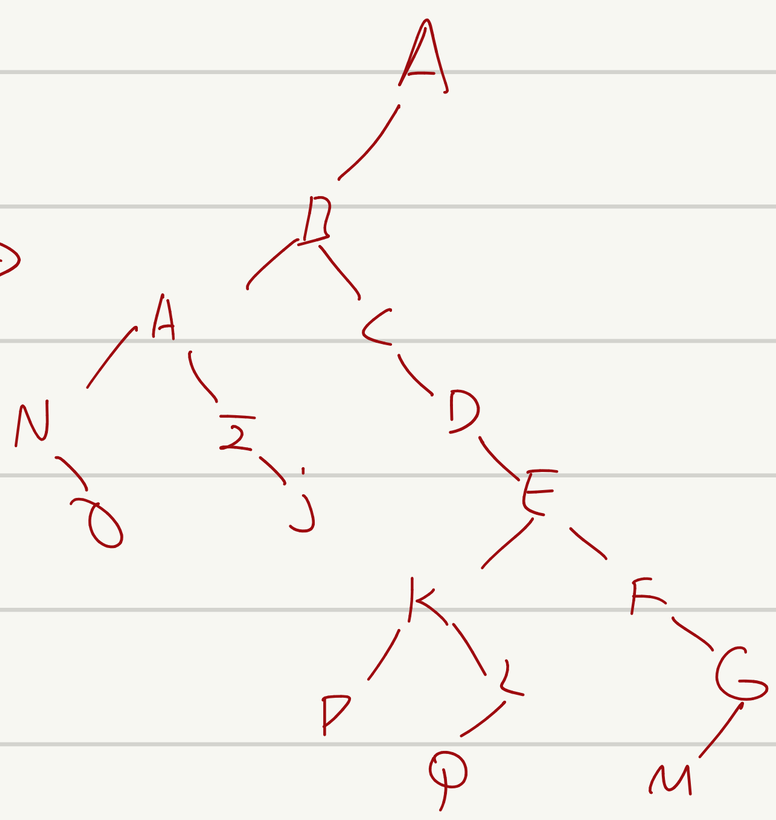


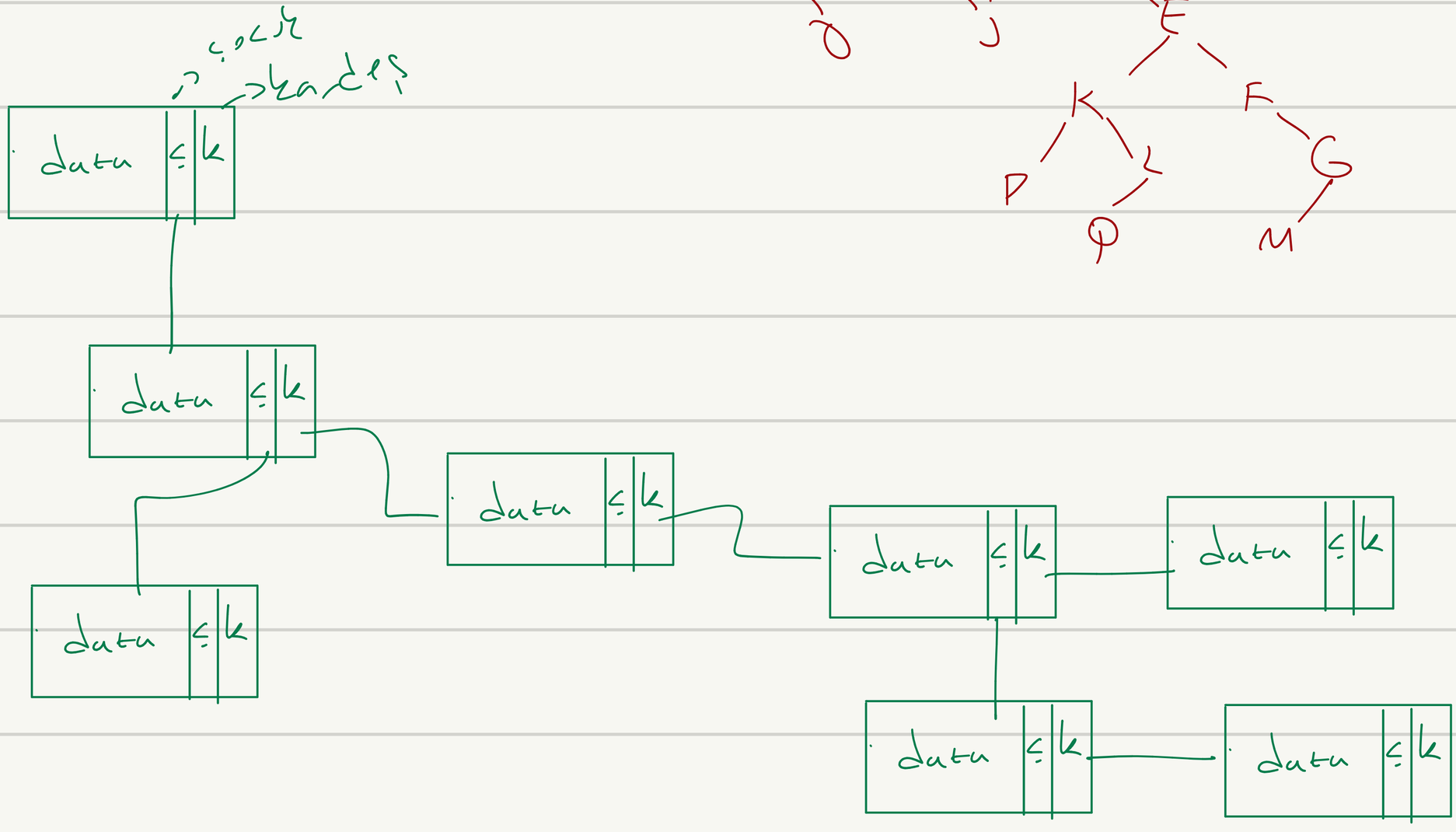
60 - 20 - 70 - 10 - 40 - 30 - 50

Tüm ağaçları ikili şekilde gösterebiliriz

Sol taraf çocuk - Sağ taraf kardeş olur

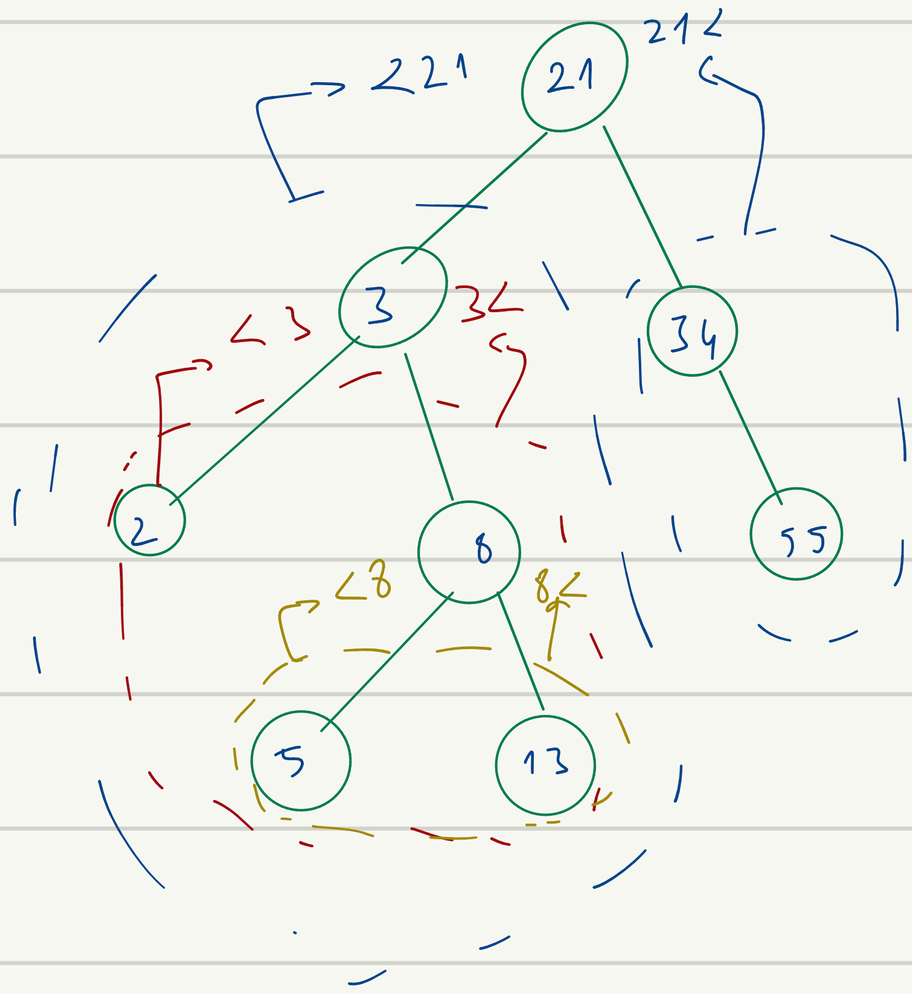


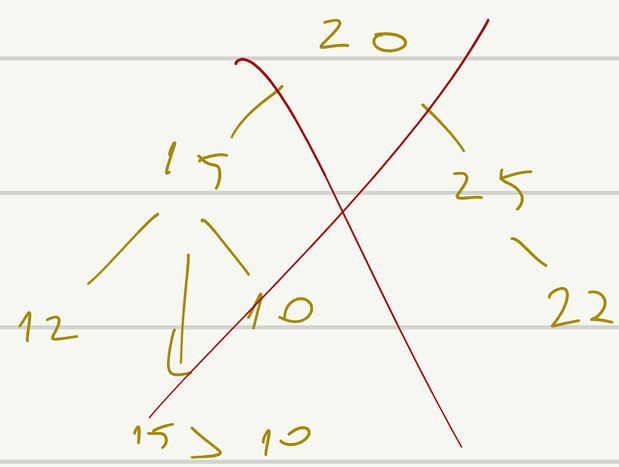


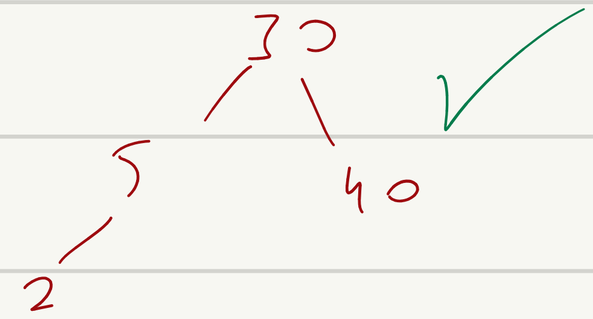


Binary Search tree

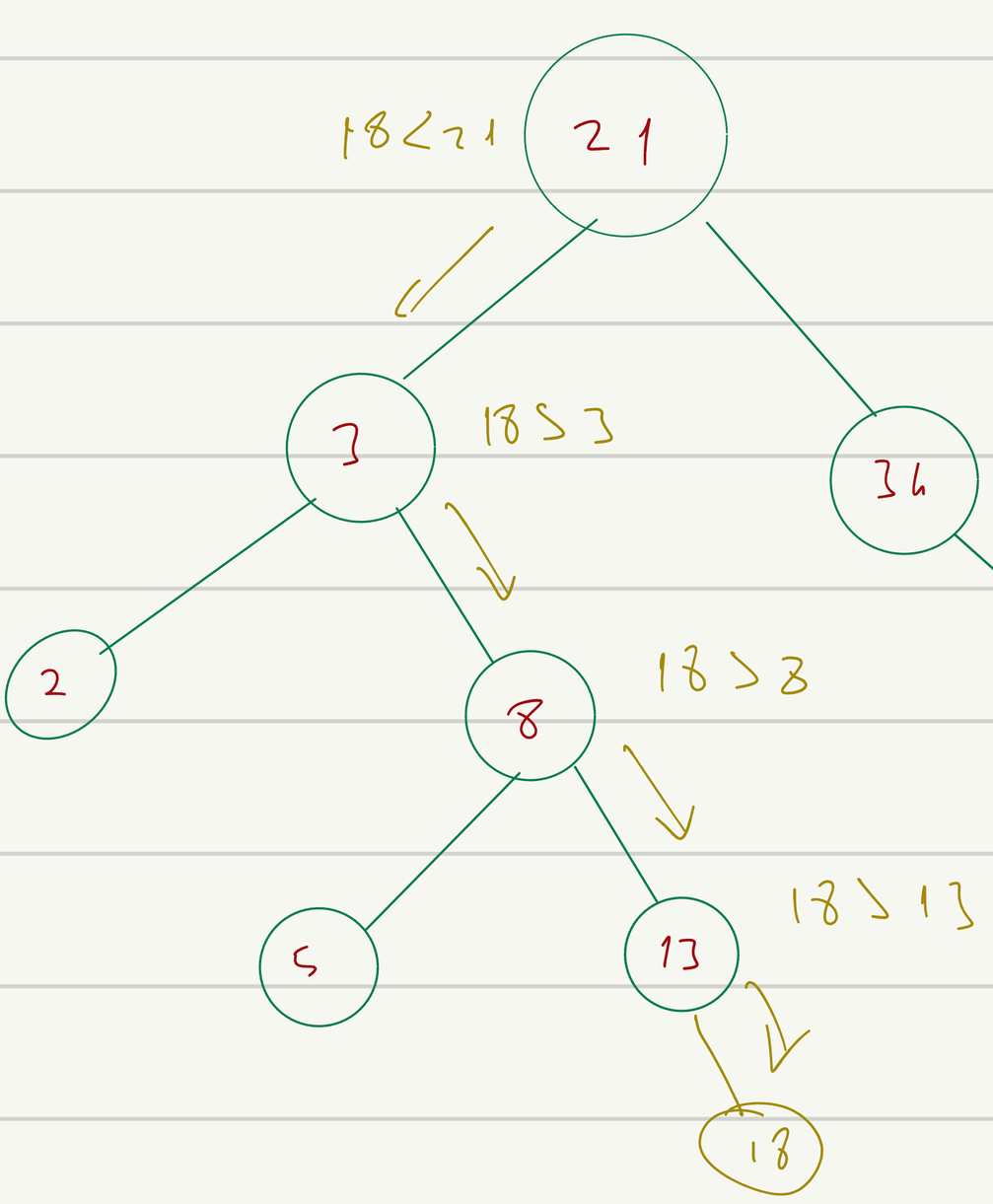
A düğümünün değeri sol alttan büyük ve sağ alttan küçükse (ağaç)





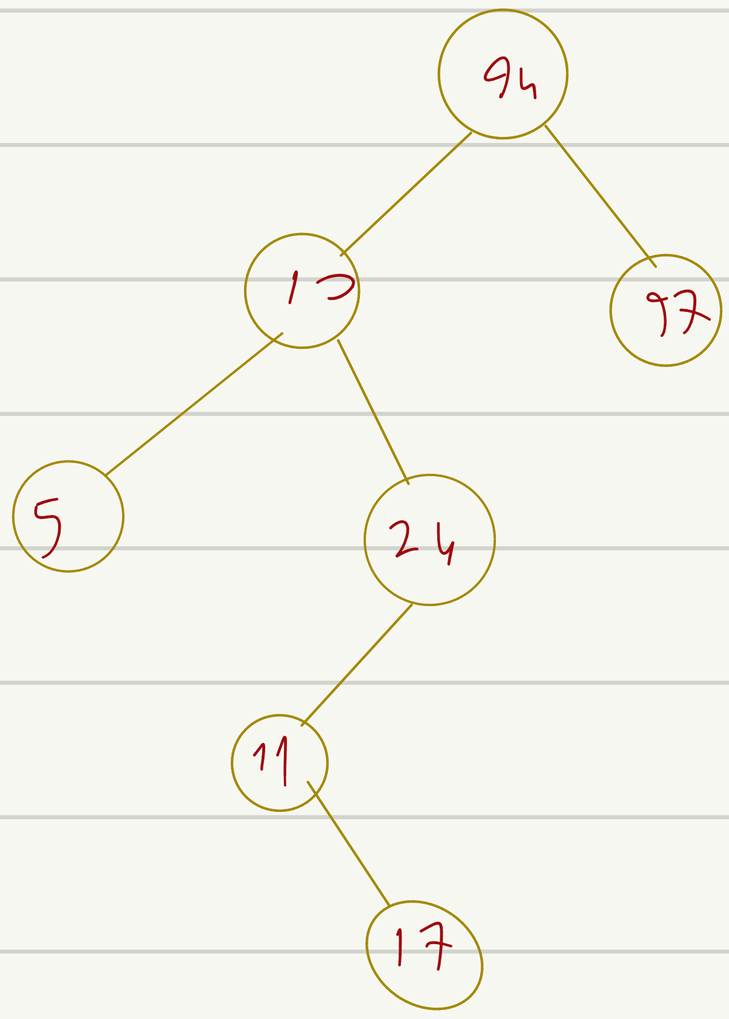


18 : ekle




```
tree* insert(tree *
node)int data){
if (node==NULL)
return
createTree(data);
if(data<node->data){
node-
>left=insert(node->left)
data);}
else if(data>node-
>data){
node-
>right=insert(node-
>right)data};
```

Eleman sil



Sil → çocuk sol

ara ve free et bu kadar

Tek çocuk varsa

24'ü sil

önce 24'ün üstündeki sağına 24'ün çocuklarını bağla → sağ ise sağa sol ise sola.

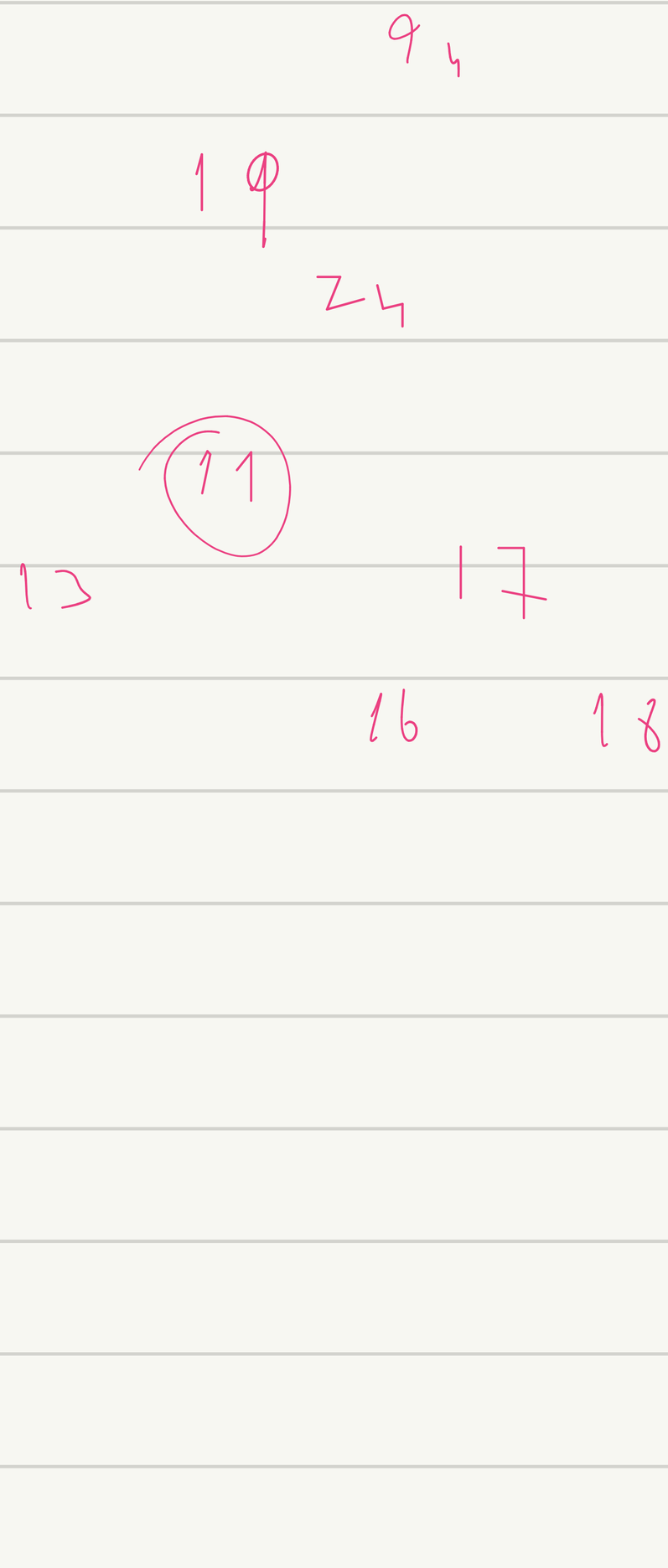
11'i sil

Önce üstündeki sağının soluna 11'in çocuğunu bağla sonra free et.

2 çocuk varsa

10'u silelim

Burada ya sol ut ağaçtaki en büyük elemanı seçip 10'un yerine tuşuz. Burada 5'i koyacağız. 2. sol sağ tarafın en küçük elemanını 10'un yerine koyarız. 17'yi de 11'in yerine koyarız.

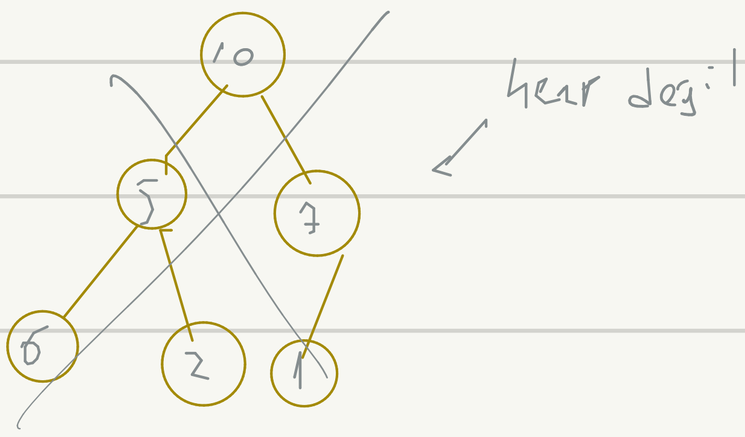


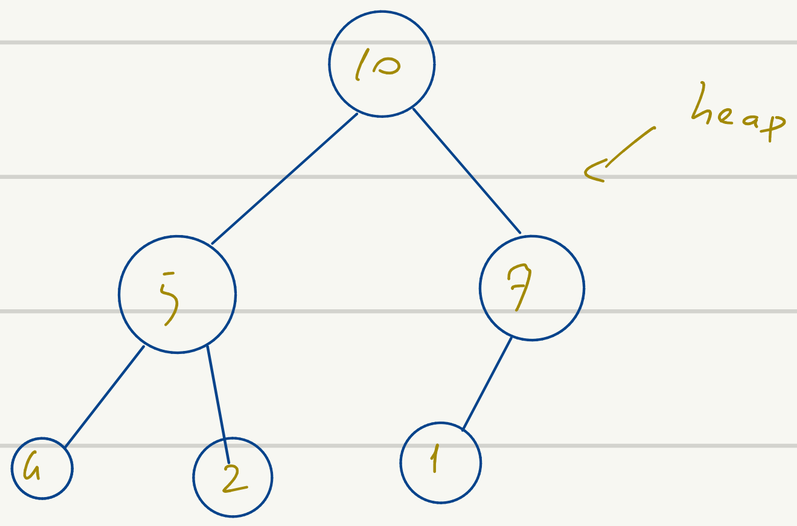
04.04.2022
6.

Heap → Neredeyse complete binary tree gibi

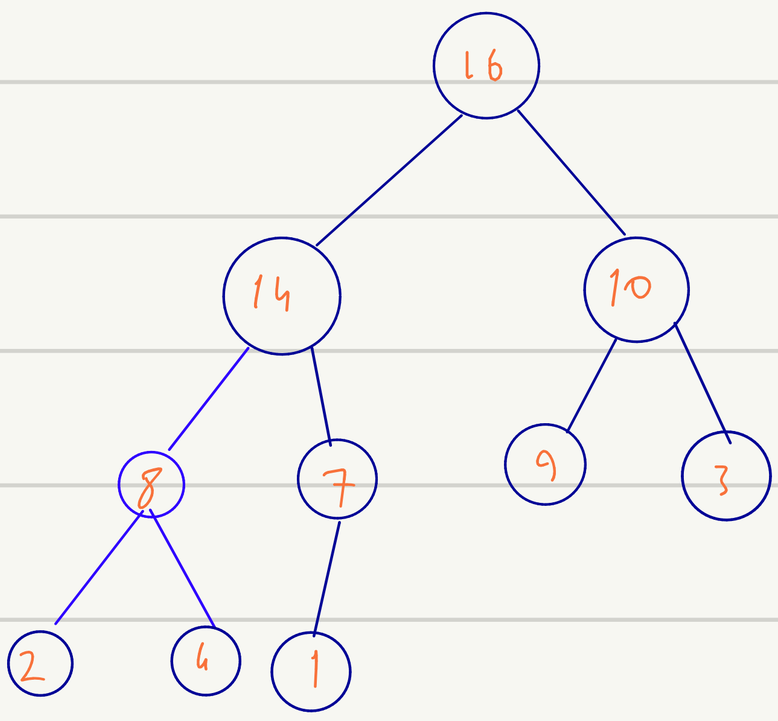
→ Bir düğümde bulunan değer çocuklarından daha büyük/küçükse heap veri yapısıdır.

A





Max Heap A[Parent(i)] >= A[i]



Min Heap A[Parent(i)] <= A[i]

A =

0	1	2	3	4	5	6	7	8	9
16	14	10	8	7	9	3	2	4	1

Root = A[0]

Node = A[i]

Parent = A[(i-1)/2]

Left(i) = A[2i+1]

Right(i) = A[2(i+1)]

Not: Son Parent

N/2. eleman

N/2'den sonraya

Labma

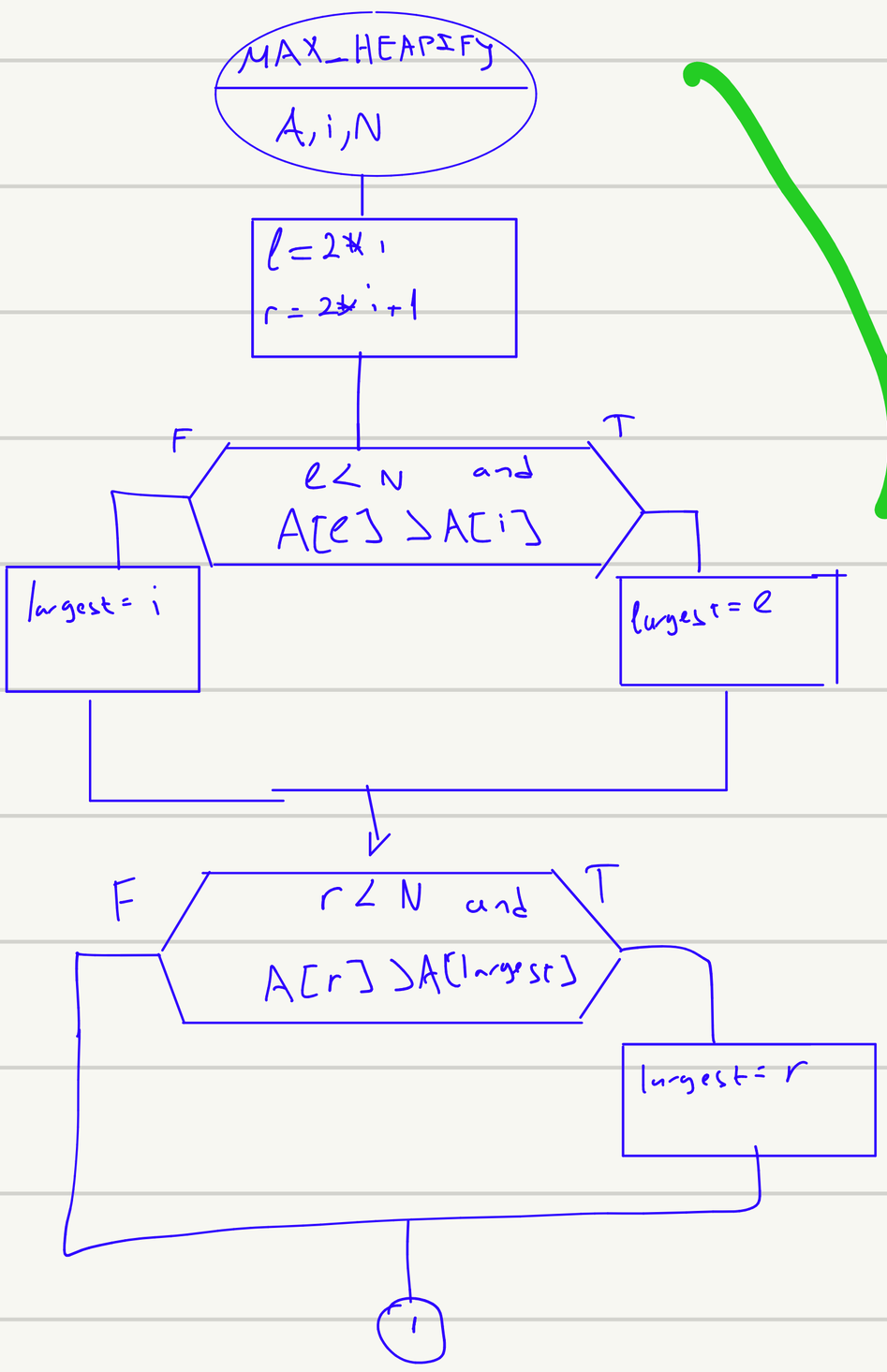
$H = \log_2 N$

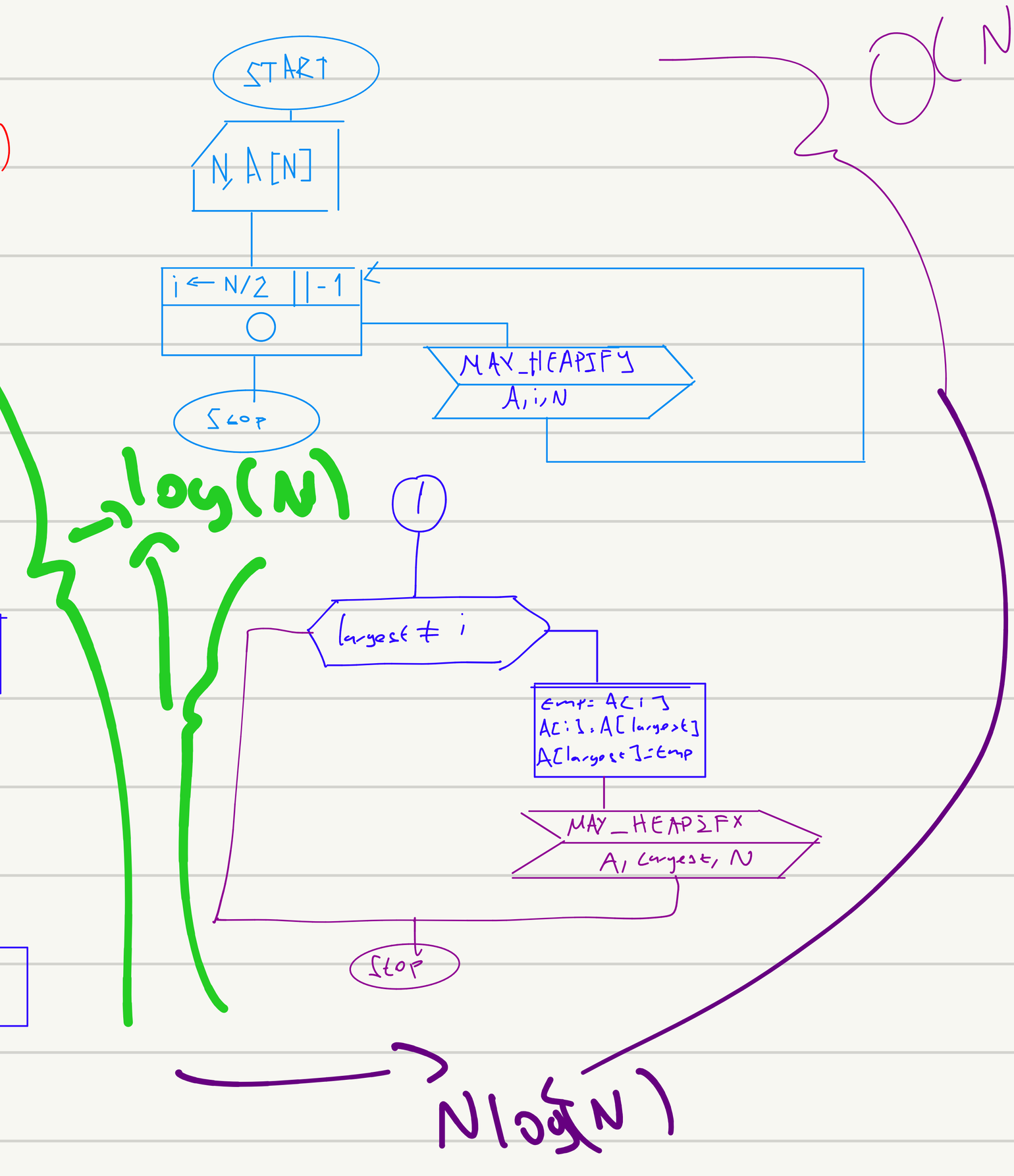
BUILD MAX_HEAP_Bottom UP

O(N)

Heap_MAX

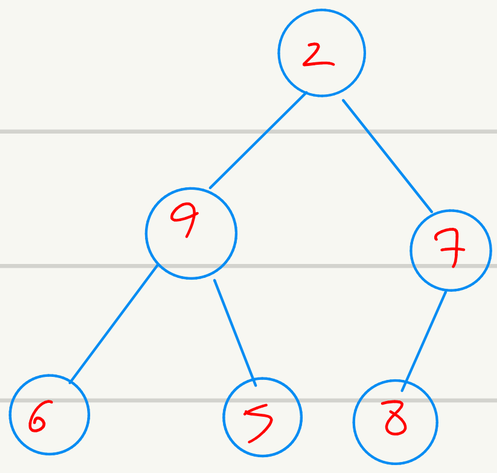
return A[0]; O(1)

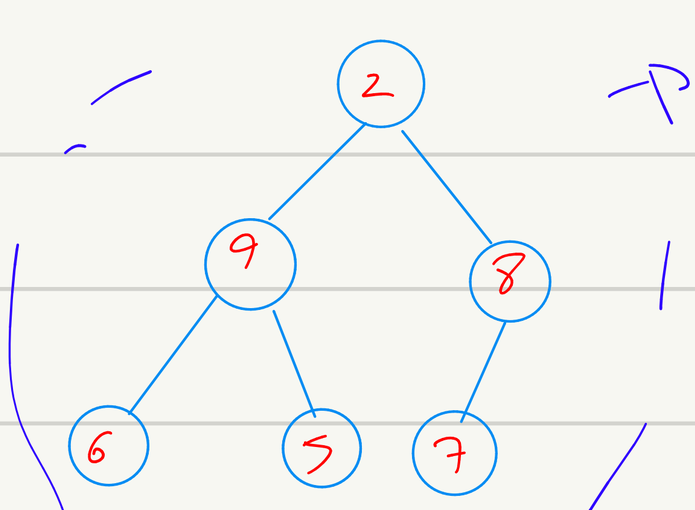


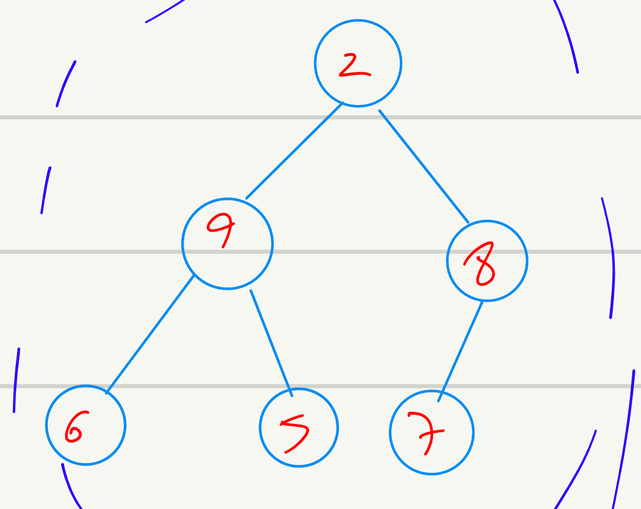


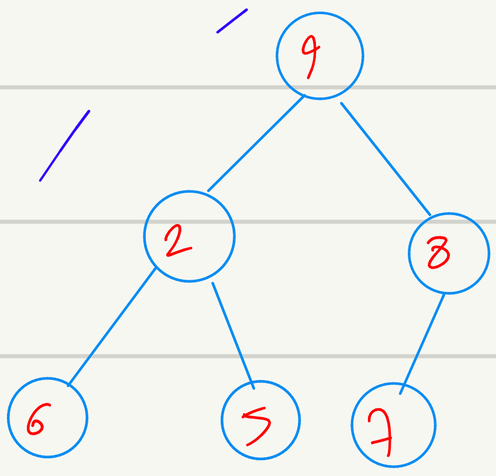
$A = 2, 9, 7, 6, 5, 8$

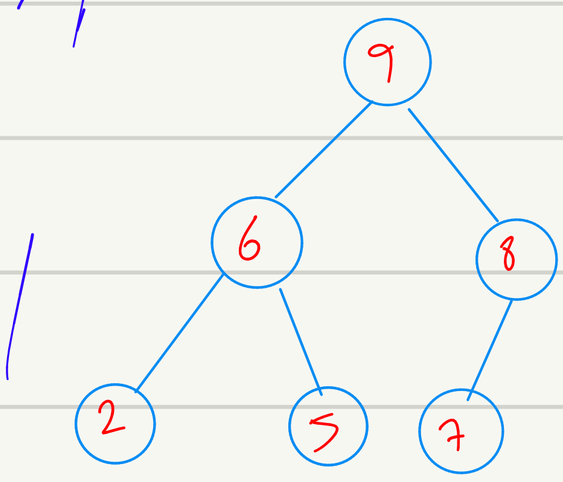
2	9	7	6	5	8



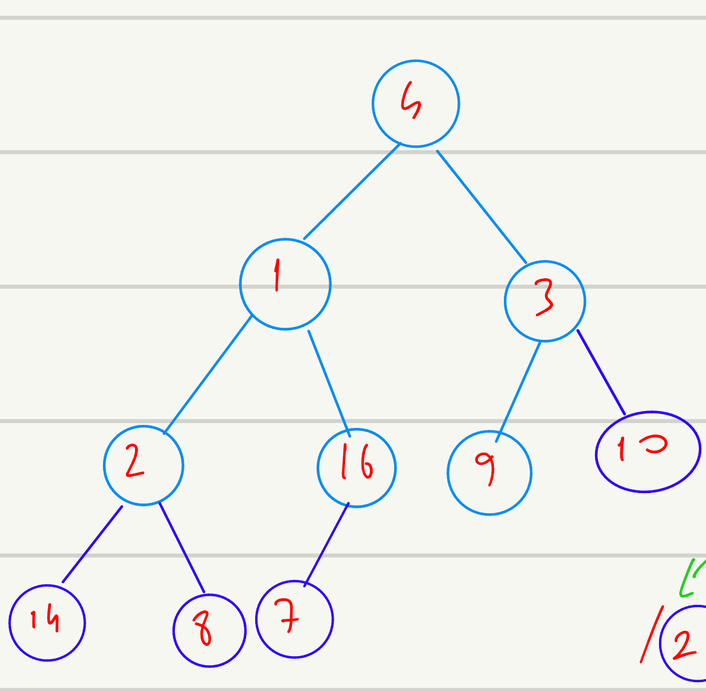


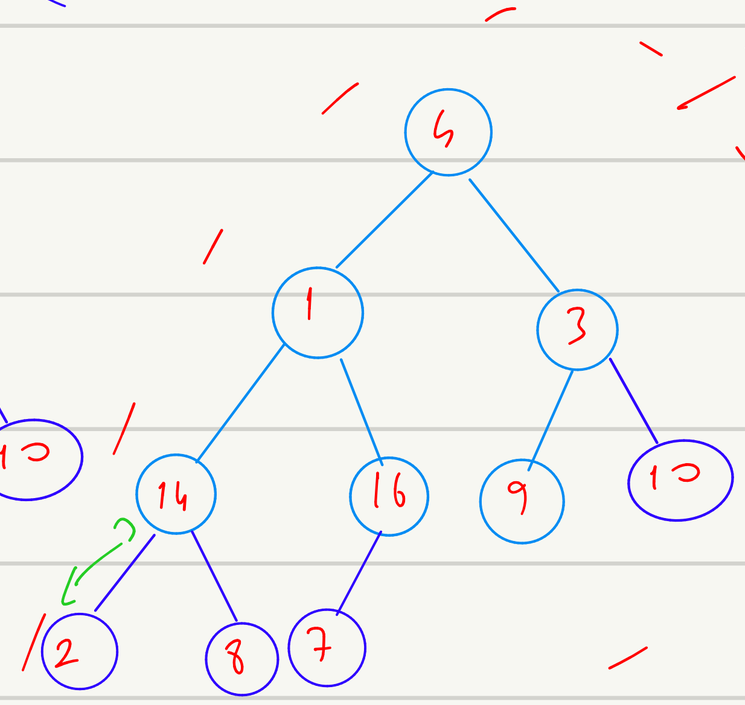


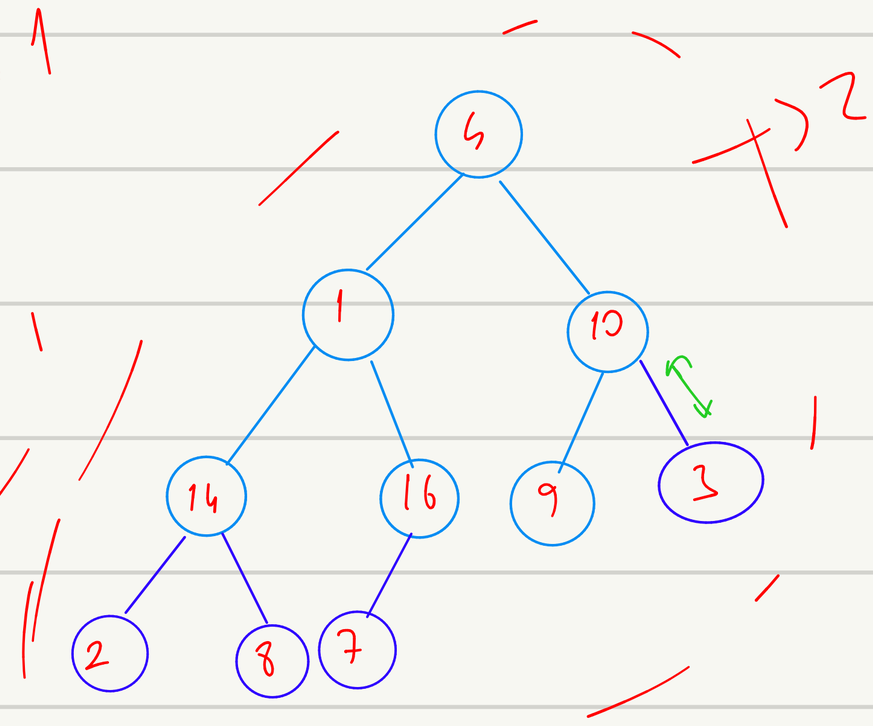


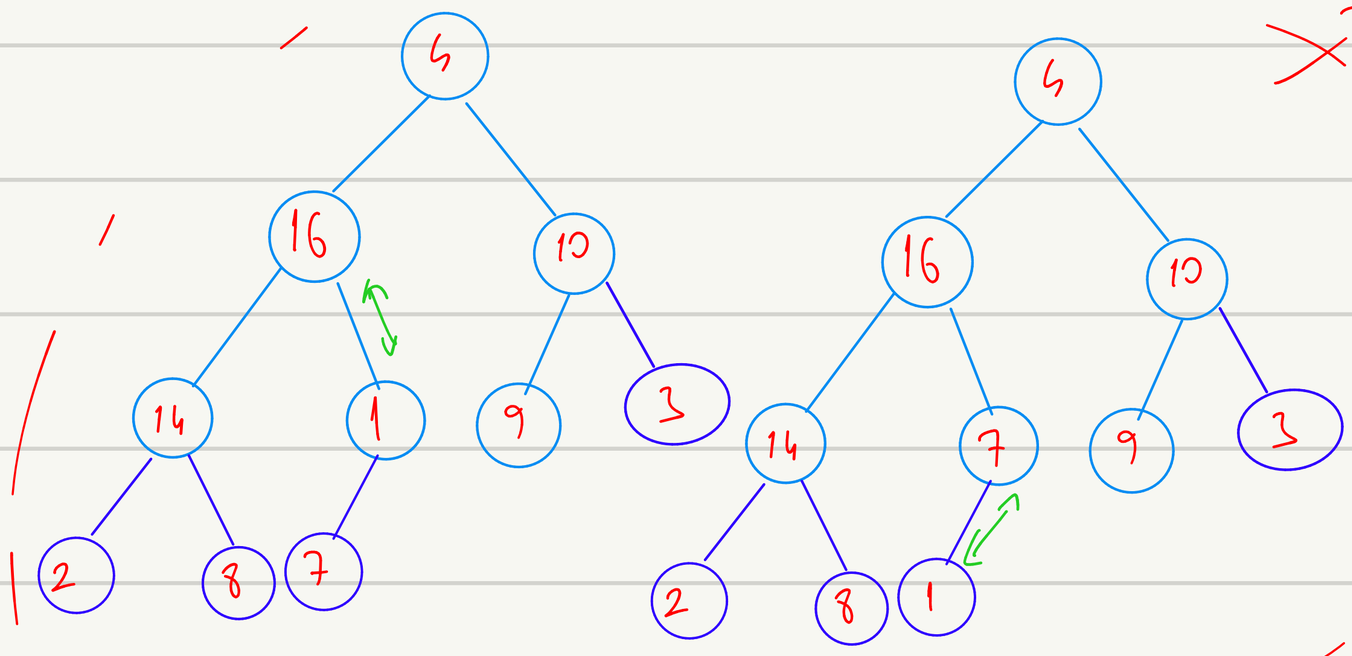


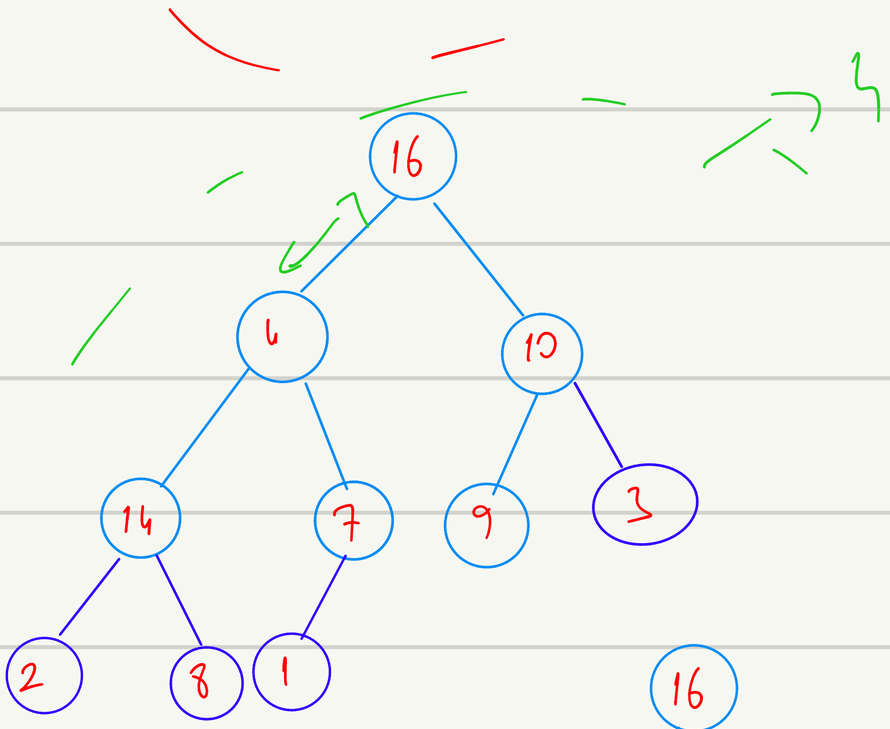
Bu veri yapısı işlem sırası önemini garanti ediyor ve işlem işlem sırası için kullanılır.

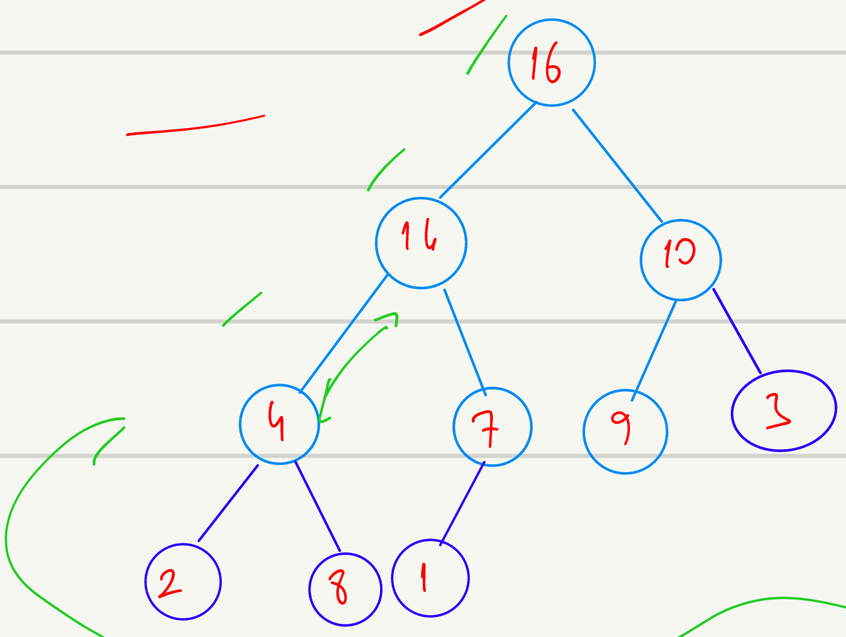


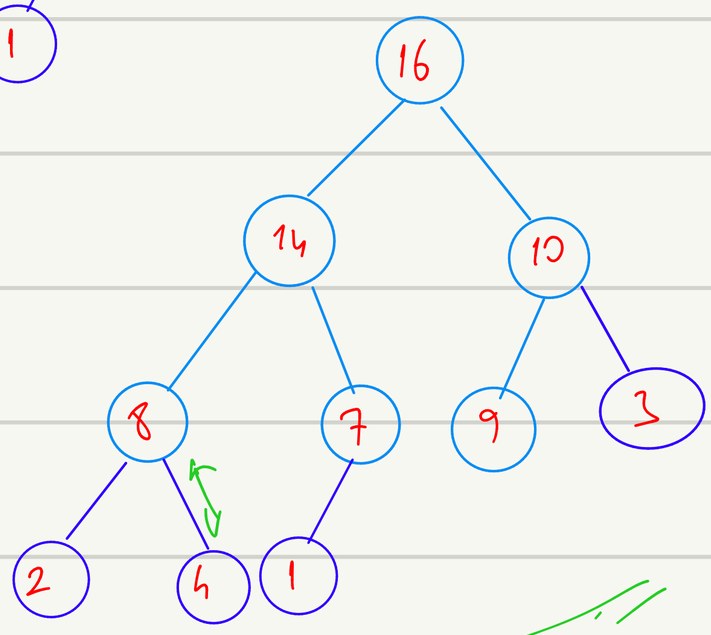






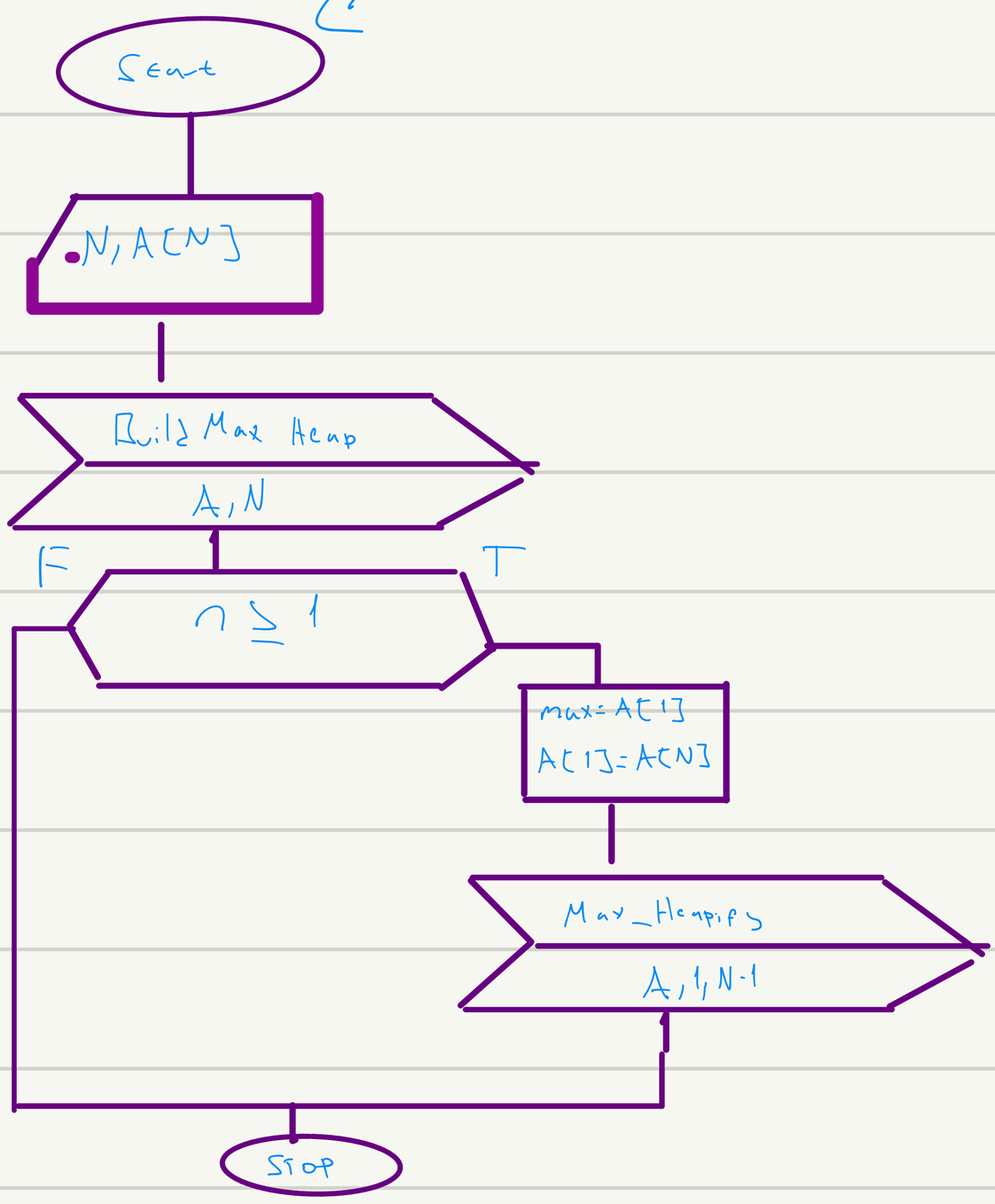




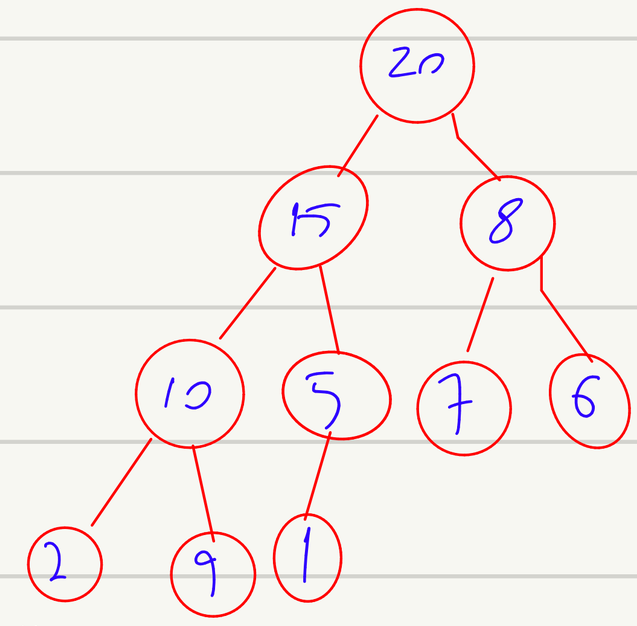


ilk elemanı silme

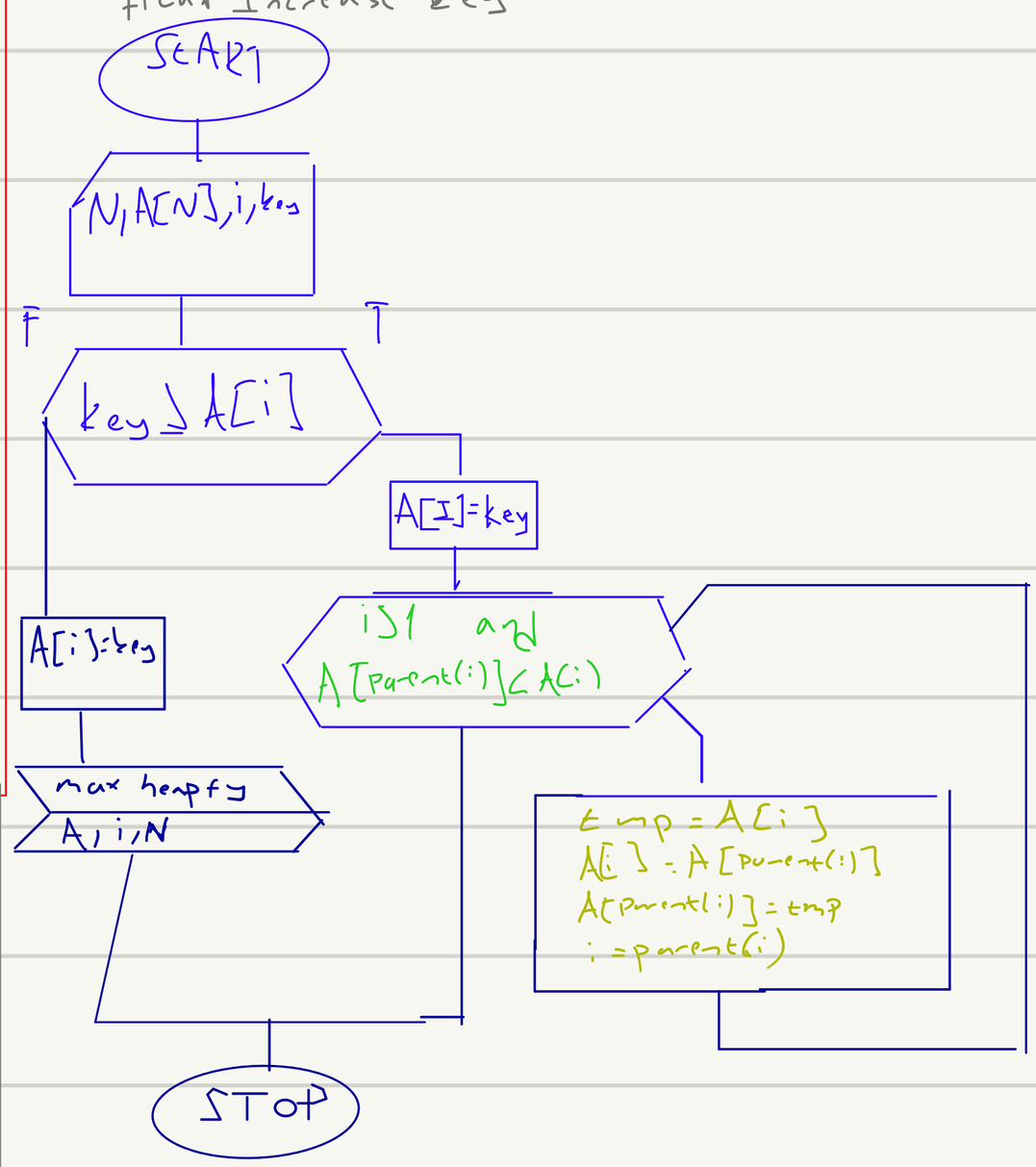
Heap Extract MAX



eleman güncelleme

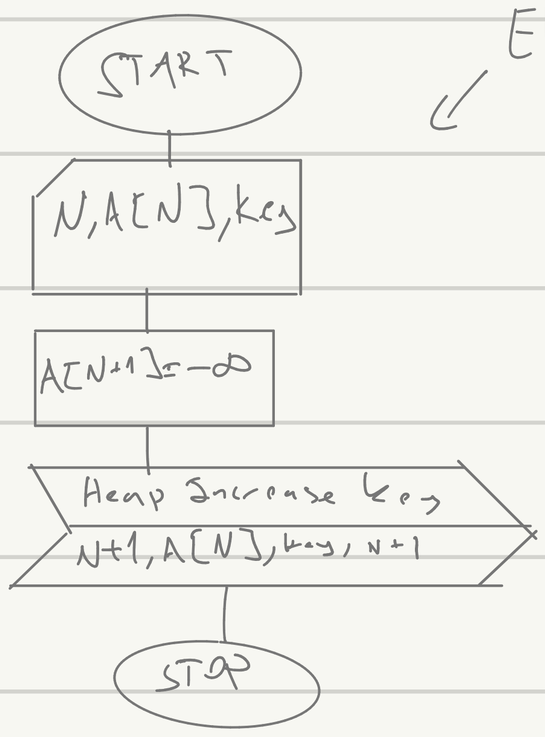


Heap increase key

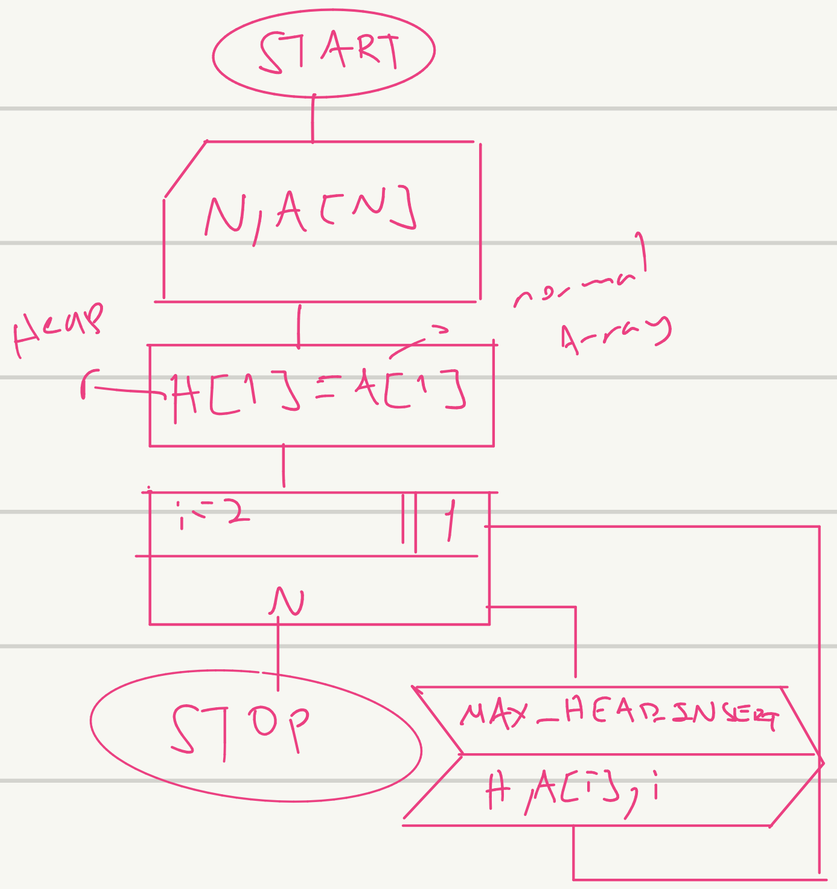


MAX HEAP INSERT

Eleman ekleme



BUILD_MAX_HEAP_TOP_DOWN



11.04.22
7.

Heap Sort

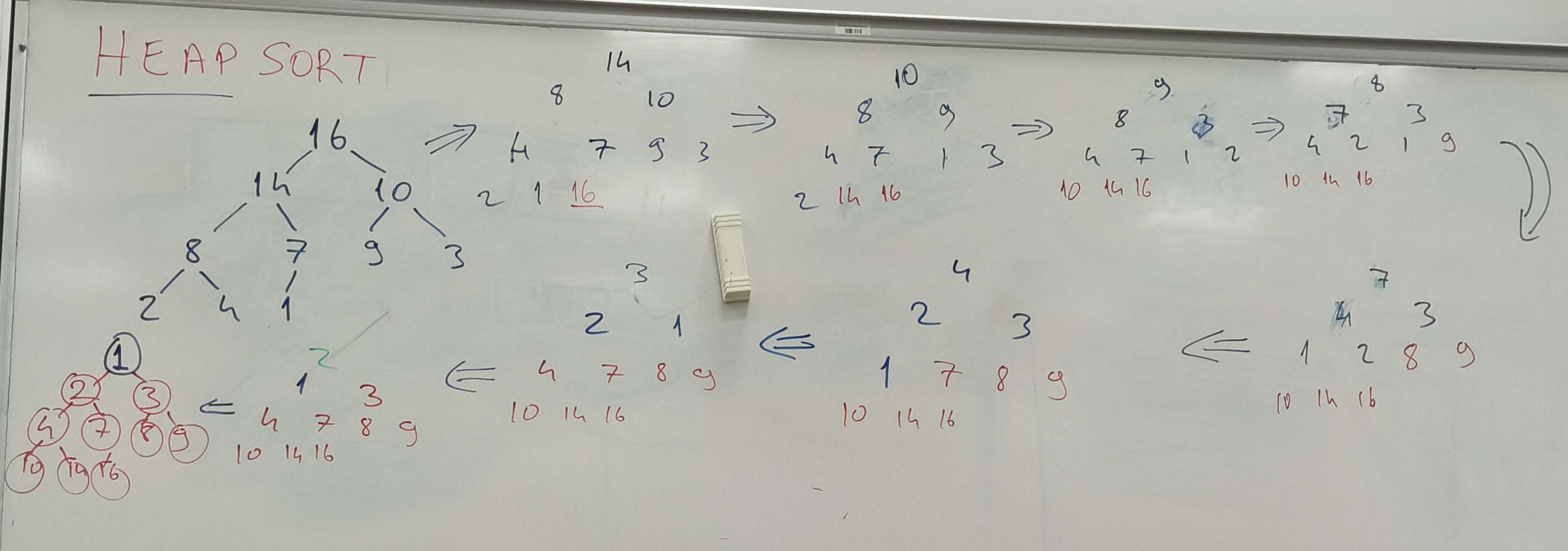
2-9-7-6-5-8 → Heapify → 9-6-8-2-5-7 → max elemanı N. elemanla değiştir → 7-6-8-2-5-9 → heapSize'ı 1 azalt → tekrar heapify ea bi size 1 olana kadar tekrarla

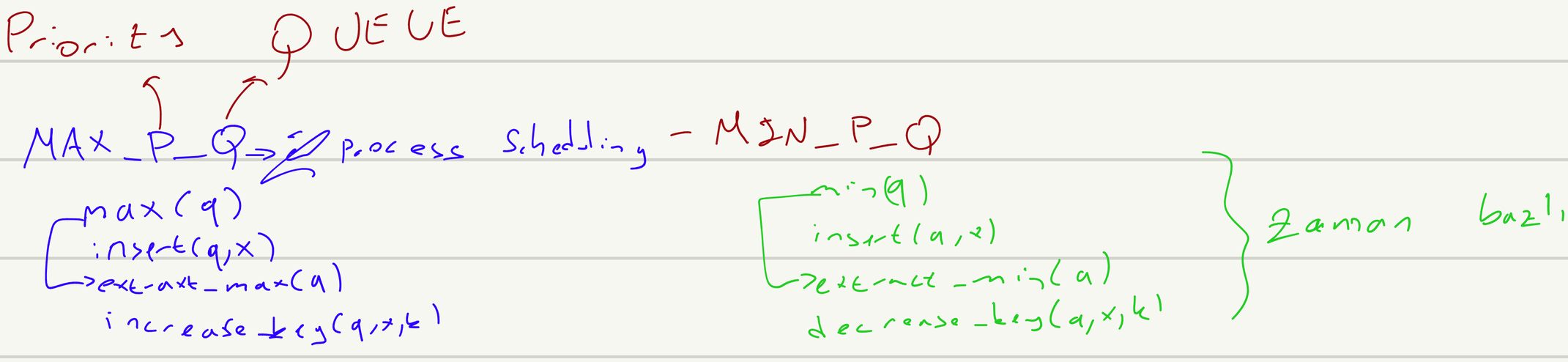
HEAP_SORT(A,n)

1. BUILD_MAX_HEAP(A,n)
2. for i←n downto 2 do → N
   2.1 swap(A[i],A[n])
   2.2 MAX_HEADIFY(A,1,i-1) → logN

} NlogN → O(NlogN)

HEAP SORT


16
14 10
8 7 9 3
2 4 1
1
2 3
4 7 8 9
10 14 16
14
8 10
4 7 9 3
2 1 16
10
8 9
4 7 1 3
2 14 16
9
8 3
4 7 1 2
10 14 16
8
7 3
4 2 1 9
10 14 16
3
2 1
4 7 8 9
10 14 16
4
2 3
1 7 8 9
10 14 16
7
4 3
1 2 8 9
10 14 16




```
int tree_array_size=20;
int heap_size=0;
const int inf=100000;

void swap(int * a) int * b){
*a+=*b;
*b=*a-*b;
*a-=*b;}

int get_right_child(int * a) int index){
        if( ((2*index)+2)<tree_array_size) && index>=1)
            return (2*index)+2;
        return -1;}

int get_left_child(int * a) int index){
        if( ((2*index)+1)<tree_array_size) && index>=1)
            return (2*index)+1;
        return -1;}

int getParent(int * a) int index){
     if(index>0 && index<tree_array_size)
              return index/2;
        retrun -1;}
```

```
void maxHeapify(int * a) int index){
    int l=get_left_child(a)index);
    int r=get_right_child(a)index);
    int largest=index;
    if(l<heap_size && l>=0)
        if(a[l]>a[largest])
            largest=l;
     if(r<heap_size && r>=0)
        if(a[r]>a[largest])
            largest=r;
     if(index!=largest){
        swap(&a[largest])&a[index]);
        maxHeapify(a)largest);}}

void buildMaxHeap(int * a){
    int i;
    for(i=heap_size/2;i>=0;i--){
        max_heapify(a)i);}

int extractMax(int * a){
    int max=a[0];
    a[0]=a[heap_size-1];
    heap_size--;
   maxHeapify(a)0);
    return max;}

void increaseKey(int * a)int index)int key){
      a[index-1]=key;
      while(index>=0 && a[get_parent(a)index)]<a[index]){
         swap(&a[index])&a[get_parent(a)index)]);
         index=get_parent(a)index);}}

void decreaseKey(int * a)int index) int key){
       a[index-1]=key;
       maxHeapify(a)index);}

void insert(int * a) int key){
        heap_size++;
       a[heap_size]=-1*inf;
        increase_key(a)heap_size)key);}
```

```
int main(){
    int a[tree_array_size];
    insert (a)20);
    insert(a)15);
printf("%d")extractMax(a);
    return 0;}
```

a - 97 0110 0001

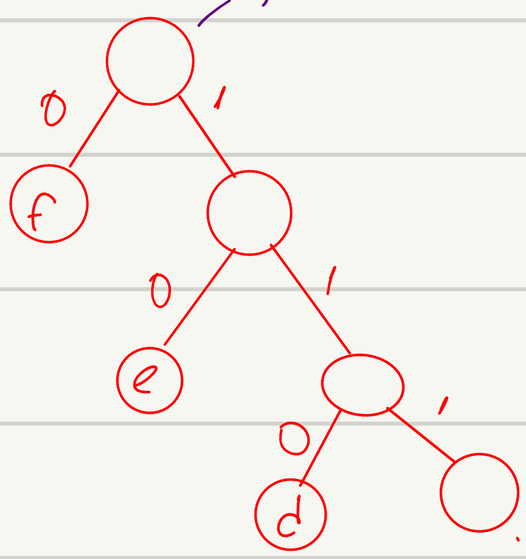
b - 98 0111 0010

c - 99 0110 0011

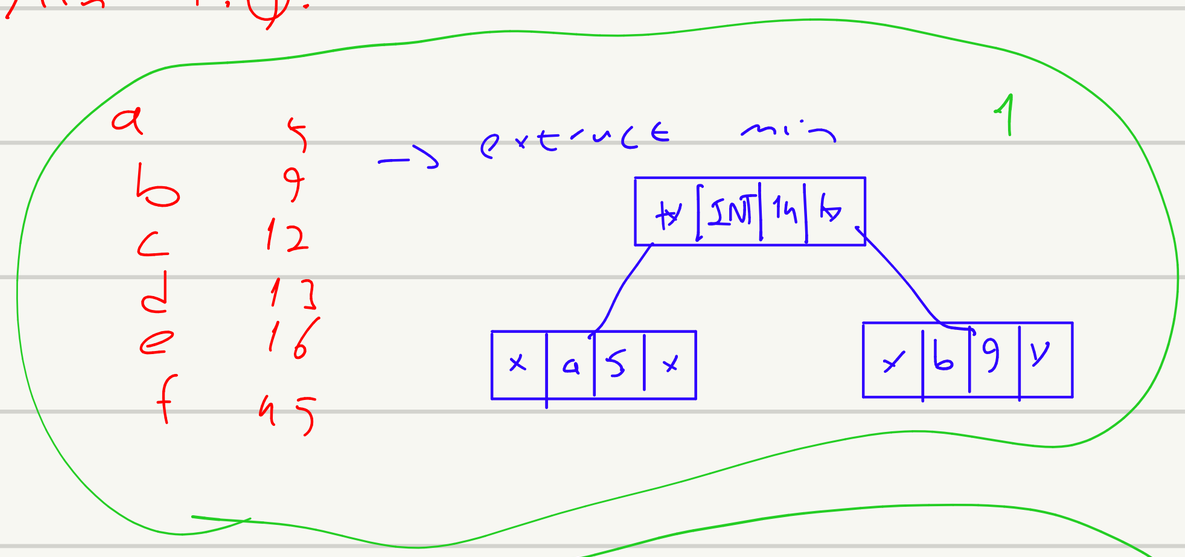
d - 100 0110 0100

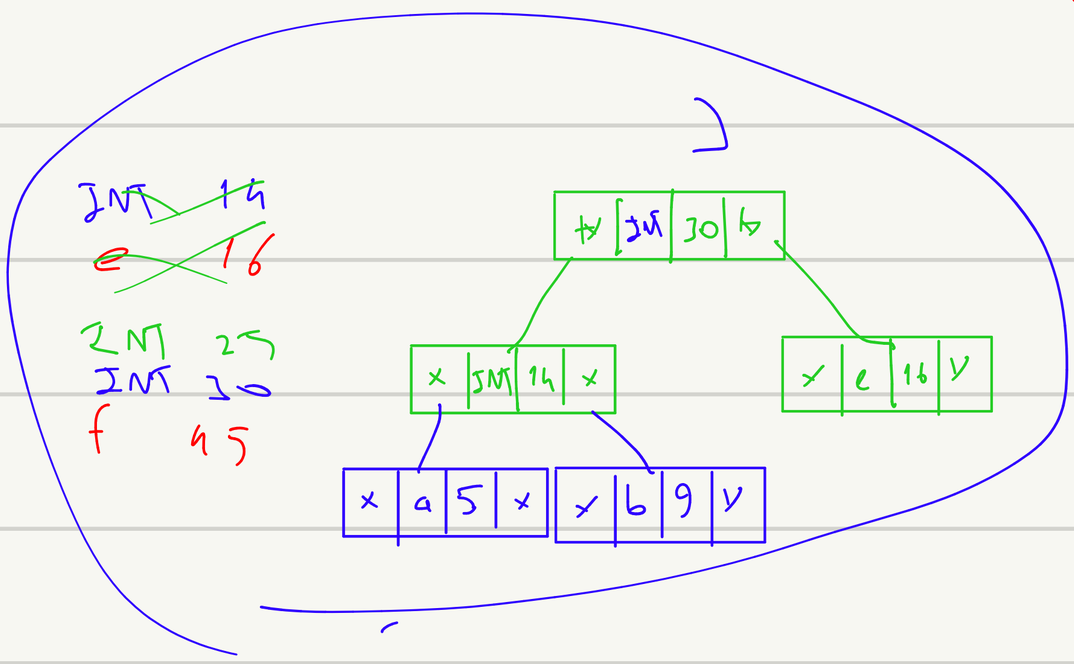
karakter	frekans
a	5
b	9
c	12
d	13
e	16
f	45

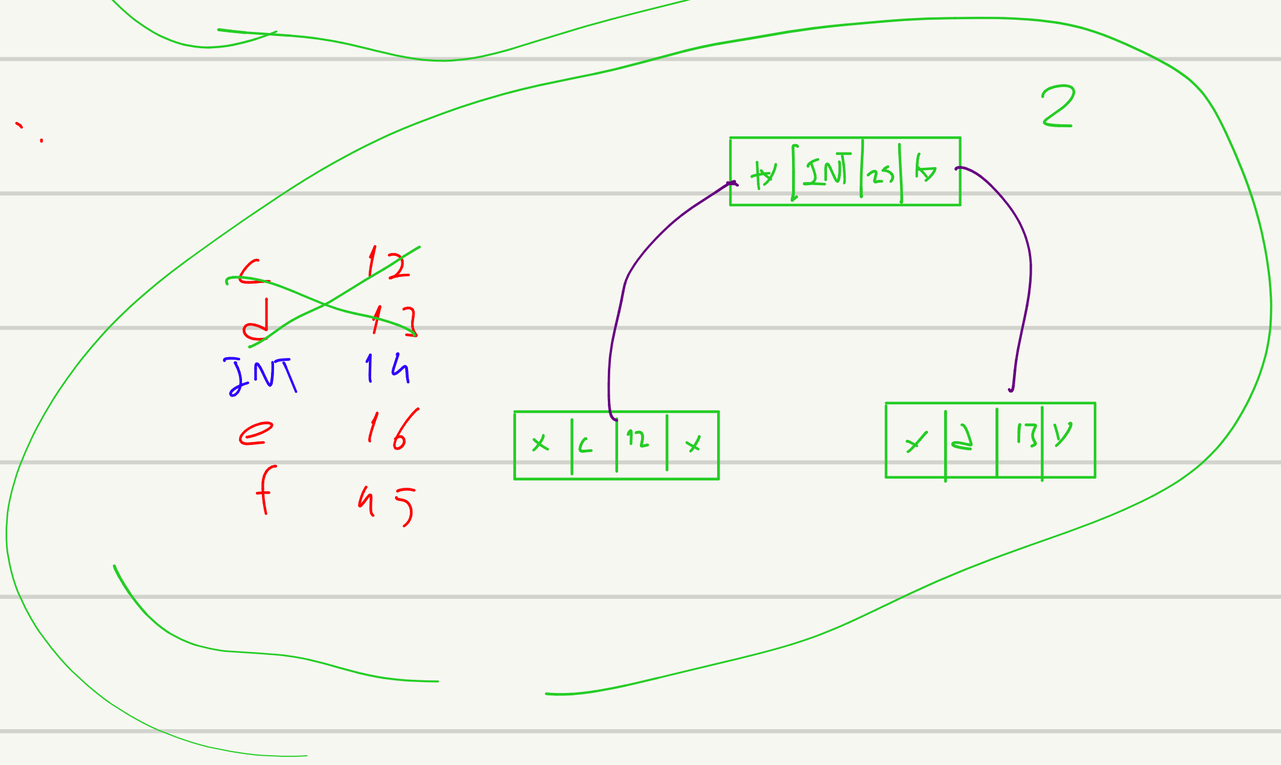
-> Huffman

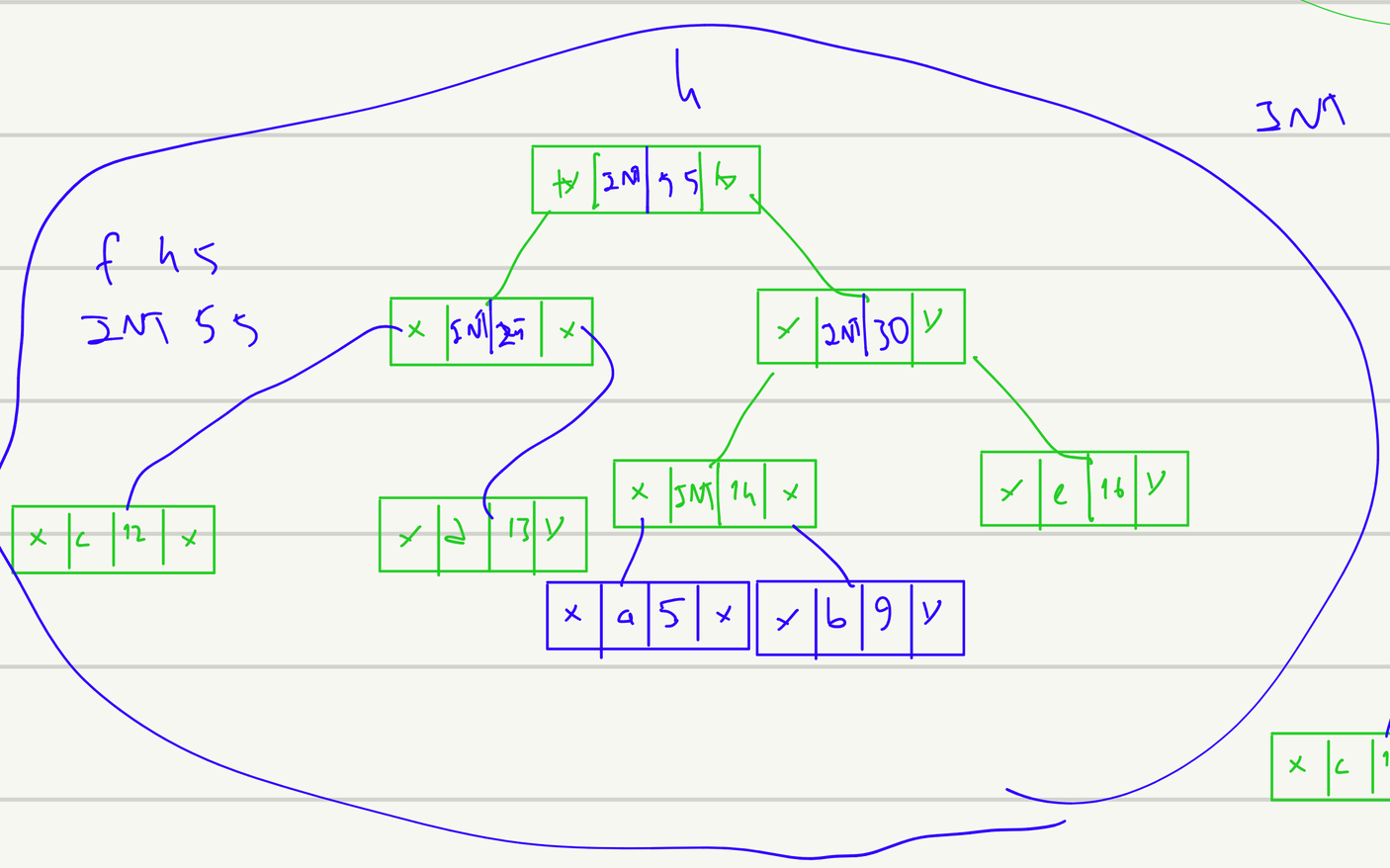


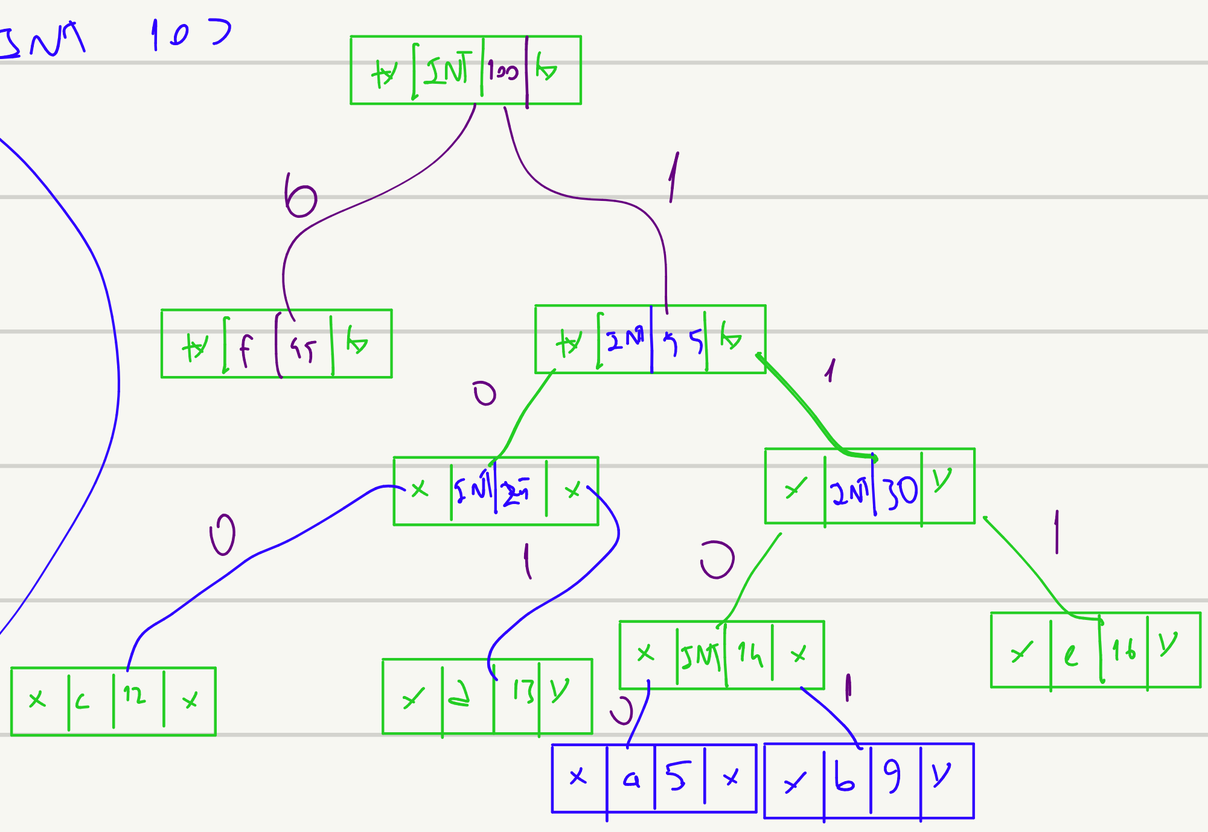
Min P.Q.









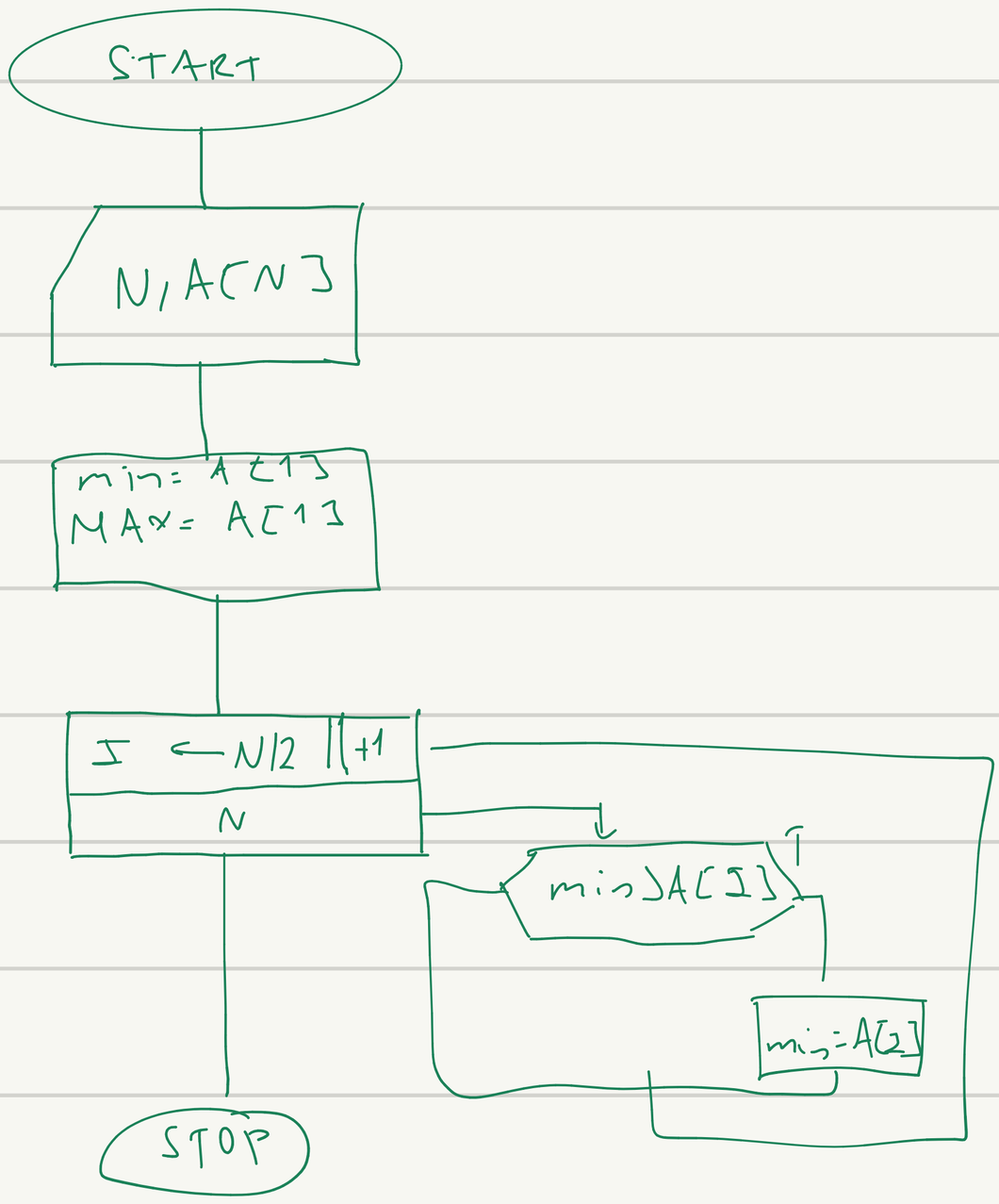


0 1100 100 1100

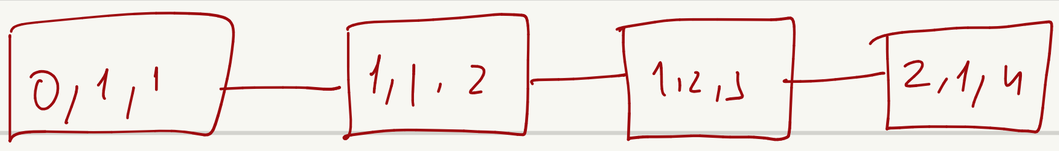
f a c a

LAB 2
13.04.22

Full dolu MAX_HEAPde en büyük ve en küçük değeri bulan akış



Sparse matrisin linkli liste şeklinde verilmiş halinde tüm bir kolon 0'sa o sütunun silinmesi gerekmektedir.




```
int * dizi;

dizi = (int*) malloc (sizeof (int)*N);
  dizi=0
while (link!=NULL){
dizi[link->col ]=1
link=link->next }

for(i=0; i<N; i++){
    if( dizi[i]==0){
        printf("%d", i);
                    }
```

25.04.22

8.

Graflar

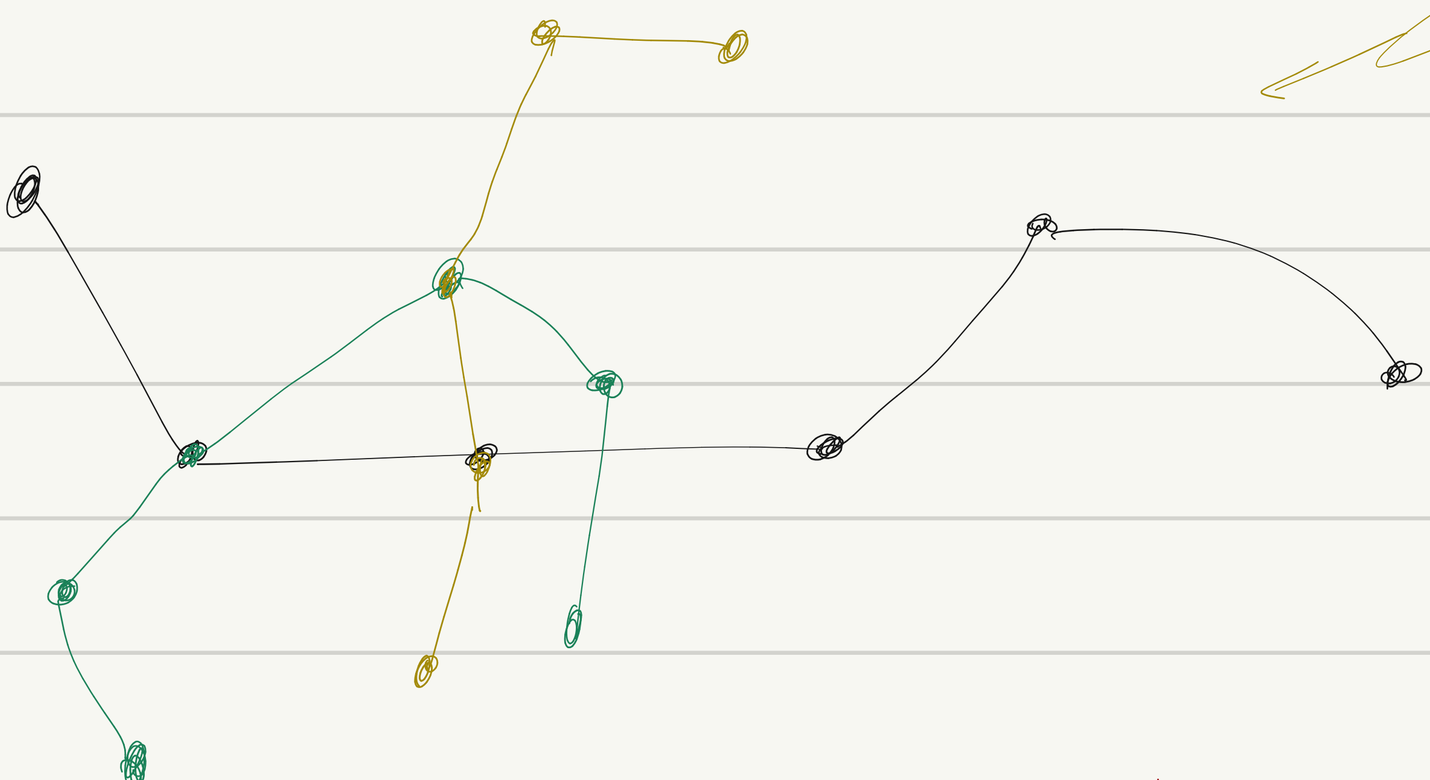
$G(V, E)$

Vertex → $|V|$

Edge → $|E|$

$O(VE)$, $O(V \log E)$

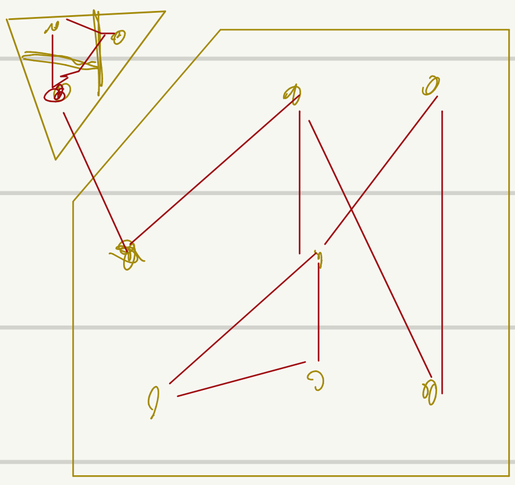
g.b. gösterilir.

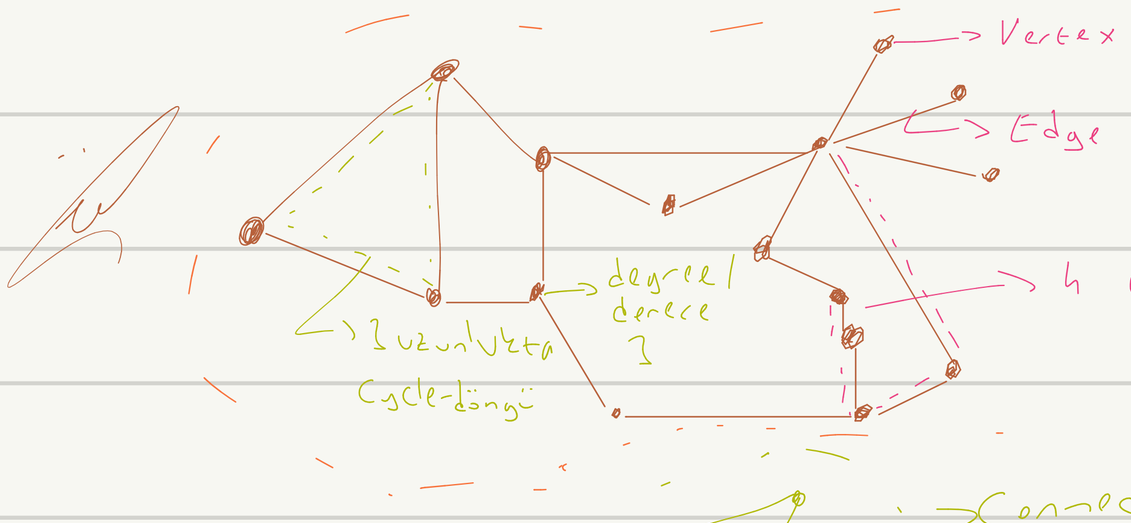


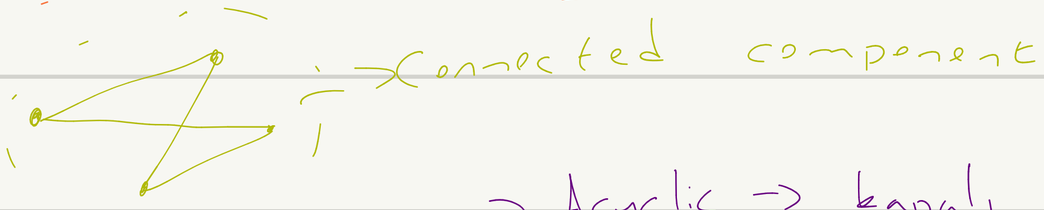
← Kuragollar

Web sayfası, Face arkadaşları, Haritalar

Baskılı devreler, Biyoenformatikte proteinler
↳ kısa devre var mı vs.
Zaman planlaması, e-ticaret vb. çok çok uğlar



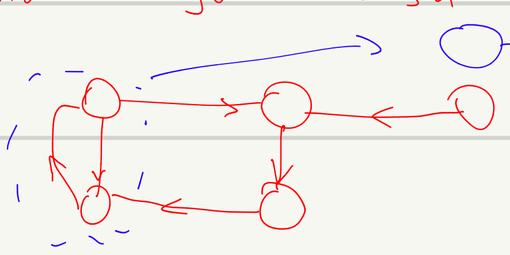




Graf tipleri

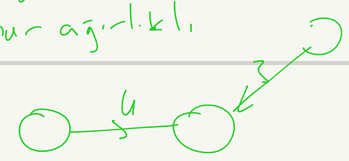
→ Acyclic → kapalı çevrim içermeyen
↳ Ağaç gibi

Yönlü
Düğümler arasındaki yolların yönü varsa



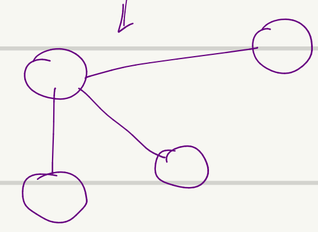
→ böyle gösterilir

Kenar ağırlıklı



Yönsüz
Düğümler arasındaki yolların yönü yoksa
şeklidir

Kenar-Ağırlıksız



Acyclic graf varsa — Kenar Sayısı V-1

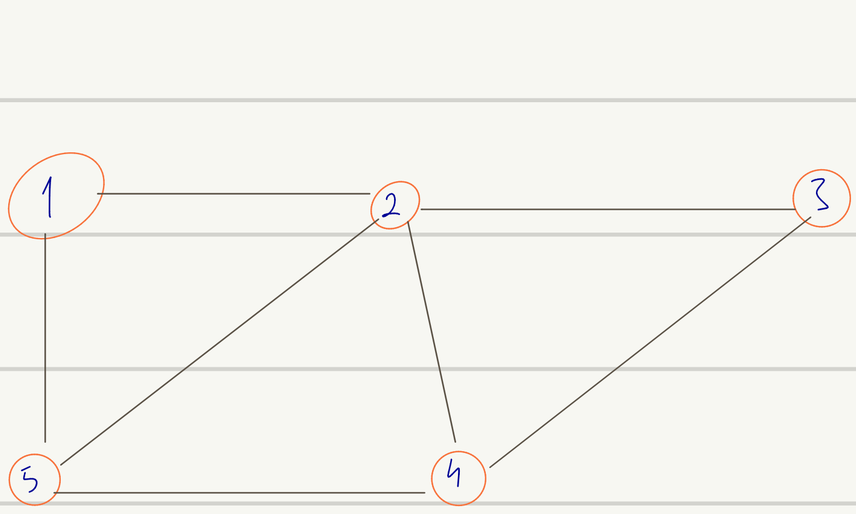
Kenar eklen diğinde çevrim oluşmalı — tüm bileşenler bağlı

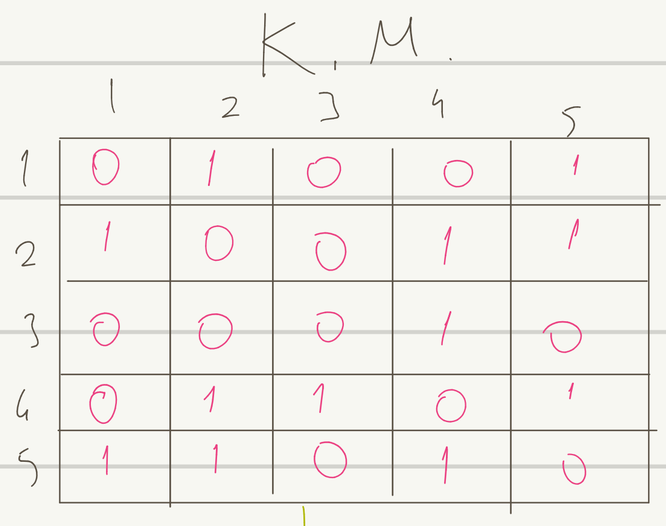
Her düğüm çiftini başka bir düğüm bağlanmalı, Kenar kaldırmak ağacı keser

Graf gösterimi

1-) Komşuluk listesi (Adjacency list)

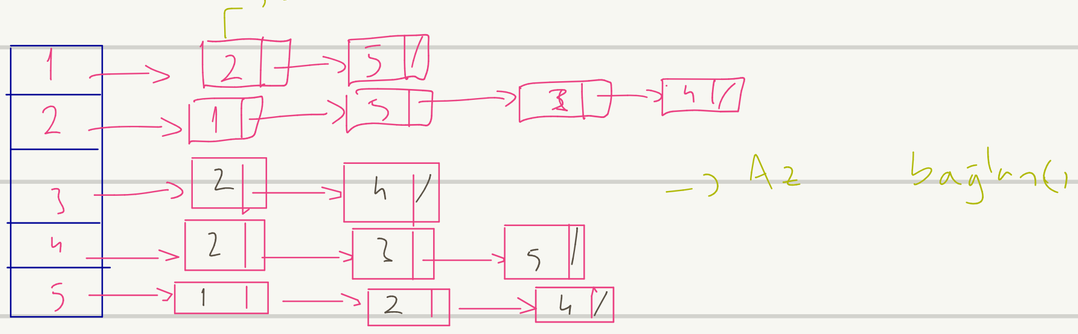
2-) komşuluk matrisi (Adjacency matrix)



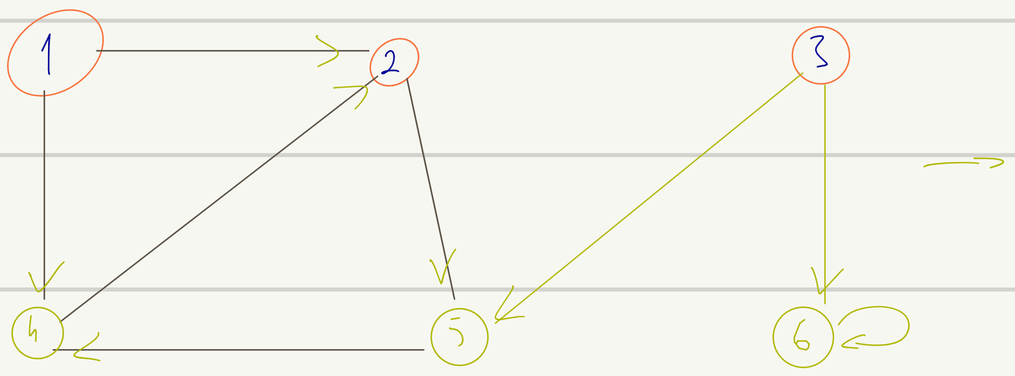


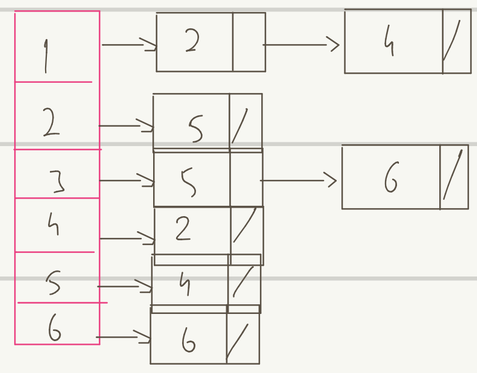
→ çok bağlantı

→ K.L.



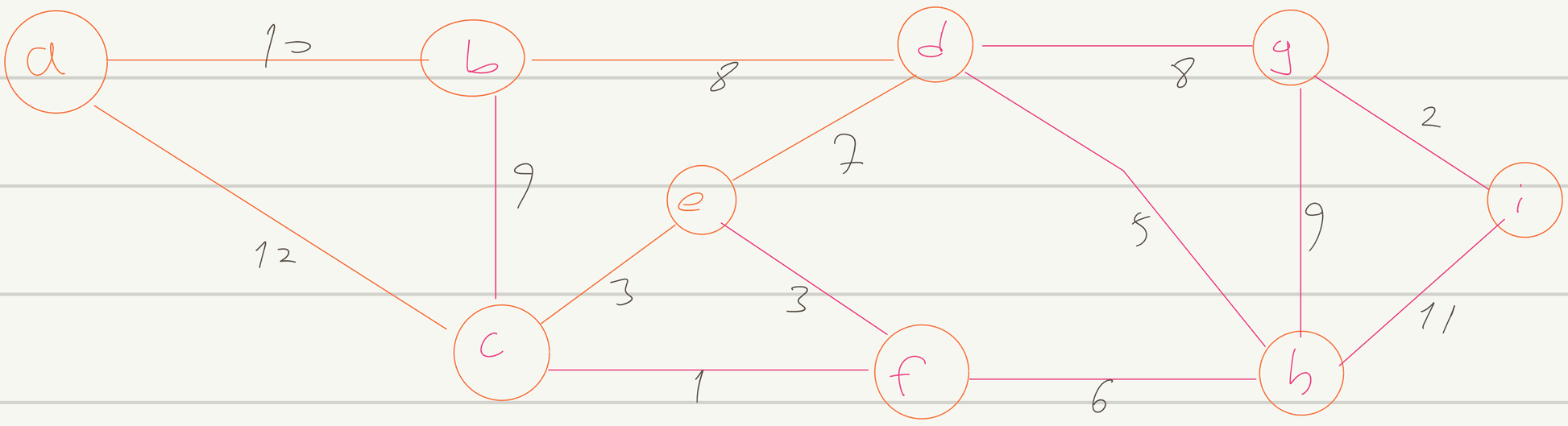
→ Az bağlantı





	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Soru → En ucuz maliyetle herkesi Bağlamak



Minimum Spanning Tree

$G(V,E)$ için $w(u,v)$ maliyeti tanımlı. $\min(\sum w)$ $T \subseteq E \Rightarrow \min(\sum w)$

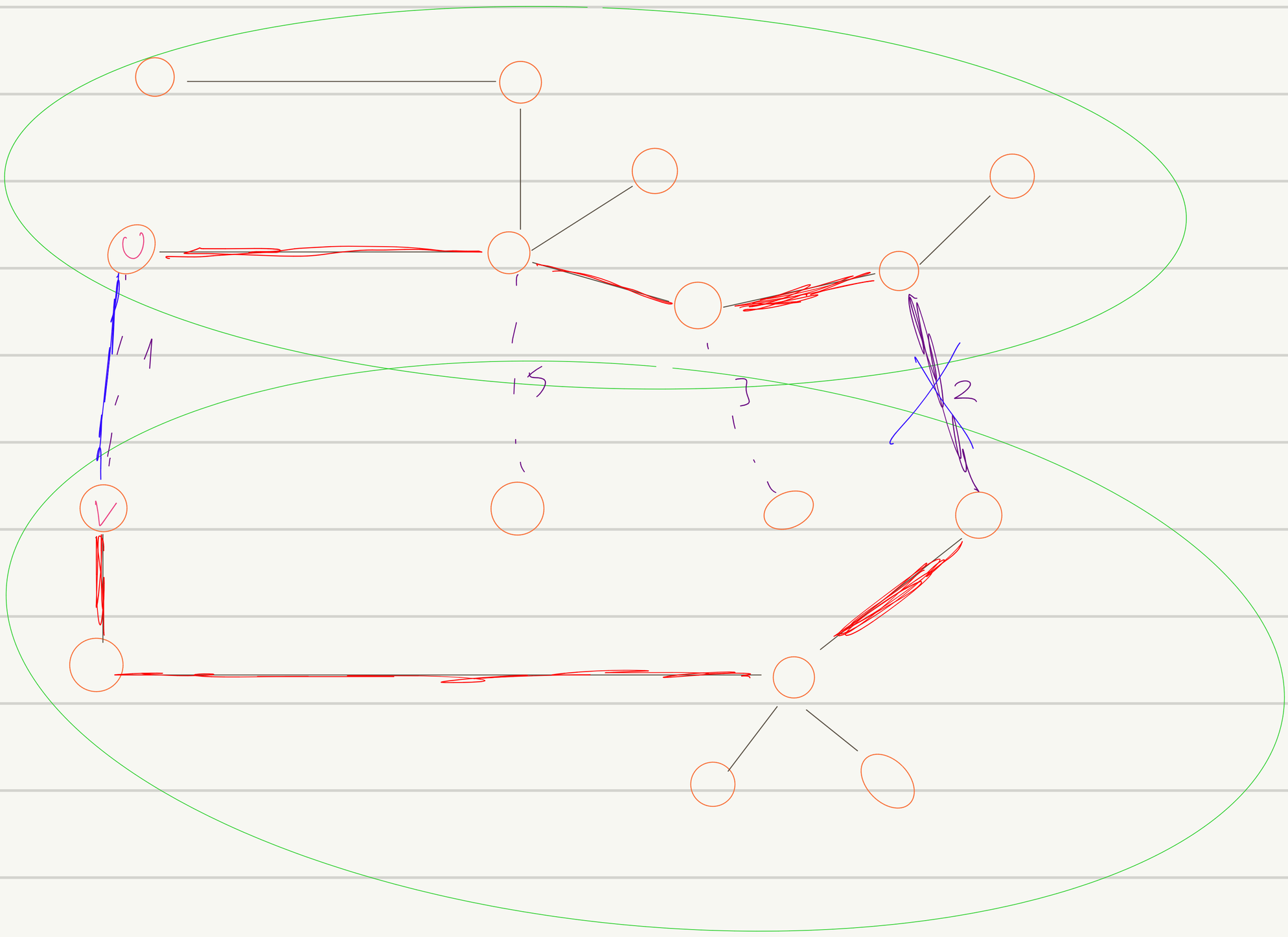
Ağaç olması için $|V|-1$ kenar

greedy algorithm

Önce en küçük kenar uzunluğunu bulacağız bunu sürekli yapacağız

↳ burada cfe gi dh ...

Not → 1 graf için 1'den çok MST oluşabilir. Eşitliklerden dolayı.



Kruskal's MST algoritması → $O(E \log V)$ — Bunu bilsek yeter

Kruskal(V, E, w) ← Ağırlık; ← Kenar; ← Düğüm

MST ← ∅ ← O(1)

for each v ∈ V ← O(V)

MakeSet(v)

Sort E by w $O(E \log E)$

$O(E)$ ← foreach (u,v) kaçıncı çevrim kontrol

if FindSet(u) = findset(v)

then MST ← MST ∪ (u,v)

Union(u,v)

return MST

Sort

ef

gi 2

ce 3

ef 3

dh 5

MST

c

f

g

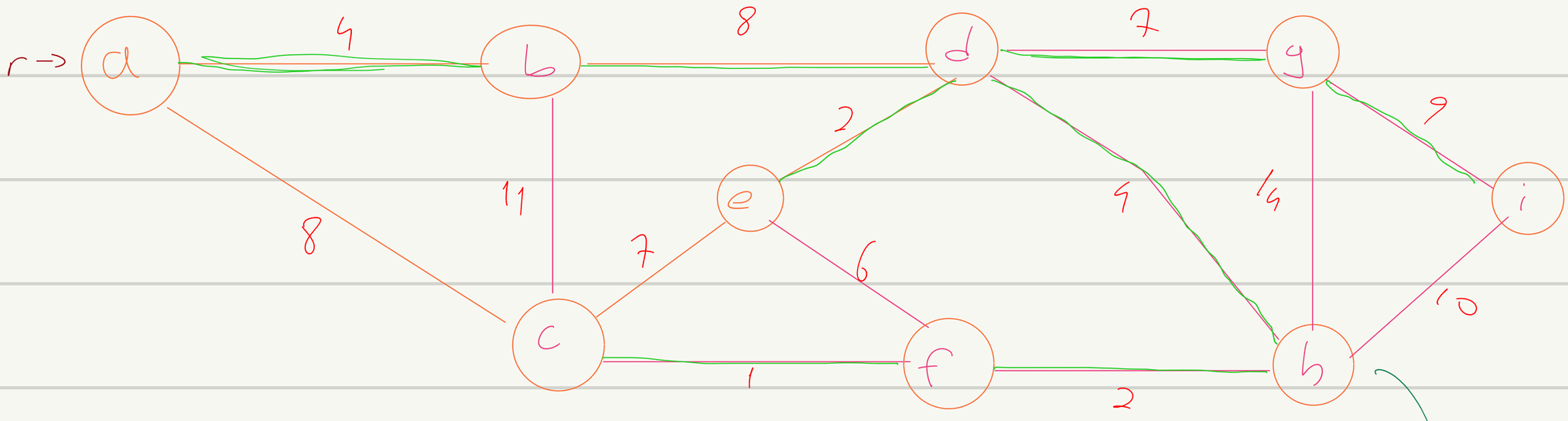
i

e

d

h

ac 12



PRİM Algoritması

Bir düğümden başlayıp en küçüğüne sırayla başla

tam doğru değil

PRIM(V,E,W,r) → $O(E \log V)$

PQ ← ∅ ← O(1)

for each VC ← $O(V \log V)$

key[u] ← ∞

Parent[u] ← NULL

Insert(PQ,u)

Decrease_key(PQ, r, 0) //key[r] ← u ← $\log V$

while (PQ != ∅)

u ← EXTRACT MIN(PQ) ← $O(V \log V)$

foreach v ∈ Adj[u] ← $O(E \log V)$

if (v ∈ PQ) And w(u,v) < key[v]

parent[v] ← u

decrease key (Q, V, w(u,v))

ab 4

dh 4

ac 8

bd 8

bc 11

dg 7

dh 6

de 2

ef 6

ec 7

hf 2

hi 10

hg 14

fc 1

gi 9

a—b

b—d

d—e

d—h

h—f

f—c

d—g

g—i

Kruskal's MST algoritması → $O(E \log V)$ — Bunu bilsek yeter

Kruskal(V, E, w) ← Ağırlık; ← Kenar; ← Düğüm

MST ← ∅ ← O(1)

for each v ∈ V ← O(V)

MakeSet(v)

Sort E by w $O(E \log E)$

$O(E)$ ← foreach (u,v) kaçıncı çevrim kontrol

if FindSet(u) = findset(v)

then MST ← MST ∪ (u,v)

Union(u,v)

return MST

Sort

ef

gi 2

ce 3

ef 3

dh 5

MST

c

f

g

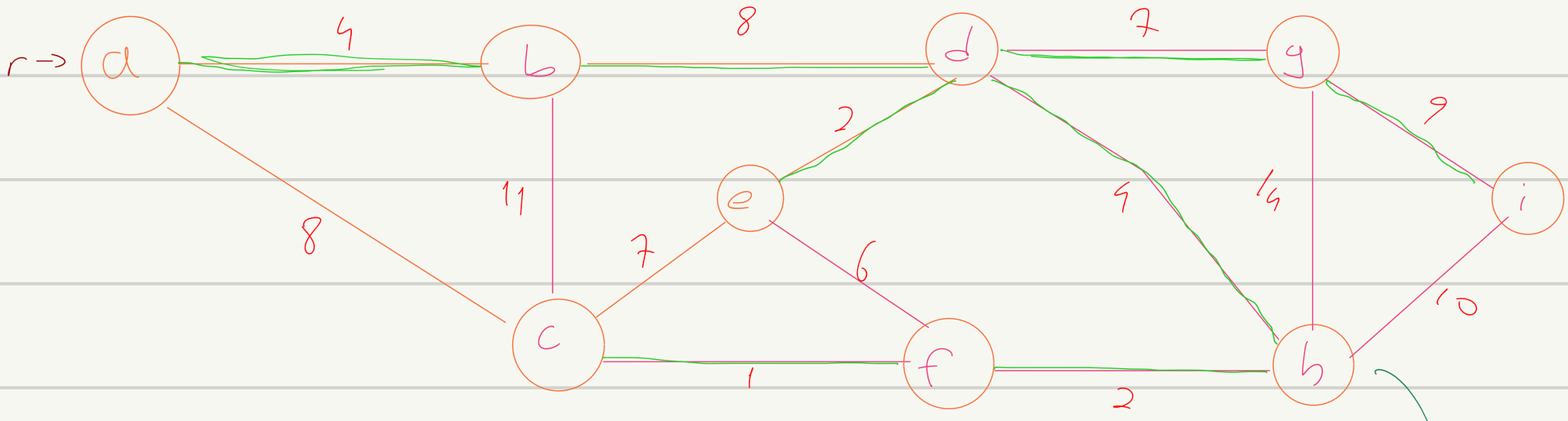
i

e

d

h

ac 12



PRİM Algoritması

Bir düğümden başlayıp en küçüğüne sırayla başla

tam doğru değil

PRIM(V,E,W,r) → $O(E \log V)$

PQ ← ∅ ← O(1)

for each VC ← $O(V \log V)$

key[u] ← ∞

Parent[u] ← NULL

Insert(PQ,u)

Decrease_key(PQ, r, 0) //key[r] ← u ← $\log V$

while (PQ != ∅)

u ← EXTRACT MIN(PQ) ← $O(V \log V)$

foreach v ∈ Adj[u] ← $O(E \log V)$

if (v ∈ PQ) And w(u,v) < key[v]

parent[v] ← u

decrease key (Q, V, w(u,v))

ab 4

dh 4

ac 8

bd 8

bc 11

dg 7

dh 6

de 2

ef 6

ec 7

hf 2

hi 10

hg 14

fc 1

gi 9

a—b

b—d

d—e

d—h

h—f

f—c

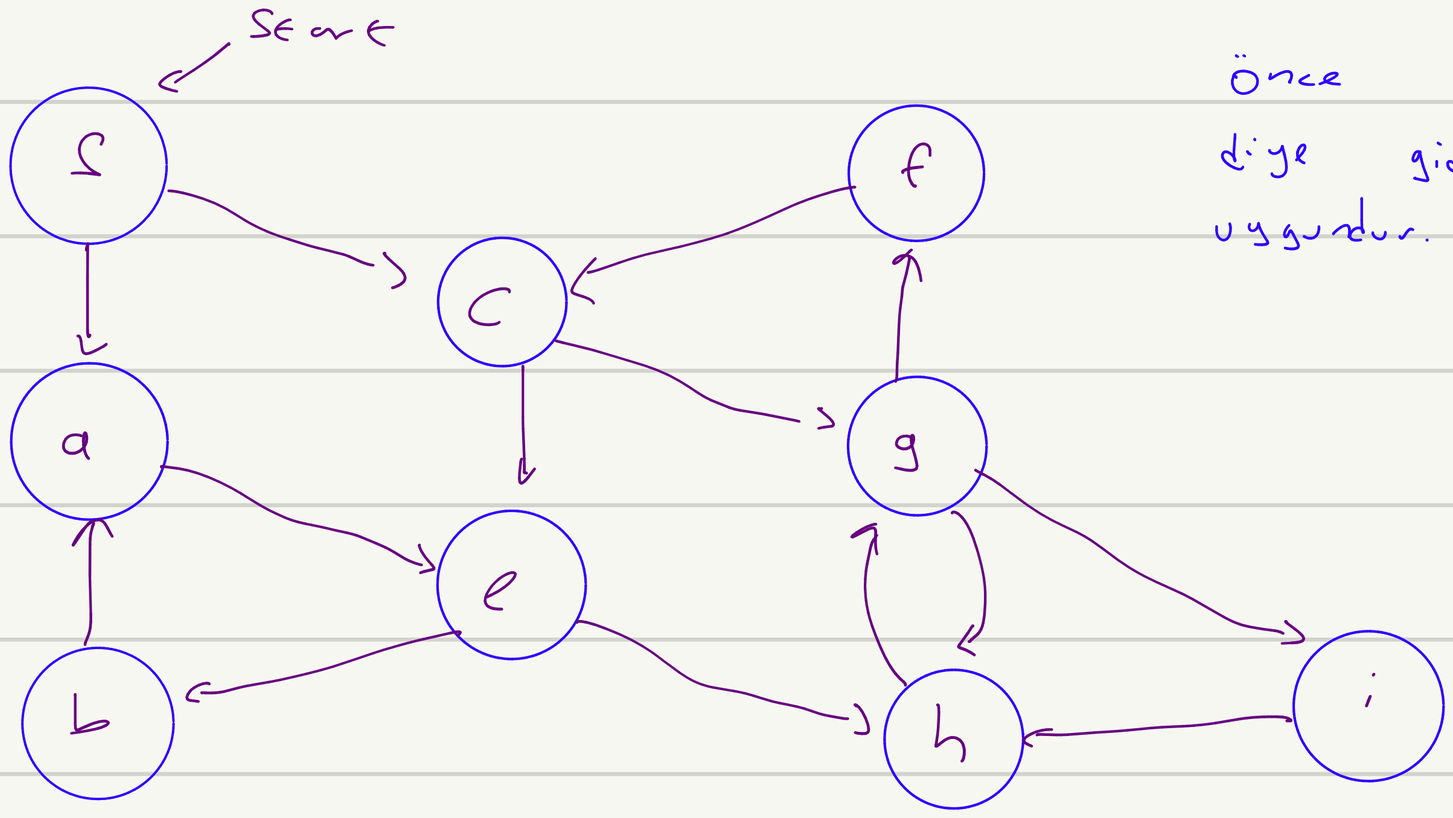
d—g

g—i

07.05.22
9

Graflarda gezinti/arama => BFS-DFS

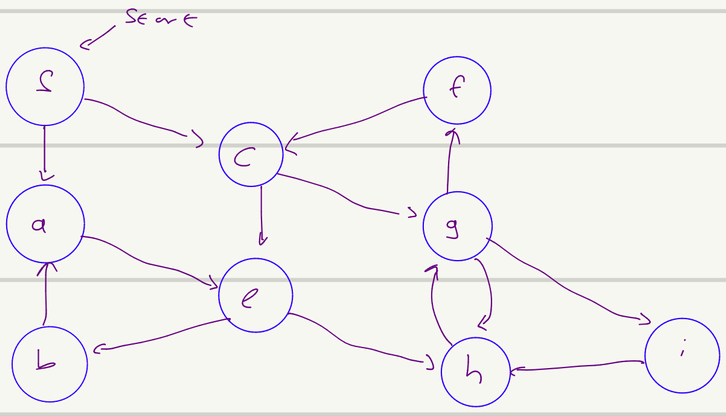
Önce genişlemesine arama - BFS (Breadth first search)



Önce k uzaklığa, sonra k+1, k+2 diye gider. Kuyruk bu iş için uygundur.

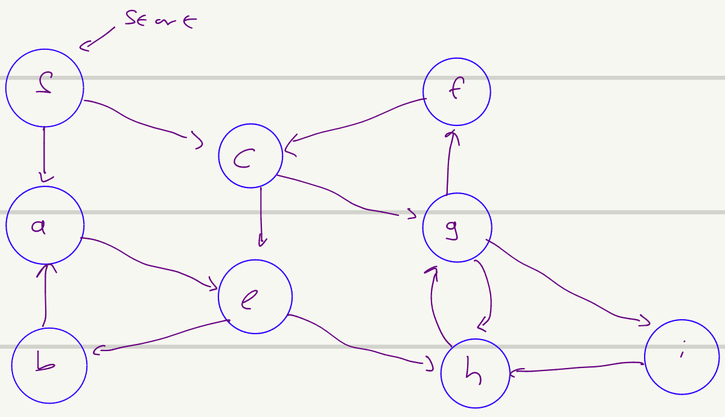
BFS(V,E,s) -> O(V+E)

```
for each u ∈ V-{s} -> O(V)
    do d[u] ← ∞
d[s] ← 0
Q ← ∅
ENQUEUE(Q,s) -> O(1)
while(Q != ∅)                          -> O(V)
    do u ← dequeue(Q)                  -> toplu
    for each v ∈ Adj[u] -> O(E)
    do if(d[v] = ∞)
       d[v] = d[u] + 1
       ENQUEUE(Q,v)
```



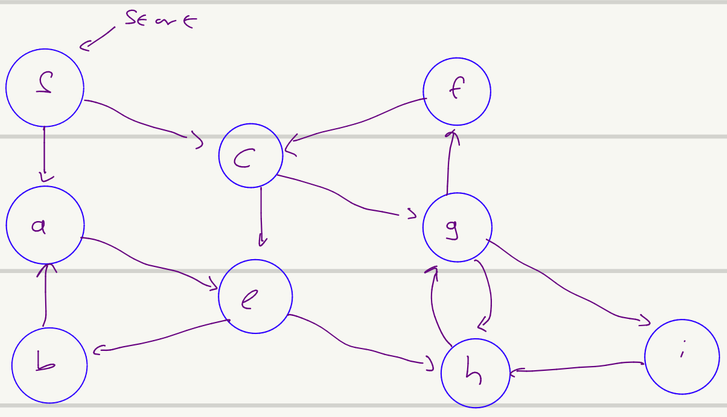
	a	b	c	e	f	g	h	i	s
distance	∞	∞	∞	∞	∞	∞	∞	∞	0

Q = {s}



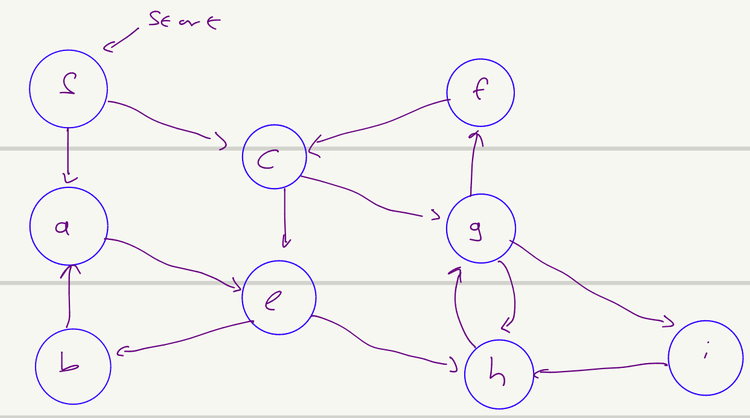
	a	b	c	e	f	g	h	i	s
distance	1	∞	1	∞	∞	∞	∞	∞	0

Q = {a, c}



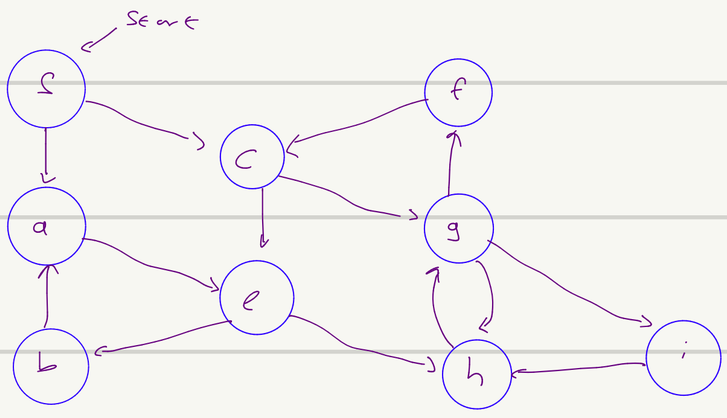
	a	b	c	e	f	g	h	i	s
distance	1	∞	1	2	∞	2	∞	∞	0

Q = {c, e}



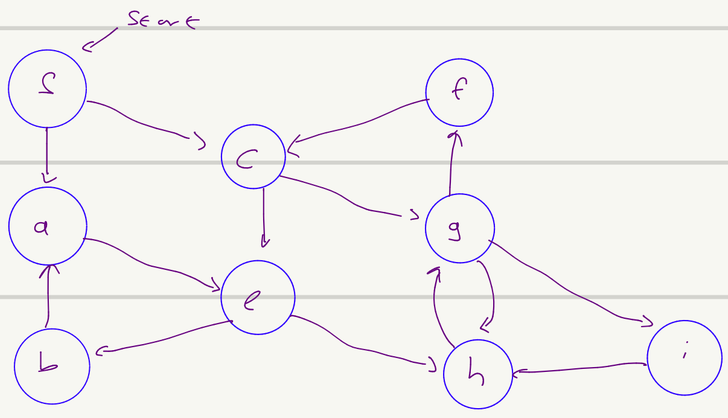
	a	b	c	e	f	g	h	i	s
distance	1	∞	1	2	∞	2	∞	∞	0

$Q = \{e, g\}$



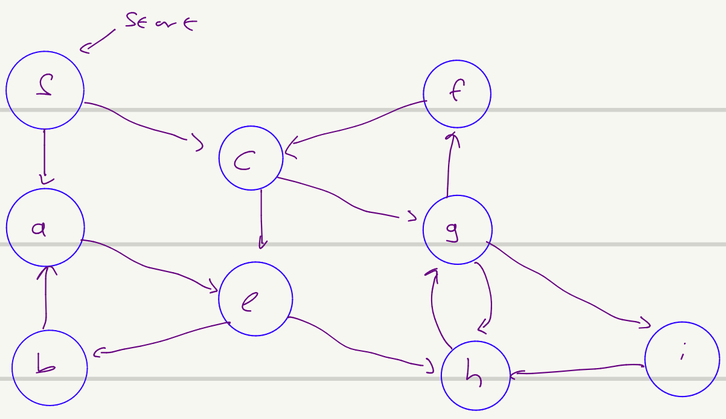
	a	b	c	e	f	g	h	i	s
distance	1	3	1	2	∞	2	3	∞	0

$Q = \{g, b, h\}$



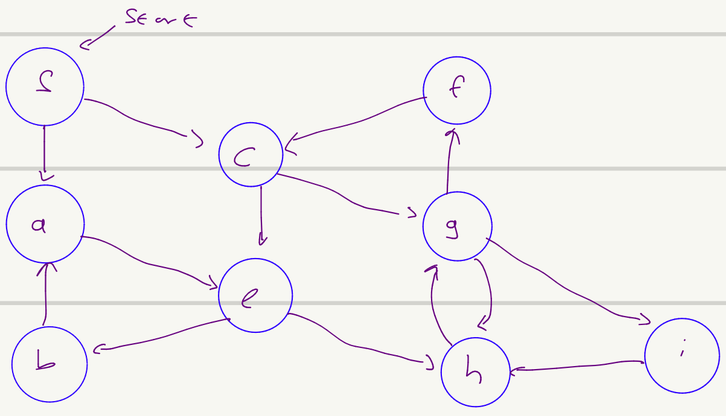
	a	b	c	e	f	g	h	i	s
distance	1	3	1	2	3	2	3	3	0

$Q = \{b, h, i, f\}$



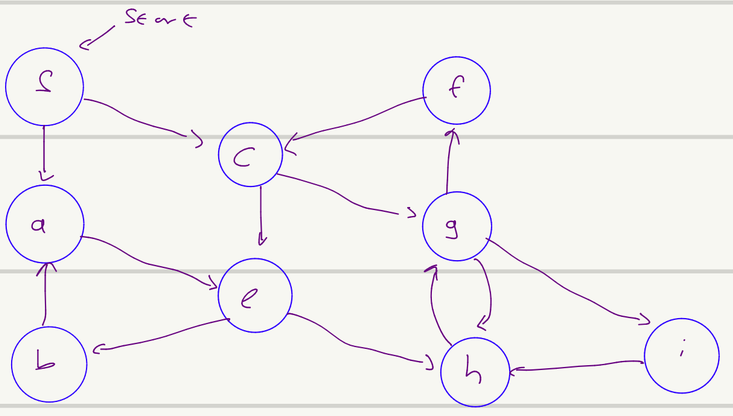
	a	b	c	e	f	g	h	i	s
distance	1	3	1	2	3	2	3	3	0

$Q = \{h, i, f\}$



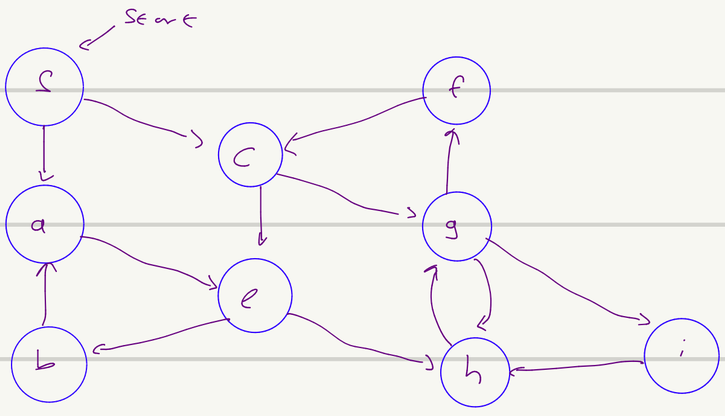
	a	b	c	e	f	g	h	i	s
distance	1	3	1	2	3	2	3	3	0

$Q = \{i, f\}$



	a	b	c	e	f	g	h	i	s
distance	1	3	1	2	3	2	3	3	0

$Q = \{f\}$



	a	b	c	e	f	g	h	i	s
distance	1	3	1	2	3	2	3	3	0

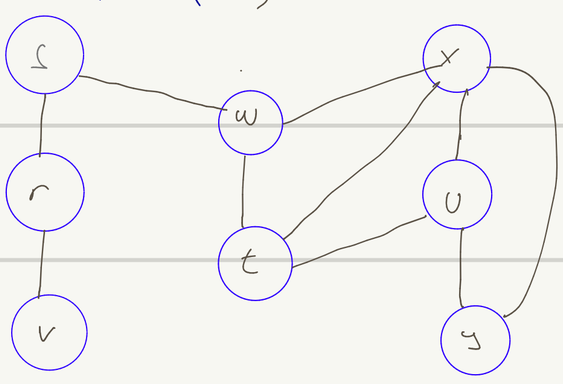
$Q = \{\}$

Artık olarak düğümün iş: bitti mi, düğüme değildi mi gib. Sıyaki kontrol etmiş için.

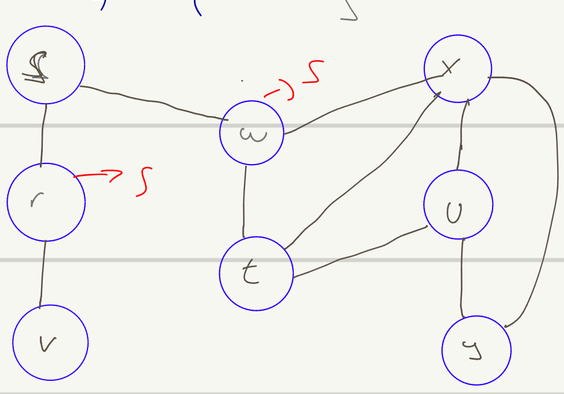
```
Color BFS(V, E, s)
  for each u ∈ V - {s}
      u.color = white
      u.d = ∞
      u.p = null
  s.color = grey
  s.d = 0
  s.p = null
  Q = ∅
  ENQUEUE(Q, s)
  while (Q != NULL)
      u = dequeue(Q)
      for each v ∈ G.Adj[u]
          if v.color = white
              v.color = grey
              v.distance = u.distance + 1
              v.Parent = u
              enqueue(Q, v)
      u.color = black
```

→ boşsalar

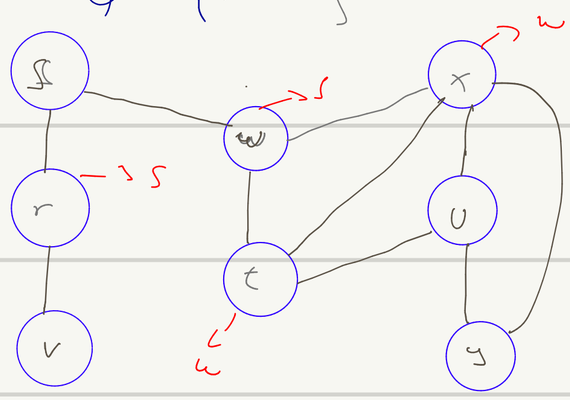
$Q = \{s\}$



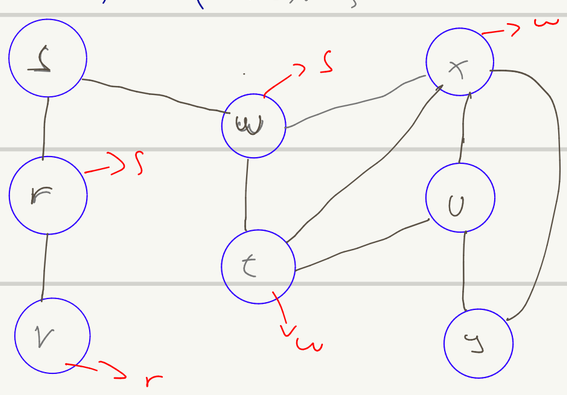
$Q = \{w\ r\}$



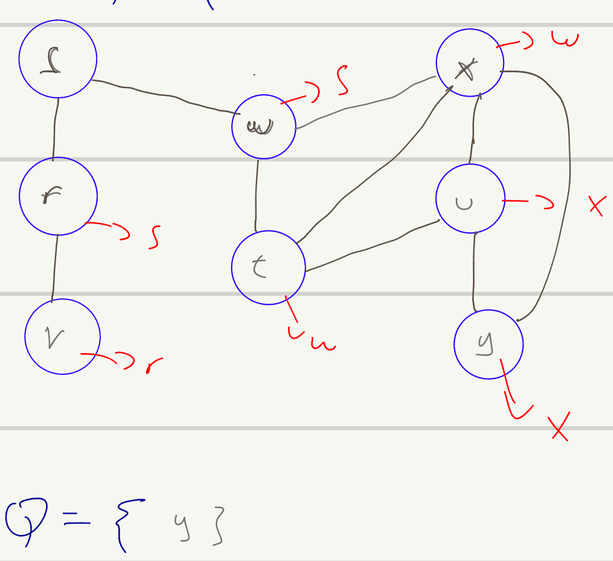
$Q = \{r\ x\ t\}$



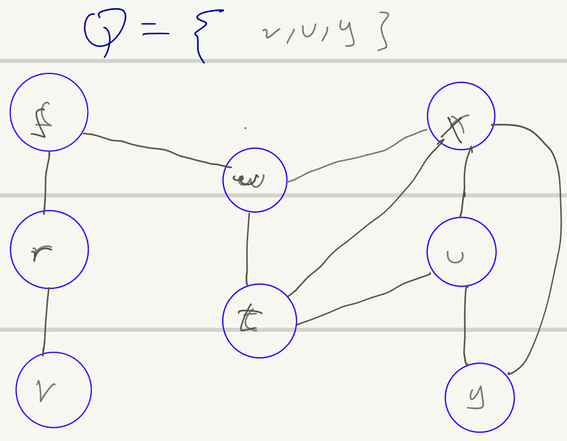
$Q = \{x, t, v\}$



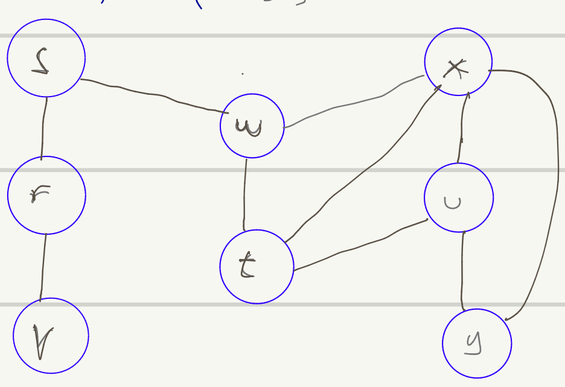
$Q = \{t, v, u, y\}$



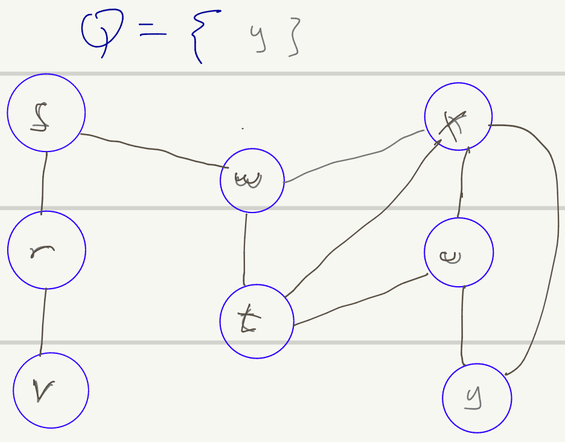
$Q = \{v, u, y\}$



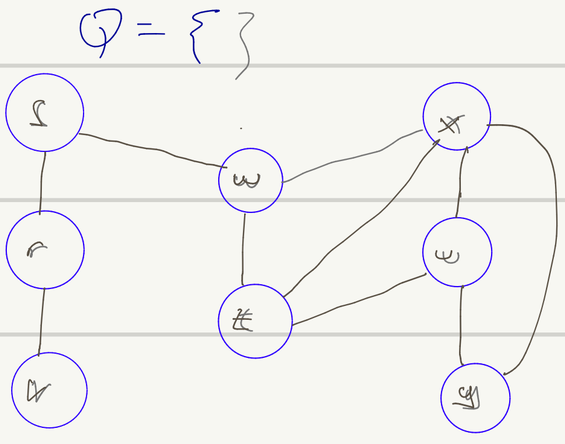
$Q = \{u, y\}$



$Q = \{y\}$

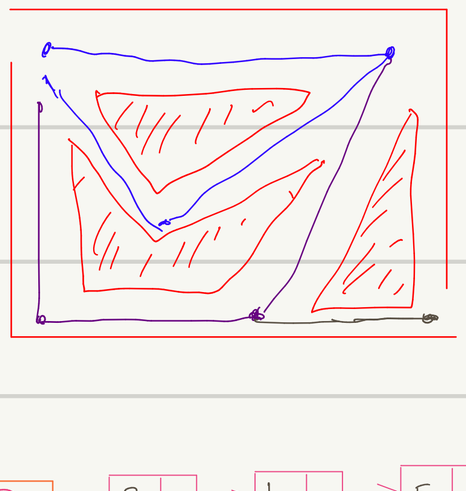


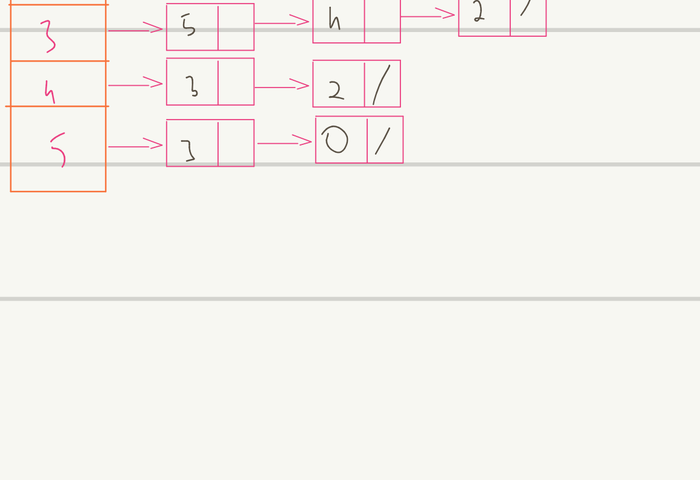
$Q = \{\}$

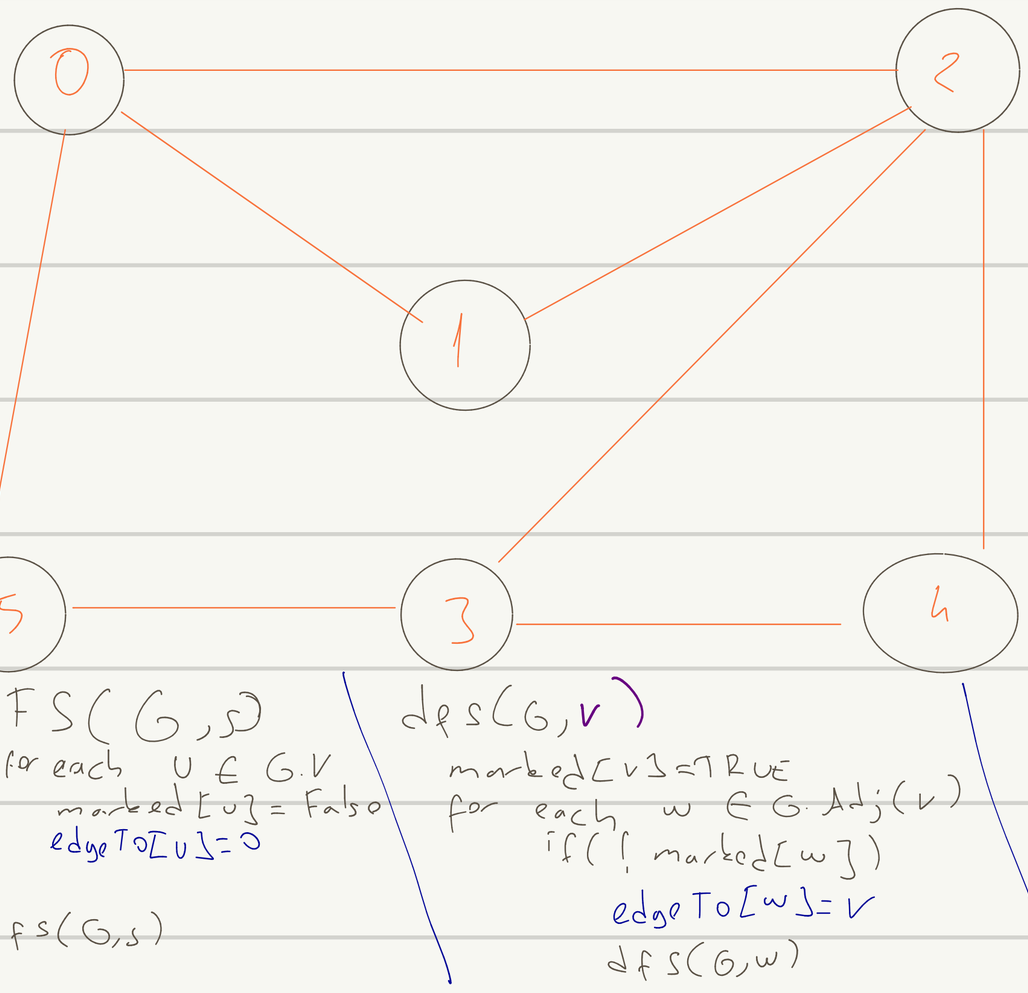


DFS - Depth first search

Labirent





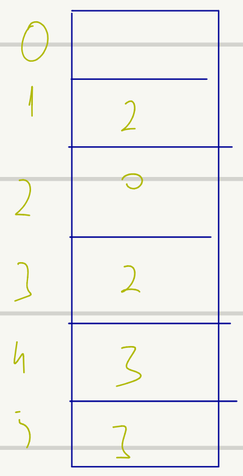



```
DFS(G, s)
  for each u ∈ G.V
      marked[u] = false
      edgeTo[u] = 0
  dfs(G, s)
```

```
dfs(G, v)
  marked[v] = TRUE
  for each w ∈ G.Adj(v)
      if(! marked[w])
          edgeTo[w] = v
          dfs(G, w)
```

```
pathTo(V)
  if(! marked[v])
      return NULL;
  for(x = v; x != s; x = edgeTo[v])
      path.push(x)
  path.push(s)
  return path
```

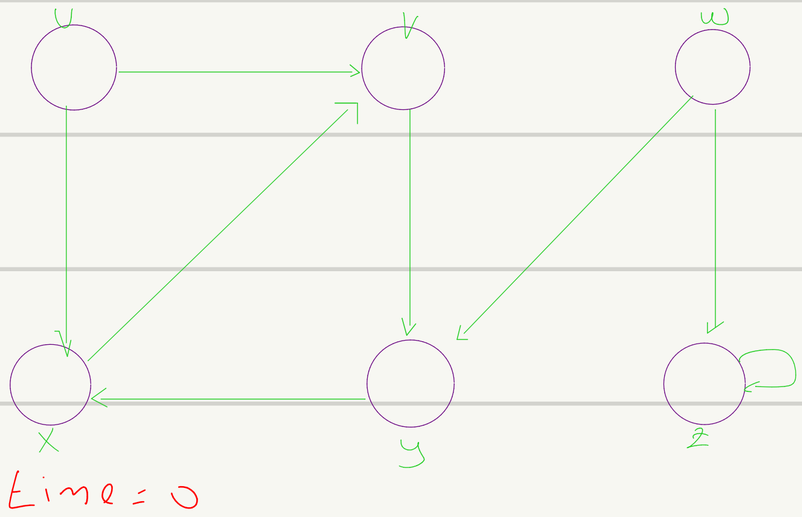
DFS → Verilen 2 düğüm bağlı mı, verilen düğümden hangi yollar var?



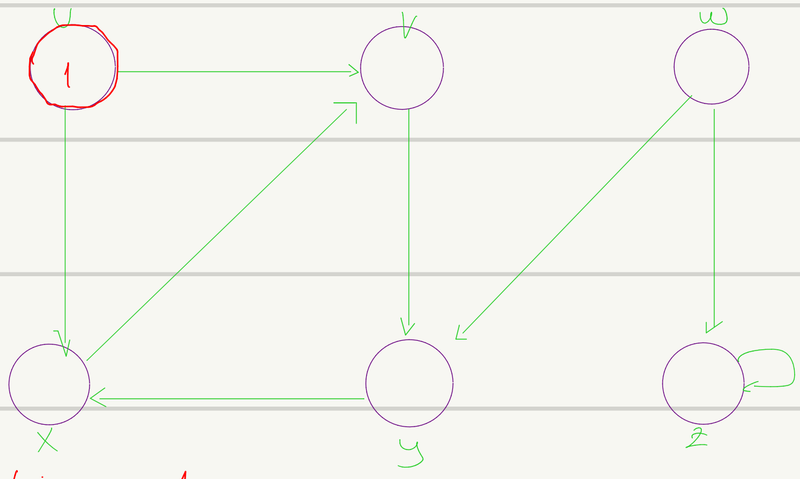
X	path
5	3 2 0
1	2 0
2	0
3	2 0
4	3 2 0

```
colored DFS (V,E)
    foreach u∈V
        color[u] ← white
time ← 0
for each u∈V
    if color[u] = white
        DFS-VISIT(u)
```

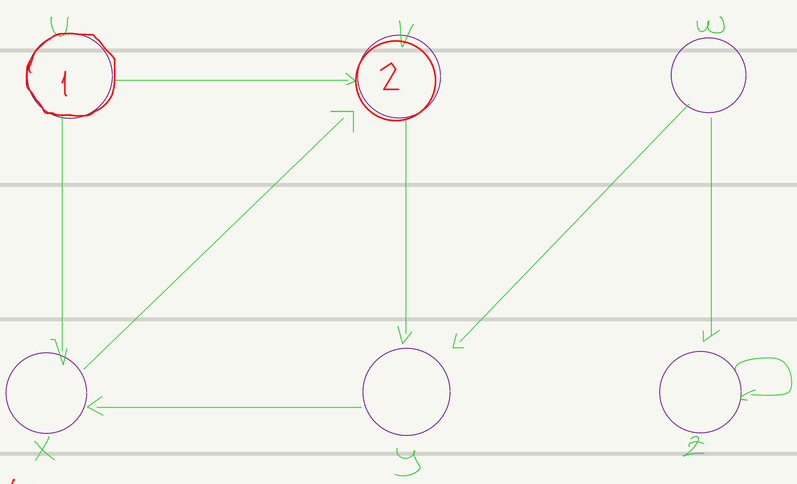
```
DFS-VISIT(u)
    color(u) ← grey
    time ← time+1
    dscur[u] = time
    foreach v∈Adj[u]
        if color[v]=white
            DFS-VISIT
    color[u] = black
    time = time+1
    fnsh[u] = time
```



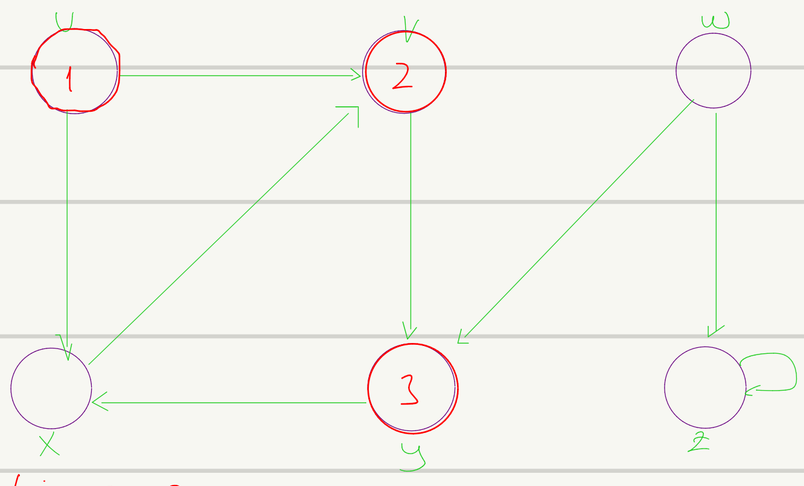
Time = 0



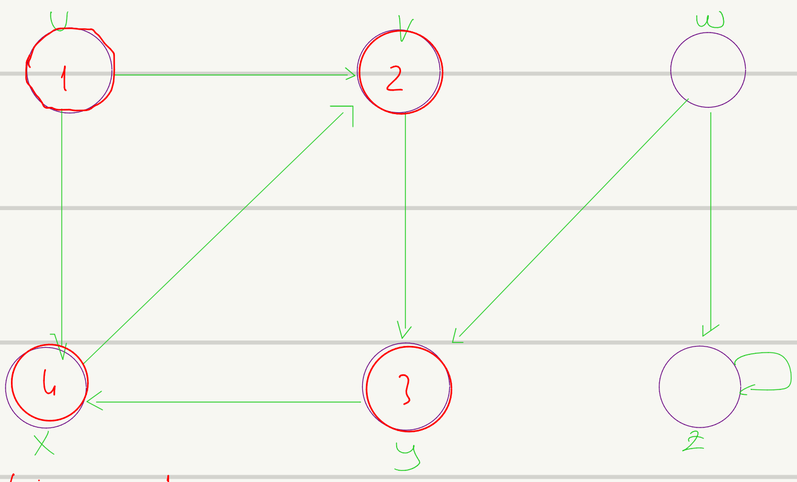
Time = 1



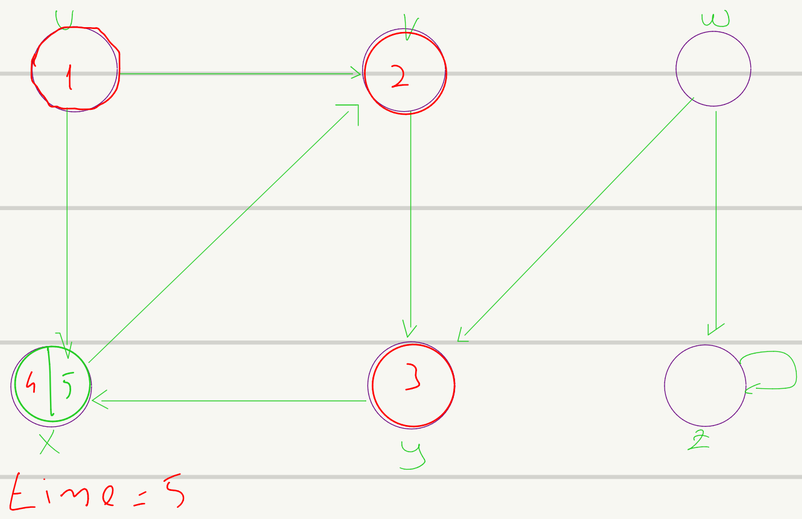
Time = 2



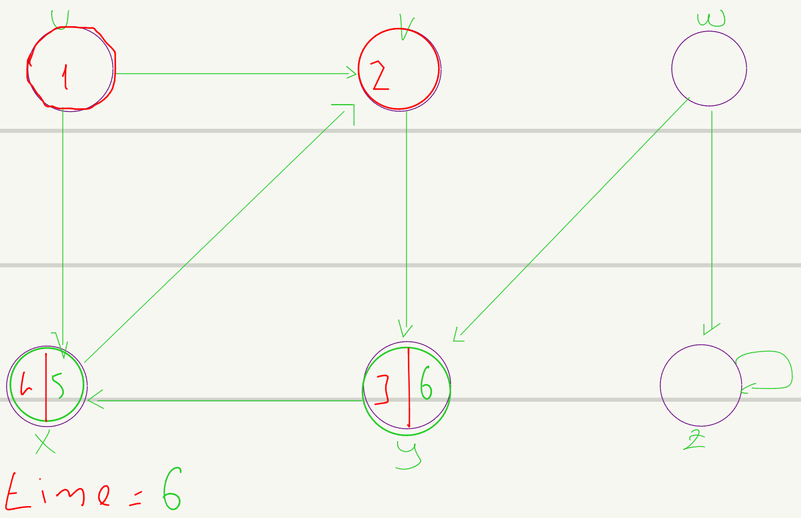
Time = 3



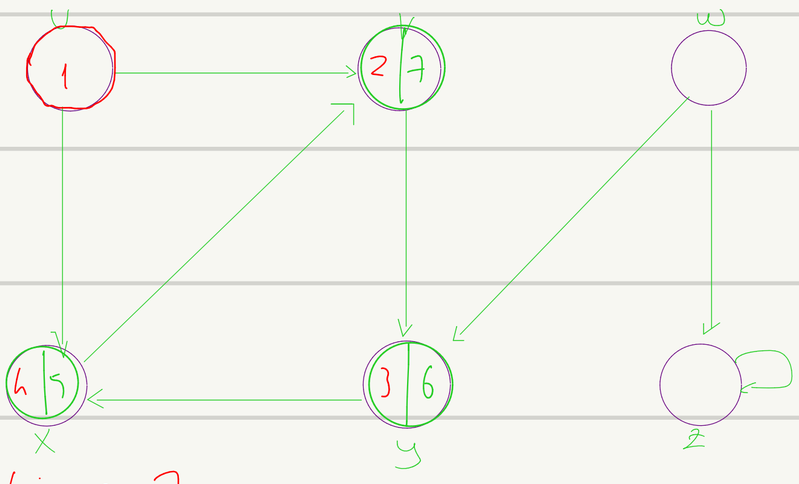
Time = 4



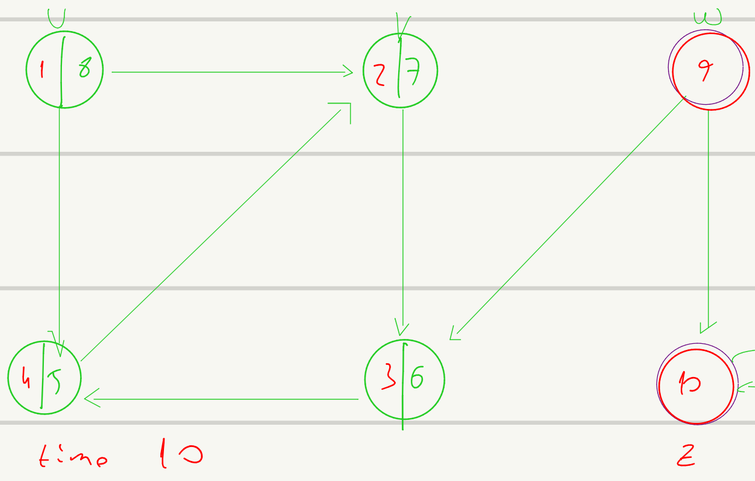
Time = 5



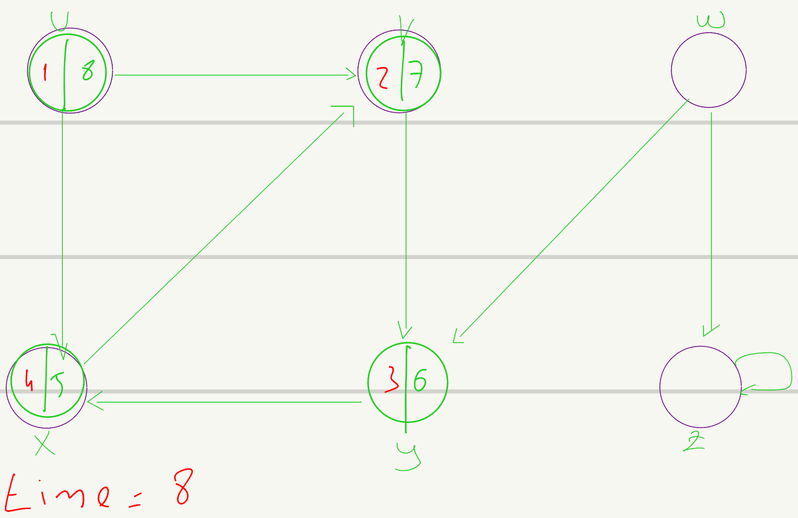
Time = 6



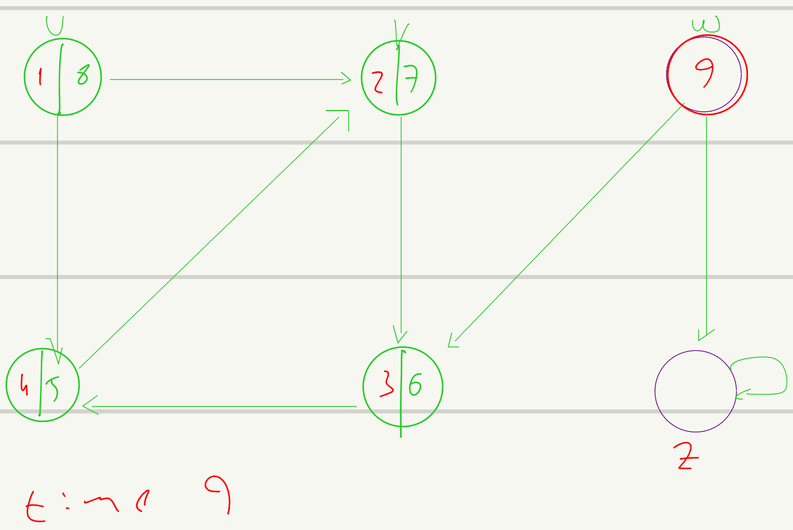
Time = 7



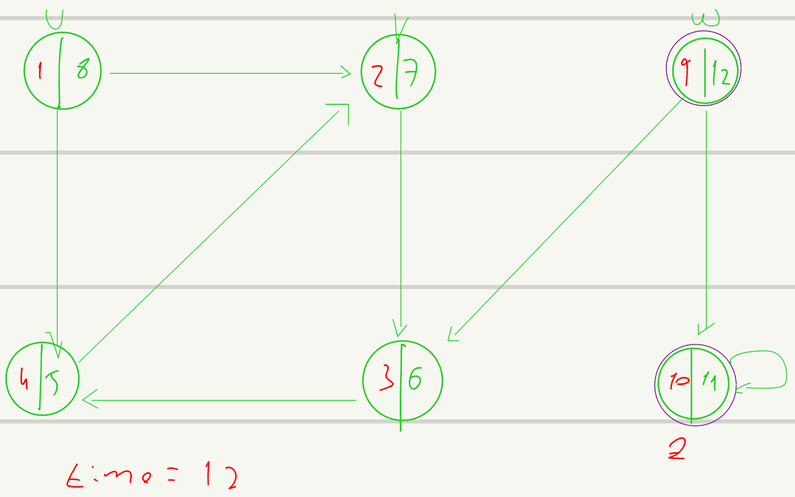
time 10



Time = 8



time 9



time = 12

(SINGLE-SOURCE) SHORTEST PATHS

G(V,E) yön (li/siz), kenar ağırlıklı graf

$w(u,v) \geq 0$ → -'lis: olursa Bellman-Ford

DİJKSTRA(V,E, $\vec{w}$ (ağırlık), s)
↳ başlangıç düğümü

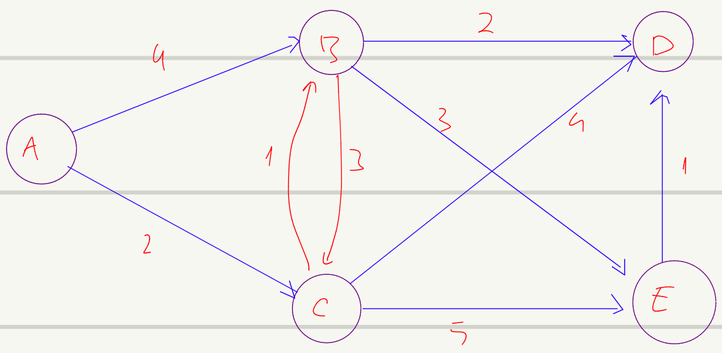
```
foreach v∈V
    dist[v]=∞
    prnt[v]=NULL
dist[s]←0
S←∅
PQ←V
```

```
while PQ≠∅ → eleman çek
u←Extract_Min
S←S∪{u}
foreach v∈Adj[u]
    RELAX(u,v,w) → büyüklük karşılaştırması yapıyor güncelleme gerekiyor mu bakıyor
```

```
RELAX(u,v,w)
  if dist[v]>dist[u]+w(u,v)
    dist[v]←dist[u]+w(u,v)
    parent[v]←u
```

50 - 8 - (7)

Karmaşıklık → PQ minheap'te olursa O(ElogV)



	A	B	C	D	E	
1)	0	∞	∞	∞	∞	Min → A
2)	0	4(A)	2(A)	∞	∞	Min → C
3)	0	3(AC)	2(A)	6(AC)	7(AC)	Min → B
4)	0	3(AC)	2(A)	5(ACB)	6(ACB)	Min → D
5)	0	3(AC)	2(A)	5(ACD)	6(ACB)	Min → E
6)	0	3(AC)	2(A)	5(ACB)	6(ACB)	PQ → ∅

23.05.22
15

(SUB)STRING ARAMA

Text[N] <=> PATTERN[M]

N > M

web aramaları, veritabanı aramaları, spam filtreleri;

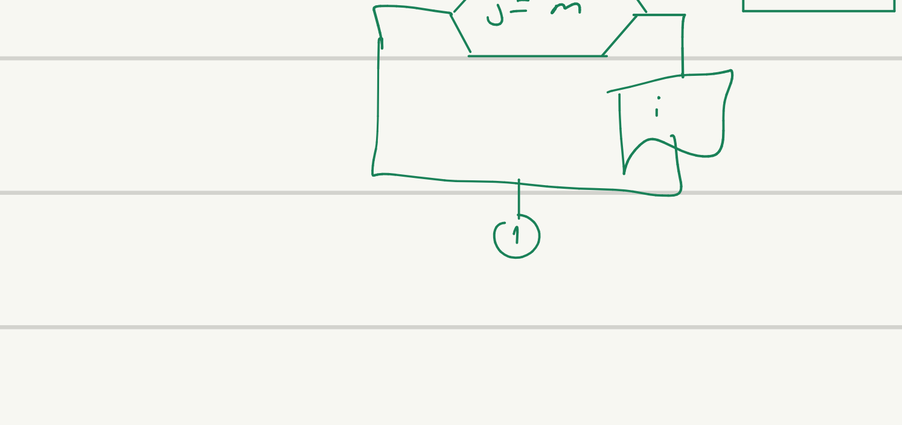
1-) Brute-Force (Naive)

T = a c a a b c

P = a a b


```
for    S = 0   to   n-m
    if   P[1..m] = T[s+1..s+m]
        print(found at S)
```

} O(NM)



kaç maçlık

2-) Horspool Algoritması (Boyer-Moore / Boyer-Moore Horspool)

Bad Character Heuristic

1-) Texttek: karakter patternde yok

T -> HELLOST
P -> BARBERA

-> M kaydırım -> HELLOST
BARBERA

2-) Karakter Var ama Sen karakter değil

T -> _ _ _ _ _ _ B -> B
BARBER BARBER

3-) Sondan eşleşme var ve sonrasında olmayan karakter geliyor. Aynı zamanda pattern kendini tekrarlamıyor.

T -> _ _ _ MER _ _ _ -> MER
LEADER LEADER

4-) 3. durumun aynısı ama kelime kendini tekrarlıyor.

T -> _ _ _ AR -> AR
REORDER REORDER

Shift table -> kaydırma miktarı

0 1 2 3 4
P = T O O T H

T	O	H	*
5	5	5	5

-> diğer durumlar

5-0-1
5-1-1
5-2-1
5-3-1

T	O	H	*
1	2	5	5

```
ShiftTable (P[0.. M-1])
    for i = 0 to size-1
      do Table[i] = m
    for j = 0 to m-2
      do Table[P[j]] = m-1-j
    return table
```

TRUST_HARD_TOOTHBRUSHES
TOOTH
TOOTH
TOOTH
TOOTH ✓

$\approx O(N)$ en kötü durumda $O(NM)$

```
Horspool Matching(P[0...M-1], T[0...N-1])
    ShiftTable(P[0..M-1])
    i = m-1
    while(i ≤ n-1) do
        k=0
        while(k ≤ m and P[m-k-1]=T[i-k] do
            k=k+1
        if(k=m)
            return i-m+1
        else
            i=i+Table[T[i]]
    return -1
```

P = BARBER

	A	B	C .... E .... R .. Z
Table	4	2	6 6 6 1 6 6 3 6 6 6

JIM_SAW_ME_IN_A_BARBERSHOP
BARBER

BARBER
BARBER
BARBER
BARBER
BARBER

BARBER ✓

3-) Boger Moore algoritması

T → .. SER
P → BARBER
BARBER

T → ... AER
P → BARBER

$d_1 = max(t_1(c) - k, 1)$
$d_2 =$ Sher karakter için

diğer tablo örnek

pattern	d_2
ABCBAB	← kendini sondan hiç tekrar etmiyorsa d_2 nin eşit değeri 1 alır
1 ABCBAB	2
2 ABCBAB	
3 ABCBAB	
4 ABCBAB	
5 ABCBAB	

4) max 4) max 4) max

$$d = \begin{cases} d_1 & \text{if } k=0 \\ d_1 & \text{if } d_1 > d_2 \\ d_2 & \text{if } d_2 \geq d_1 \end{cases}$$

P = BAOBAB

T = BESS_KNEW_ABOUT_BAOBABS
BAOBAB → d_1

BAOBAB → d_2 ($d_1 = 6-1-k = 4$, $d_2 = 5$)

BAOBAB → d_1 ($d_1 = 5$, $d_2 = 2$)
ISAOBAB ✓

Bad MAKTABLE

A	B	O	K
1	2	3	6

Good Suffix Table

1	2	3	4	5
2	5	5	5	5

4-) Rabin - KARP Hash kullanılır

T = 31415926535 8979...
P = 59265

Hash → 59265 mod 97 = 95

mod 97 = 94 → 18
31415 92 65 35 8979...
mod 97 → 76 → 59 = 95 → = 59265 → ✓
= 84 ≠ 95

mod alınada kolaylık var

31415 mod 97 = 84
(31415 - 30000) * 10 + 9 = 14159

14159 mod 97 = (31415 mod 97 - 30000 mod 97) * 10 + 9 mod 97

10000 mod 97 = 9
bunu bilip sabit gibi kullanırız

(87 - 9*3)*10+9 → 579 mod 97

Sıralama

I: {a, b, c ... z}

O: {a', b' ... z'} $a' \leq b' \ldots \leq z'$

Tercih kriterleri

1-) Performans

- Karşılaştırma / Yer değiştirme miktarı
- Hız

2-) Bellek kullanımı

- In place → ekstra bellek kullanmaz
- Out-of-place → ekstra bellek kullanır.

3. Tekrar eden değerler

- Stable → girdideki sıraya aynı değerleri çıktıda veriyorsa 3 3 3
- Unstable → " " " " " " veriyorsa 3 3 3

Temel yöntemler

Brute force	Decrease and Conquer	Transform and Conquer
• Bubble Sort	• Insertion sort	• Heap Sort
• Shell sort		
• Selection Sort		

• Divide and Conquer → Simülasyon yok

Counting Sort

0 1 2 3 4 5 6

1	3	2	1	1	3	1

• Merge Sort

• Quick Sort

→ Bir dizide çok benzer elemanlar varsa işlemsel.

0 1 2 3 → max eleman

0	1	1	1

kümülatif toplam

0 1 2 3

0	4	1	2

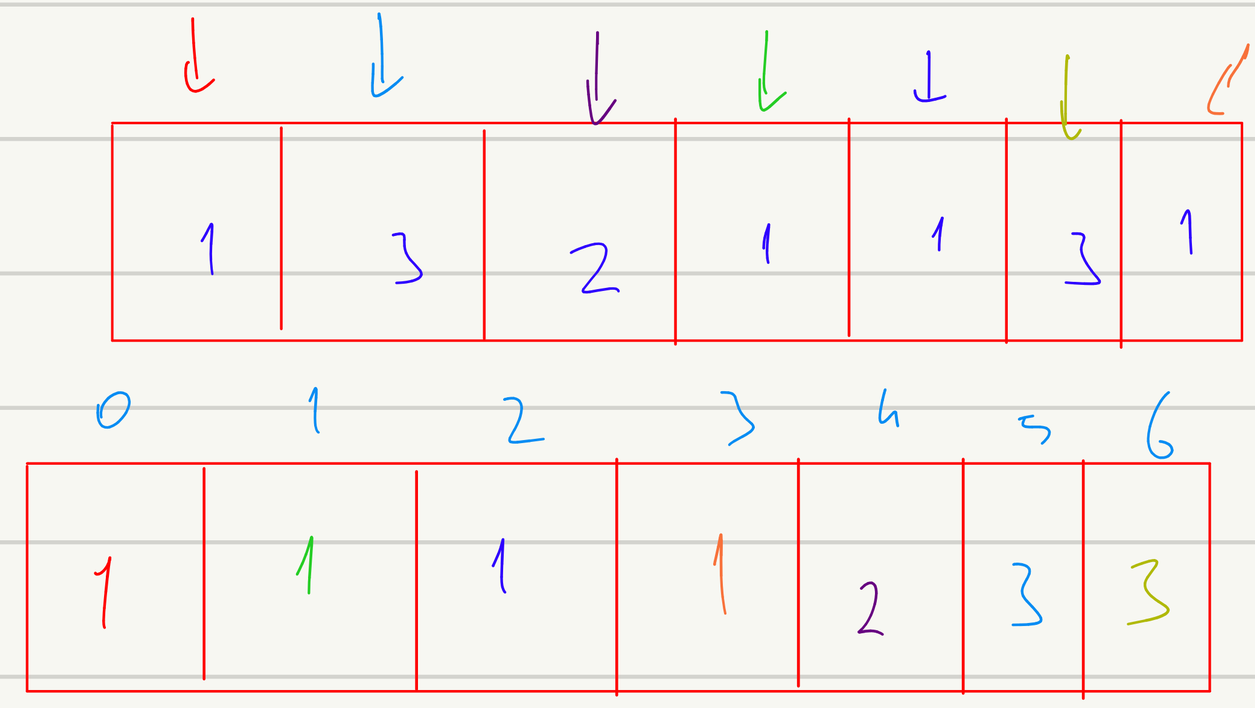
→

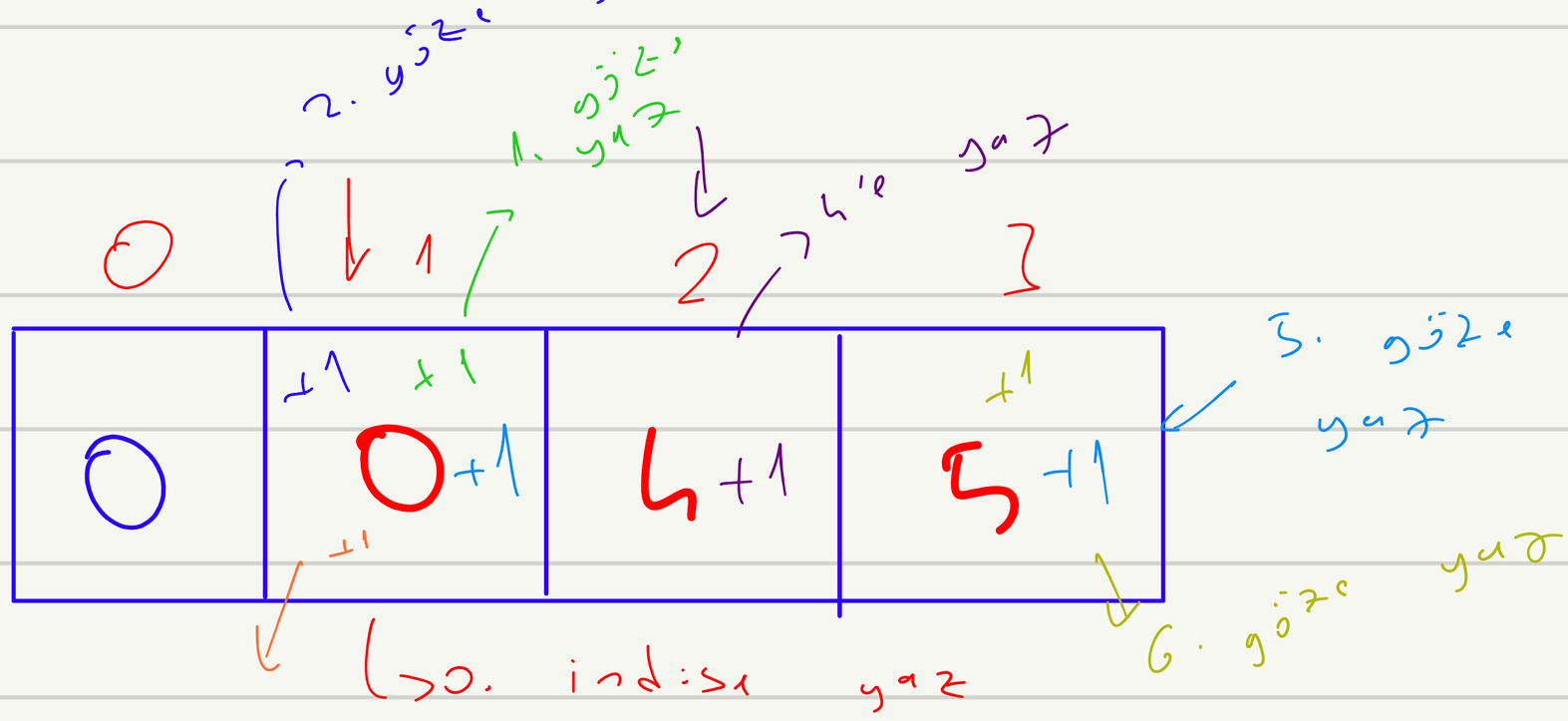
0 1 2 3

0	4	5	7

0→0 4→1 5→2

0	0	4	5





62	31	84	96	19	47

Comperating Counting Sort (A[0..n-1])

```
for i<-0 to n-1
    do count[i]<-0

for i<-0 to n-2 do
    for j<-i+1 to n-1 do
        if A[i]<A[j]
            count[j]<-count[j]+1
        else
            count[j]<-count[i]+1

for i<-0 to n-1
    do S[count[i]]<-A[i]
return S
```

count	0	0	0	0	0	0
	1	0	1	1	0	0
	3	1	2	2	0	1
	3	1	4	3	0	1
	3	1	4	5	0	1
	3	1	4	5	0	2
	1. indis		4.	5.	0.	2.

Radix Sort

↳ LSD → least Significant Digit

↳ MSD Radix Sort

kısının de

Lexicology

$O(|R| + m + \alpha)$

|R| → String sayısı; m → String uzunluğu; α → alphabet

LSD Radix Sort (R)

for i ← m-1 to 0 do

countingSort(R, i)

return R

cat
him
ham
bat

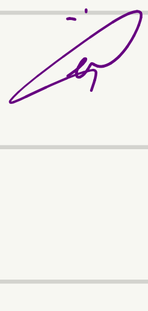
→ Stable olduğu için

Sıralı kalır

Counting sort

→

him → ham → bat
ham → cat → cat
cat → bat → hat
bat → him → him



0300
0841
0012
2023
1154

→

0300
0841
0012
2023
1154

→

0300
0012
2023
0841
1154

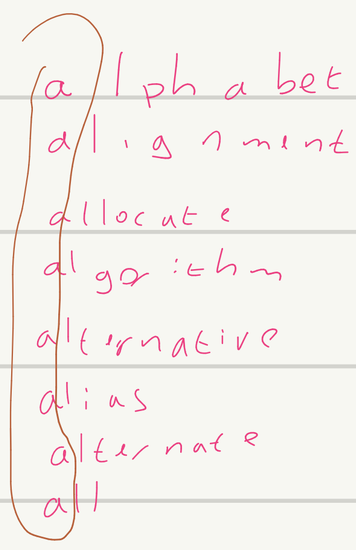
→

0012
2023
1154
0300
0841

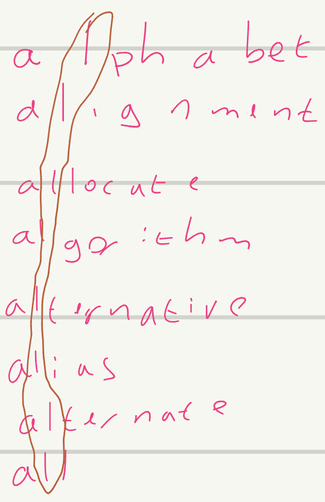
→

0012
0300
0841
1154
2023

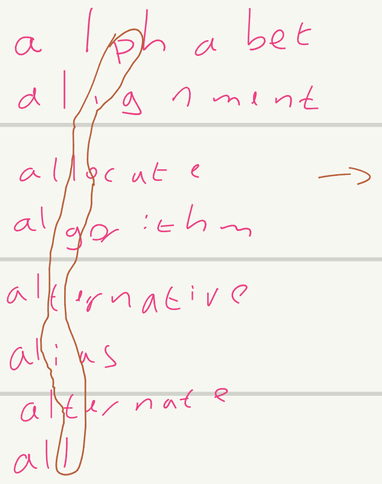
MSD Radix Sort



alphabet
alignment
allocate
algorithm
alternative
alias
alternate
all



alphabet
alignment
allocate
algorithm
alternative
alias
alternate
all



alphabet
alignment
allocate
algorithm
alternative
alias
alternate
all

→

algorithm
alignment
alias
allocate
all
alphabet
alternative
alternate

algorithm
alignment
alias
all
allocate
alphabet
alternative
alternate

alias
alignment
all
allocate
alphabet
alternative
alternate

→

alias
alignment
all
allocate
alphabet
alternate
alternative

she
sells
seashells
by
the
she
shore
the
shells
she
sells
are
surely
seashells

→

are
by
she
sells
seashells
she
shore
shells
she
sells
surely
seashells
the
the

Alg	Ort kurmaş.	Stable	Σn place	Ekstra alan
Selection Sort	$O(n^2)$	Hayır	✓	1
Insertion Sort	$O(n^2)$	Evet	✓	1
Shell	$O(n^2)$	✗	✓	1
Merge	$O(n \log n)$	✓	✗	N
Quick	$O(n \log n)$	✗	✓	$c \log N$
Radix LSD	$O(n)$	✓	✗	$N+k$
Radix MSD	$O(n)$	✓	✗	$N+Dk$ (D: stack)
Heap Sort	$O(n \log n)$	✗	✓	1